From Spark to Flame: Neural Networks That Learn

Danna Gurari
University of Colorado Boulder
Spring 2026

CC BY NC SA

https://dannagurari.colorado.edu/course/neural-networks-and-deep-learning-spring-2026/

Review

- Last lecture:
  - Biological inspiration (late 1800s & early 1900s): discovery of neurons
  - Biological modeling (1957): mimicking “intelligence” to understand it
  - Crisis (1969): models proven incapable of many forms of “intelligence”
  - Programming tutorial: basics of PyTorch
- Assignments (Canvas):
  - Lab assignment 1 due in one week
- Questions?

Today's Topics

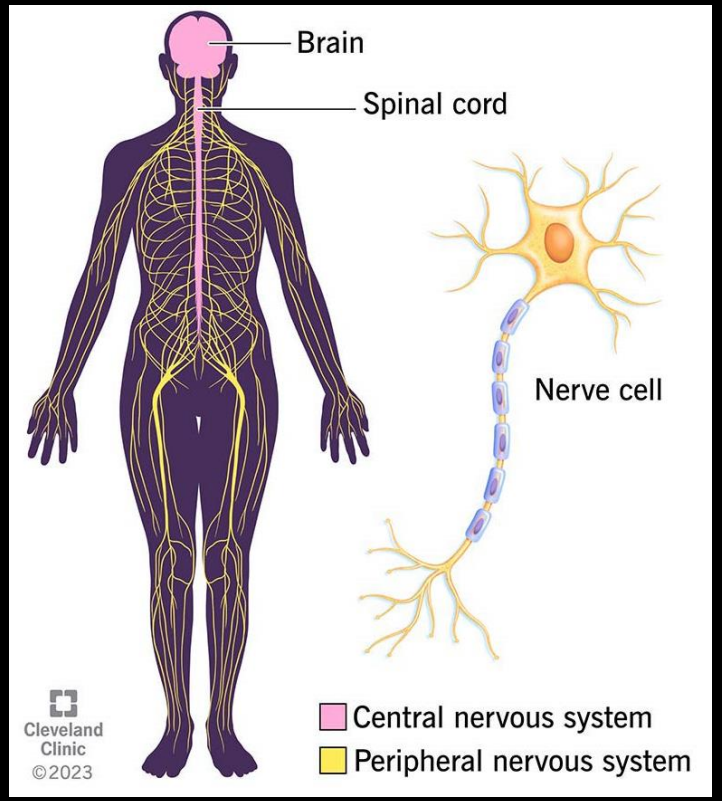
- Vision: multi-layer networks provide sufficient representational power
- Breakthrough: gradients make multi-layer networks trainable
- Machine abstraction: neural networks as computational graphs
- Programming tutorial: developing neural networks in PyTorch

Today's Topics

- Vision: multi-layer networks provide sufficient representational power
- Breakthrough: gradients make multi-layer networks trainable
- Machine abstraction: neural networks as computational graphs
- Programming tutorial: developing neural networks in PyTorch

Why the Perceptron's Spark Survived

Insufficient single-layer networks were just a stepping stone towards multi-layer networks; e.g.,



https://www.britannica.com/science/human-nervous-system

Humans: ~100,000,000,000 neurons



Why the Perceptron's Spark Survived

Insufficient single-layer networks were just a stepping stone towards multi-layer networks; e.g.,

A Review of "Perceptrons:
An Introduction to Computational Geometry"

by Marvin Minsky and Seymour Papert.

The M.I.T. Press, Cambridge, Mass., 1969.
112 pages. Price: Hardcover $12.00; Paperback $4.95.

H. D. Block

Department of Theoretical and Applied Mechanics, Cornell University, Ithaca, New York 14850

(1970)

"… the simple Perceptron was studied first… because it showed that adaptive neural nets, in their simplest forms, could, in principle, improve. This suggested that more complicated networks might exhibit more interesting behavior… what the Perceptronists took to be a temporary handhold, Minsky and Papert interpret as the final structure."

- pg. 514

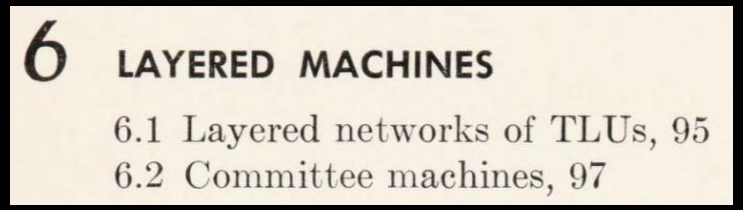


Why the Perceptron's Spark Survived

Insufficient single-layer networks were just a stepping stone towards multi-layer networks; e.g.,

Hawkin's IRE paper

"[limitations of single-layer networks] are extremely severe. . . [using] multiple-layer logical circuit arrangements… any function can thereby be realized."



Nilsson's Learning Machines book

1961 — 1962 — 1965 — 1969

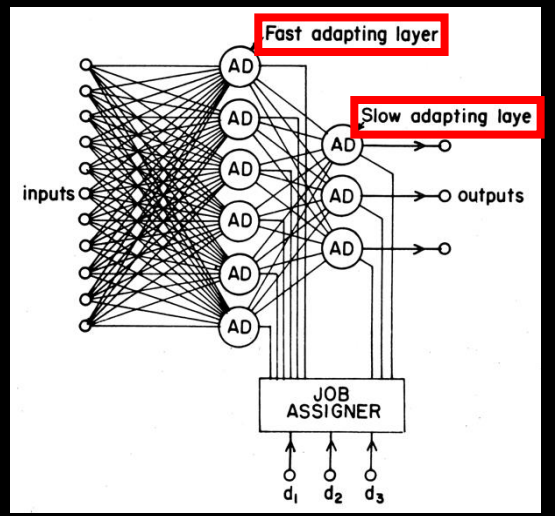
Rosenblatt's Neurodynamics book

"The addition of a fourth layer of signal transmission units… permits the solution of generalization problems, over arbitrary transformation groups."



Widrow's Self-Organizing Systems paper

2 hidden layers →



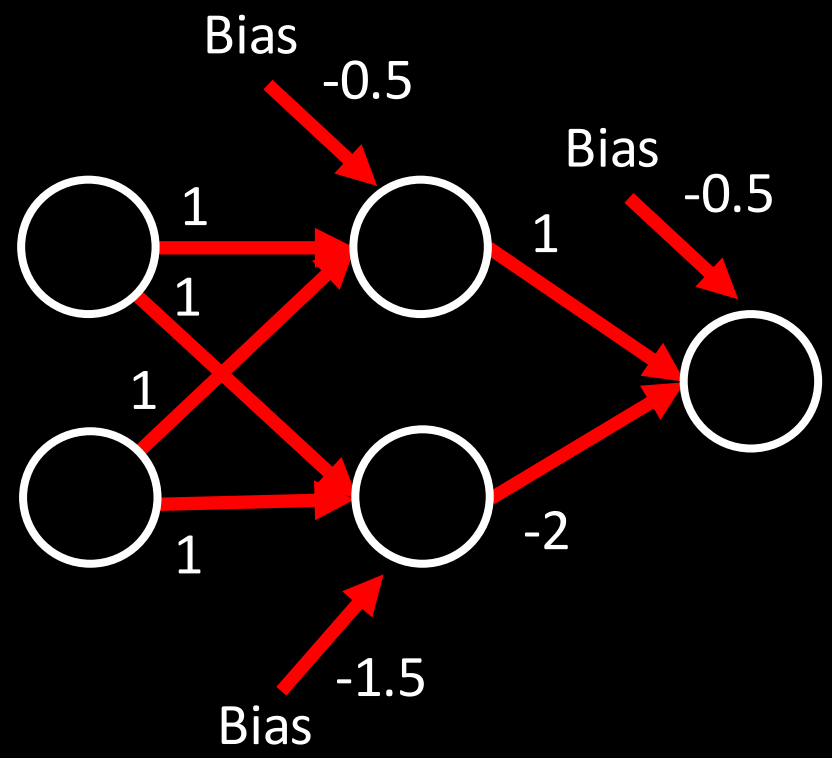
Minsky's and Papert's book

Why the Perceptron's Spark Survived

e.g., solving XOR problem with a multi-layer network and thresholding (0 if less than 0; 1 otherwise)

Input		Output
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



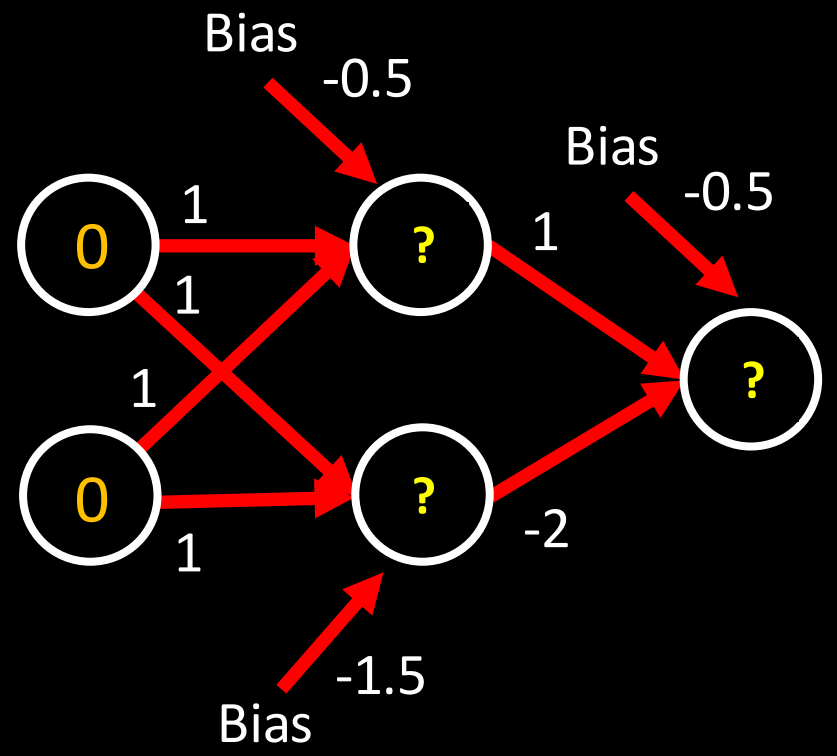


Why the Perceptron's Spark Survived

e.g., solving XOR problem with a multi-layer network and thresholding (0 if less than 0; 1 otherwise)

Input		Output
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



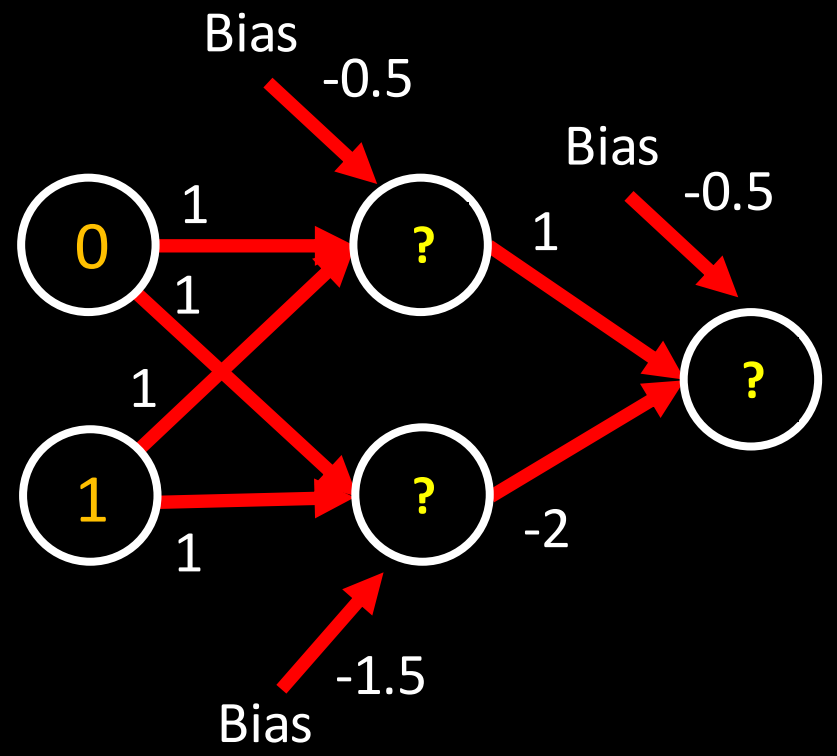


Why the Perceptron's Spark Survived

e.g., solving XOR problem with a multi-layer network and thresholding (0 if less than 0; 1 otherwise)

Input		Output
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



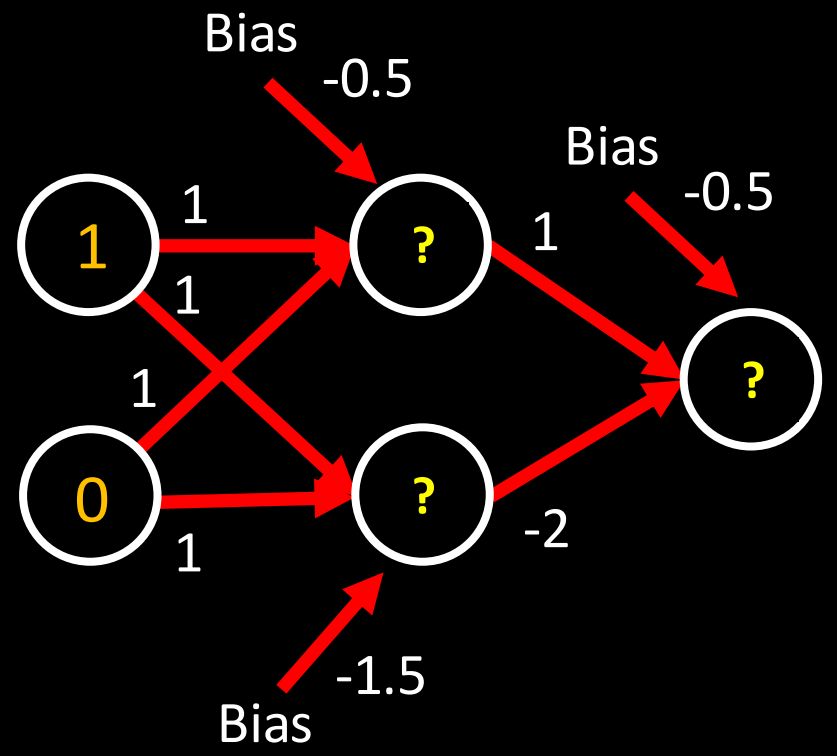


Why the Perceptron's Spark Survived

e.g., solving XOR problem with a multi-layer network and thresholding (0 if less than 0; 1 otherwise)

Input		Output
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



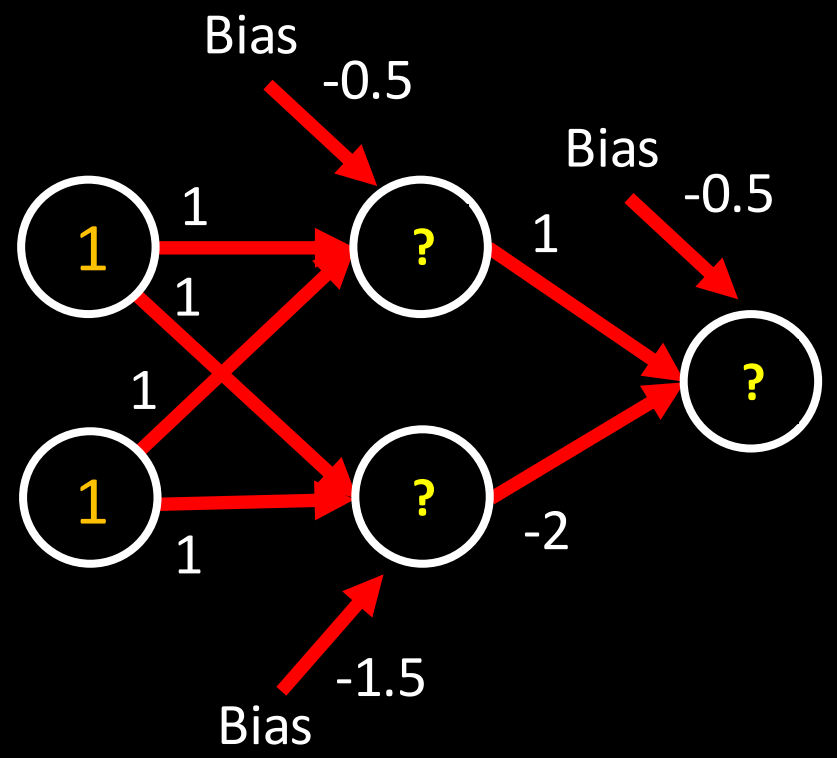


Why the Perceptron's Spark Survived

e.g., solving XOR problem with a multi-layer network and thresholding (0 if less than 0; 1 otherwise)

Input		Output
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0



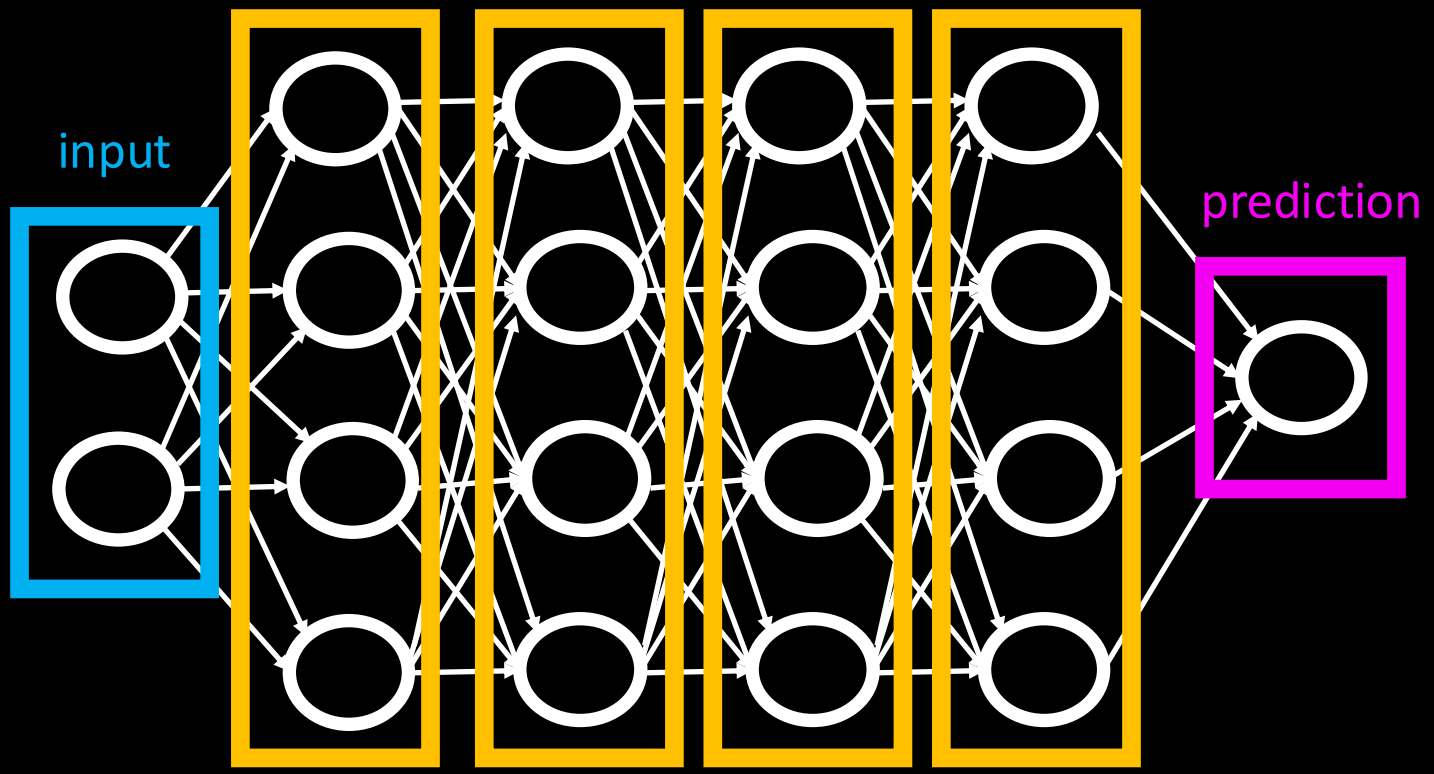


Why the Perceptron’s Spark Survived

Popular terms for multi-layer models: artificial neural networks, connectionist networks, parallel distributed systems, and neural computing systems



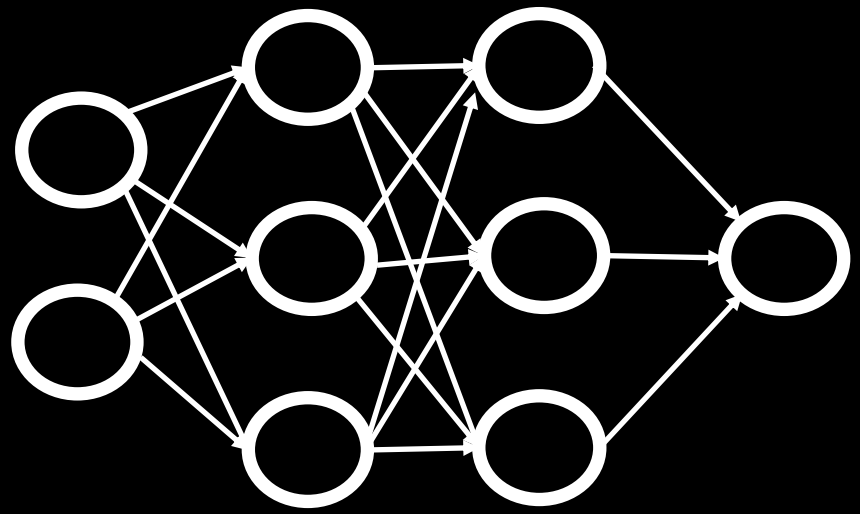
Multi-Layer Network Basics



Called “hidden” because learning determines how to use these layers to transform the input into features useful for prediction!

This is a 5-layer neural network (i.e., count number of hidden layers plus output layer) with four neurons per hidden layer

Multi-Layer Network Basics



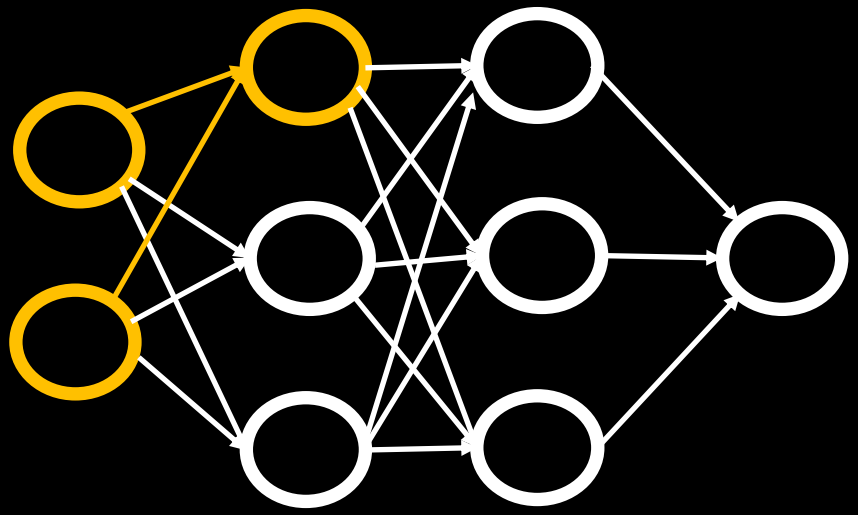
How many layers are in this network?

- 3

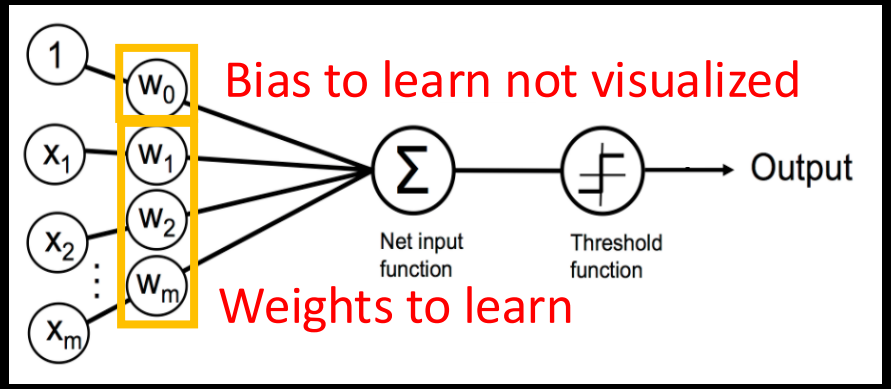
How many neurons are in each hidden layer?

- 3

Multi-Layer Network Basics

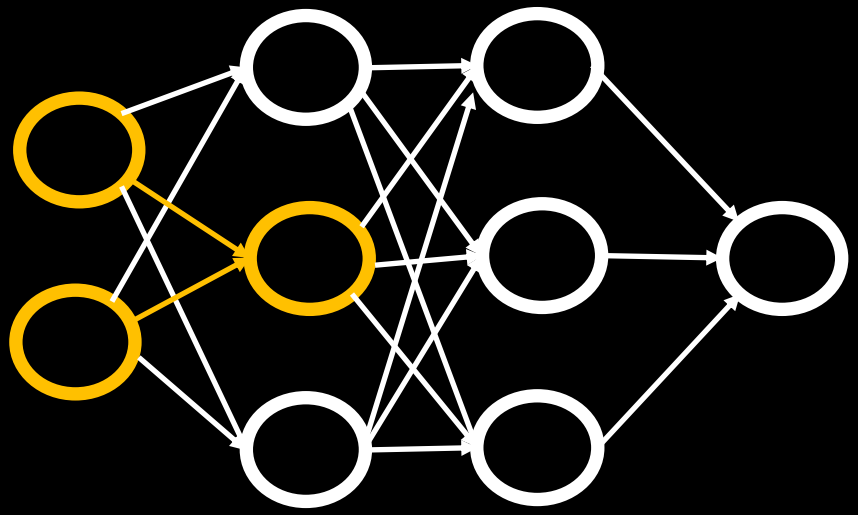


How this relates to a perceptron:

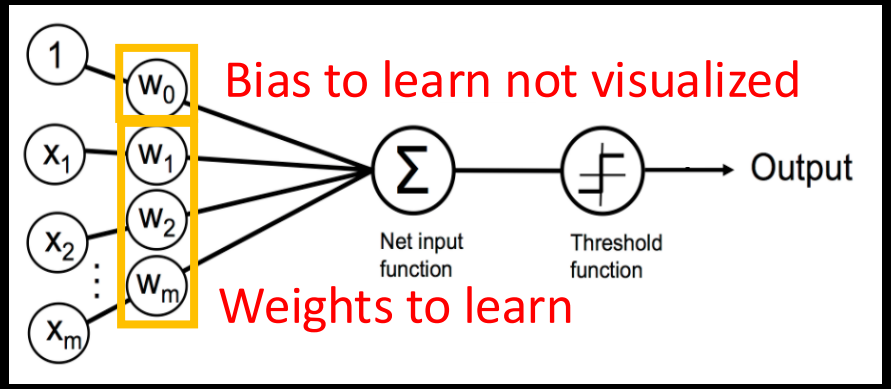


Python Machine Learning; Raschka & Mirjalili

Multi-Layer Network Basics

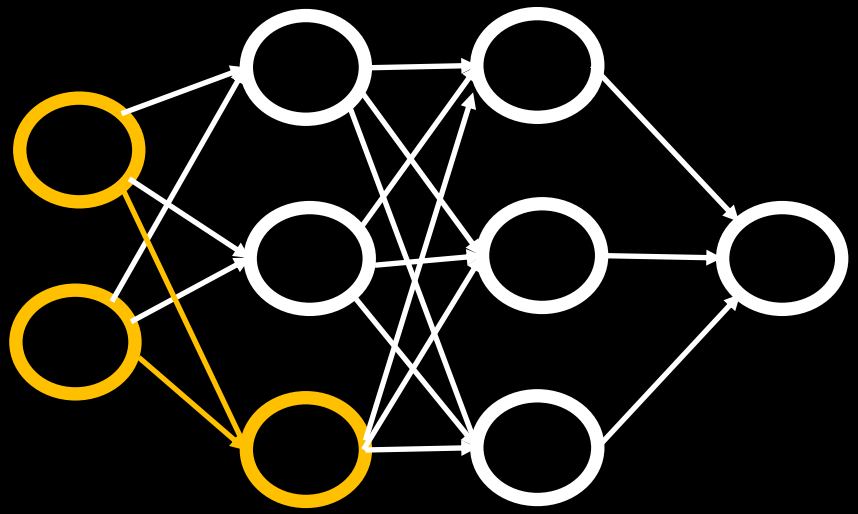


How this relates to a perceptron:

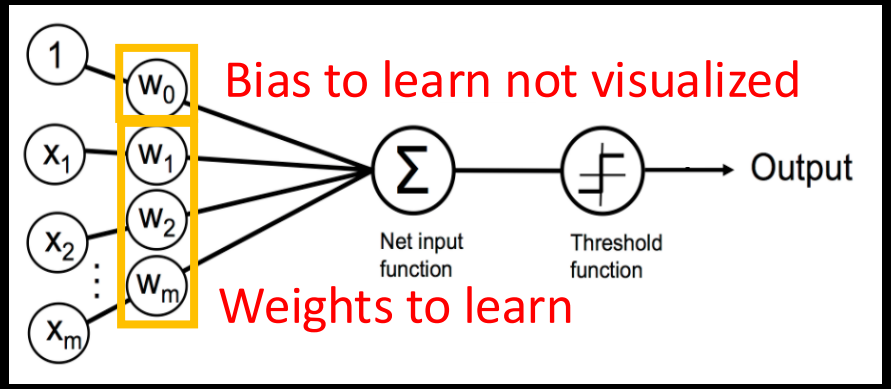


Python Machine Learning; Raschka & Mirjalili

Multi-Layer Network Basics

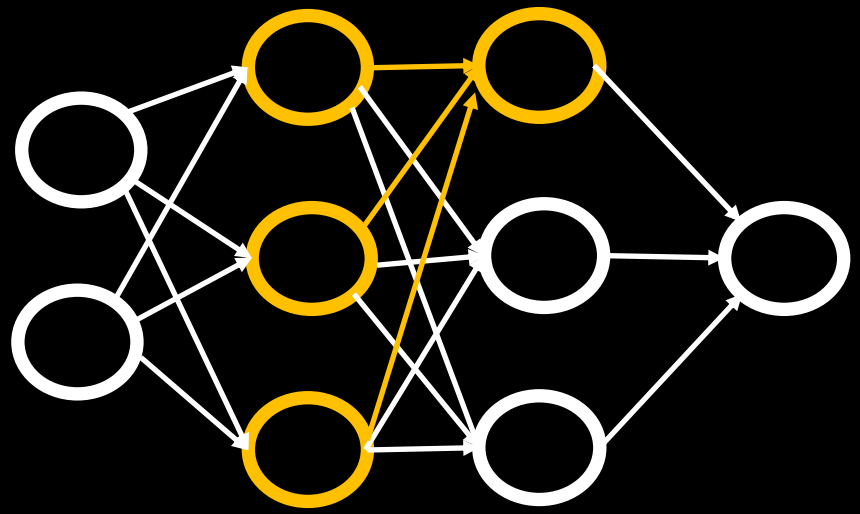


How this relates to a perceptron:

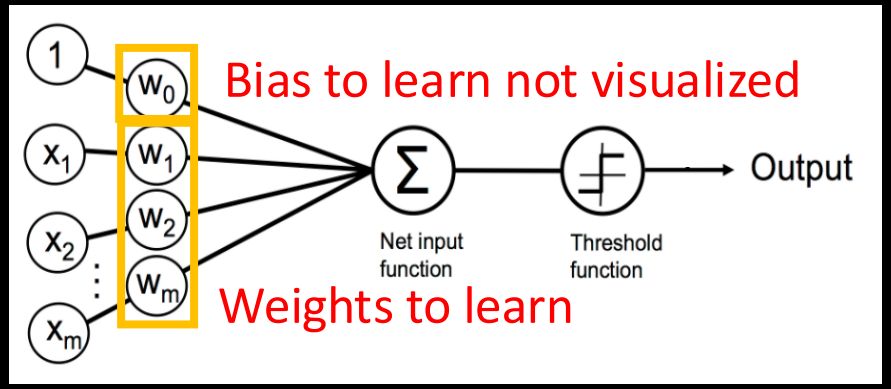


Python Machine Learning; Raschka & Mirjalili

Multi-Layer Network Basics

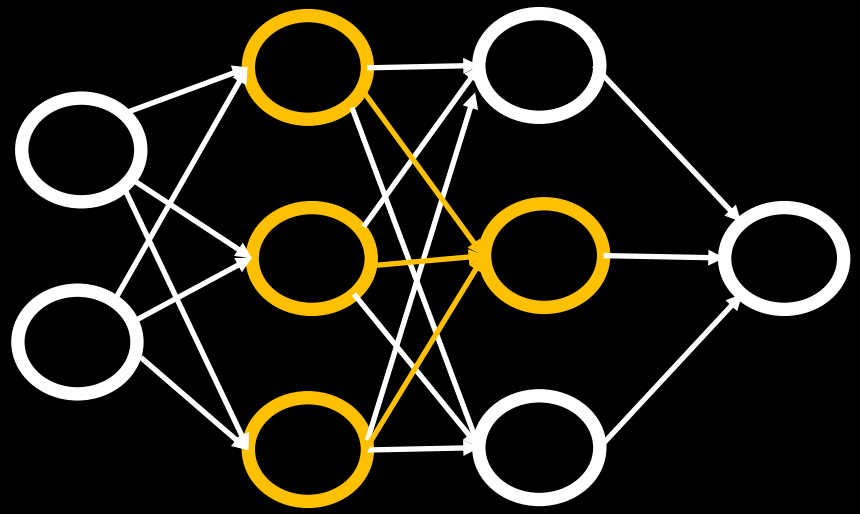


How this relates to a perceptron:

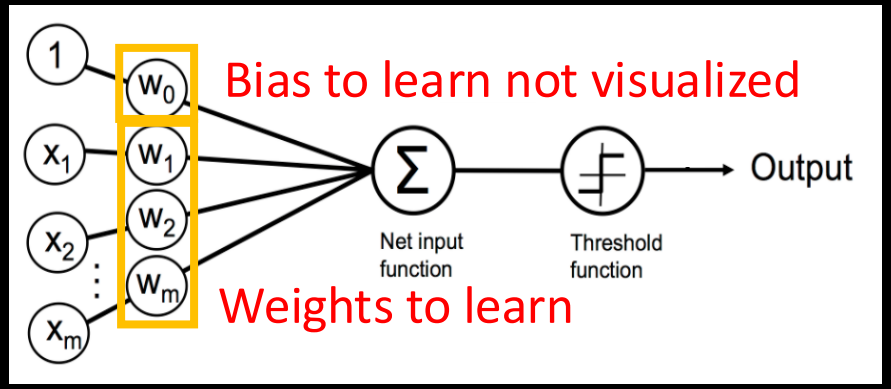


Python Machine Learning; Raschka & Mirjalili

Multi-Layer Network Basics

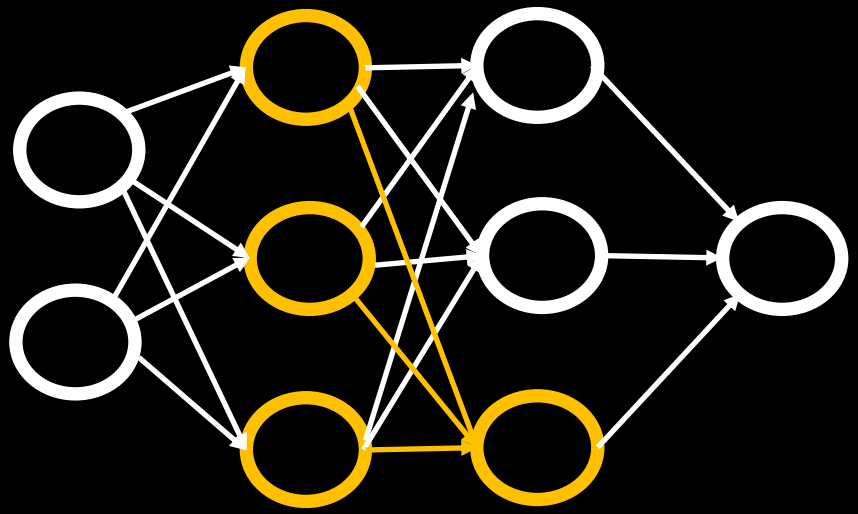


How this relates to a perceptron:

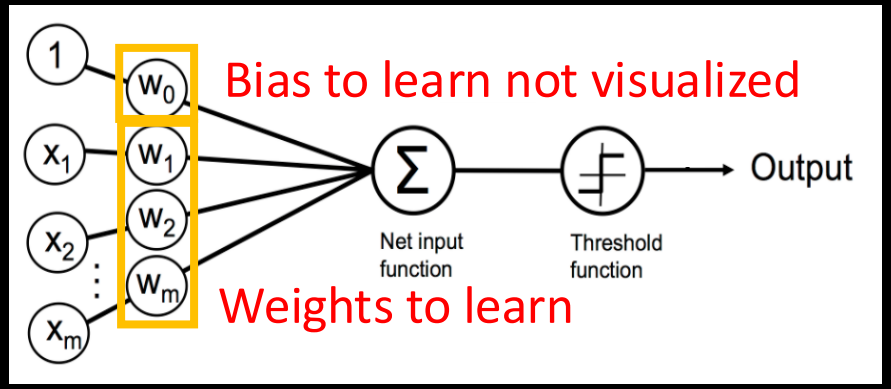


Python Machine Learning; Raschka & Mirjalili

Multi-Layer Network Basics

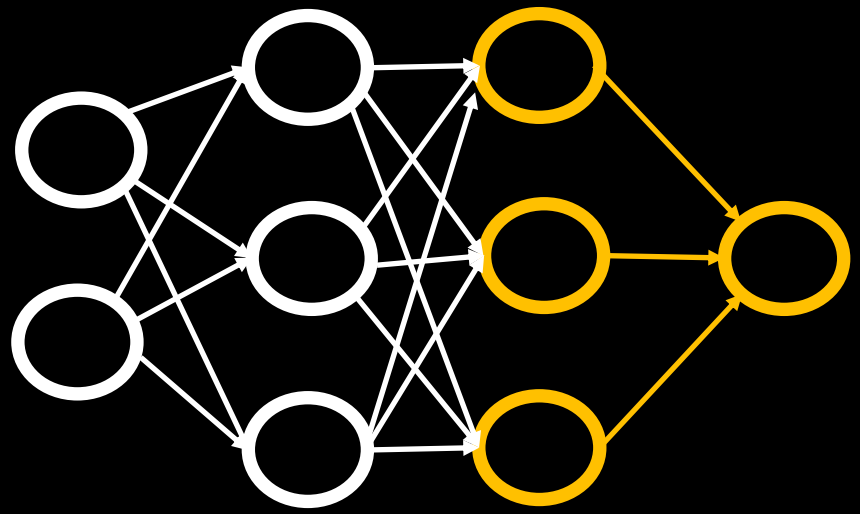


How this relates to a perceptron:

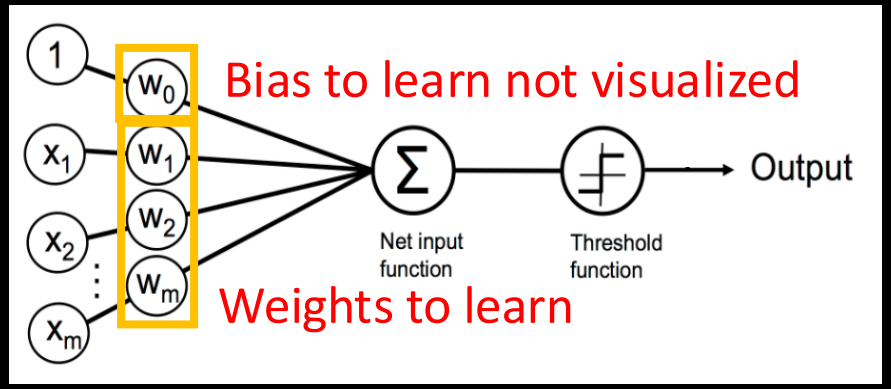


Python Machine Learning; Raschka & Mirjalili

Multi-Layer Network Basics

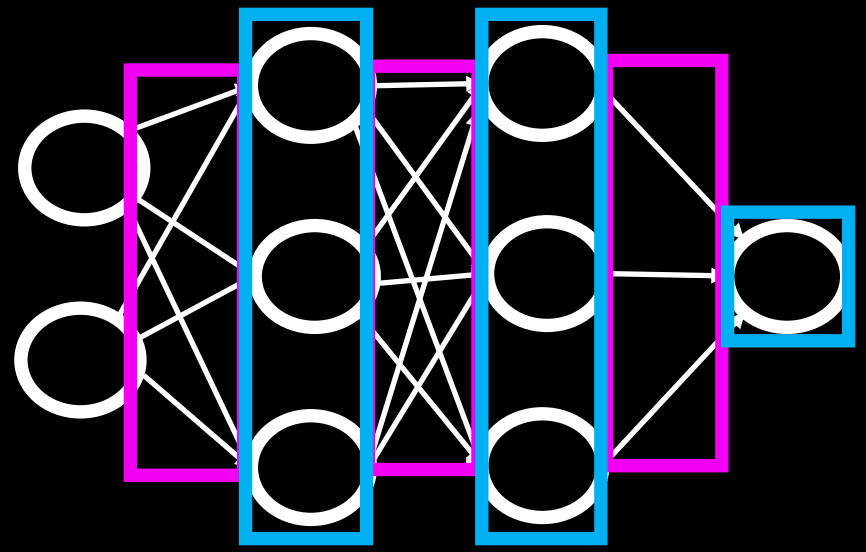


How this relates to a perceptron:



Python Machine Learning; Raschka & Mirjalili

Multi-Layer Network Basics

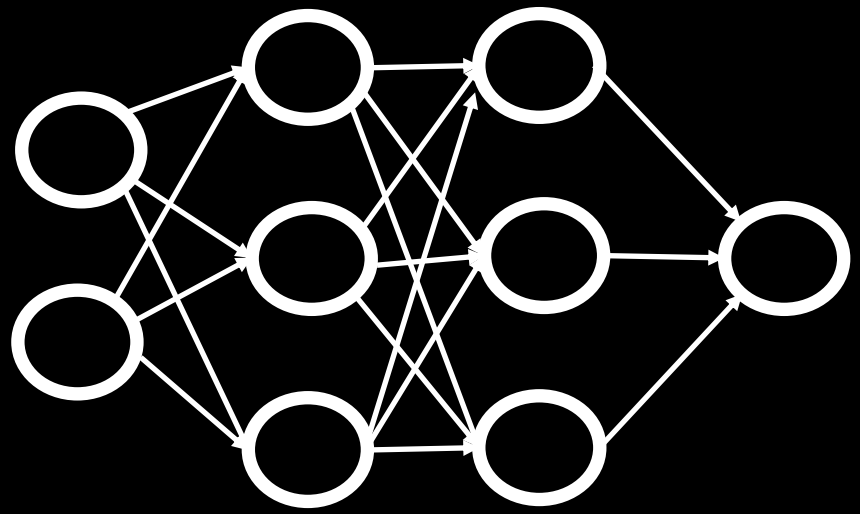


Goal: learn network parameters

How many parameters are in this network?

- Input to Hidden Layer 1 weights
  - 2x3 = 6
- Hidden Layer 1 to Hidden Layer 2 weights
  - 3x3 = 9
- Hidden Layer 2 to Output Layer weights
  - 3x1 = 3
- Number of bias terms
  - 3 + 3 + 1 = 7
- Total
  - 6 + 9 + 3 + 7 = 25

Multi-Layer Network Basics: Fully Connected, Feedforward Network



- Fully connected: Each unit provides input to each unit in the next layer
- Feedforward: Each layer serves as input to the next layer with no loops

Obstacles for Multi-Layer Networks (1970s)

- Problem difficulty: struggles to find a generalizable learning approach for training multi-layer neural networks
- Recognition: researchers moved to problems with less resistance and more positive feedback (e.g., symbolic AI, signal processing)
- Funding: agencies shifted support to a lesser-known AI approach called symbolic logic

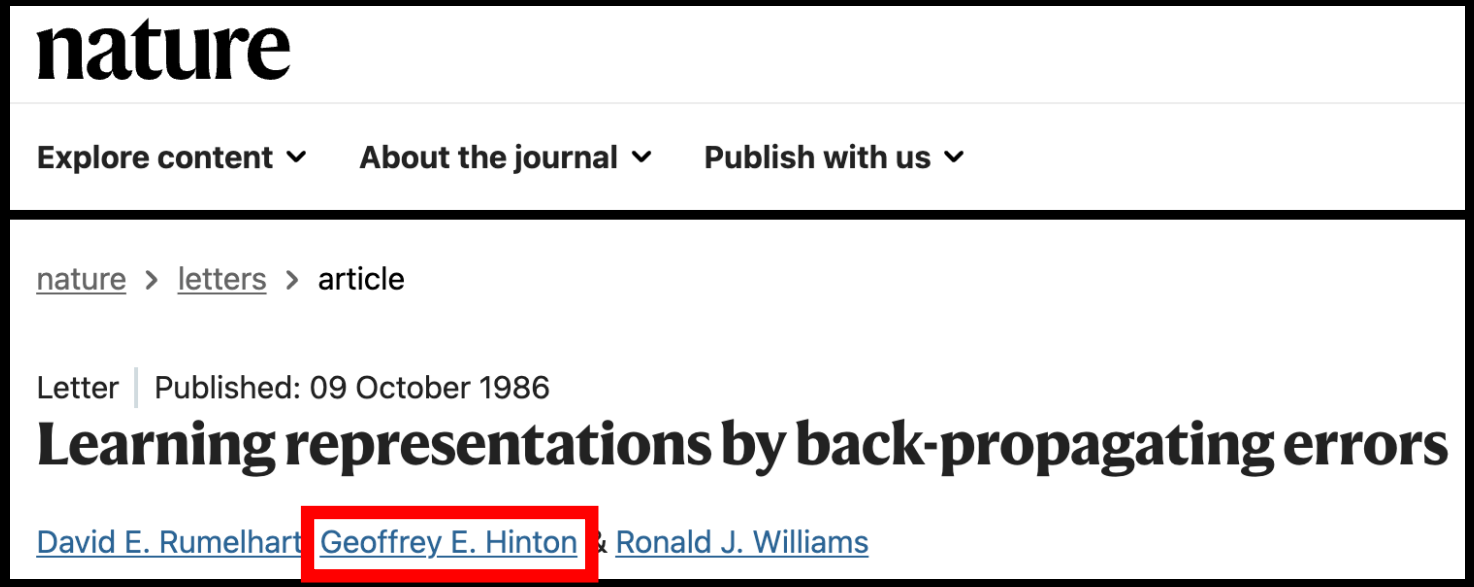
Training Attempts for Multi-Layer Networks; e.g.,

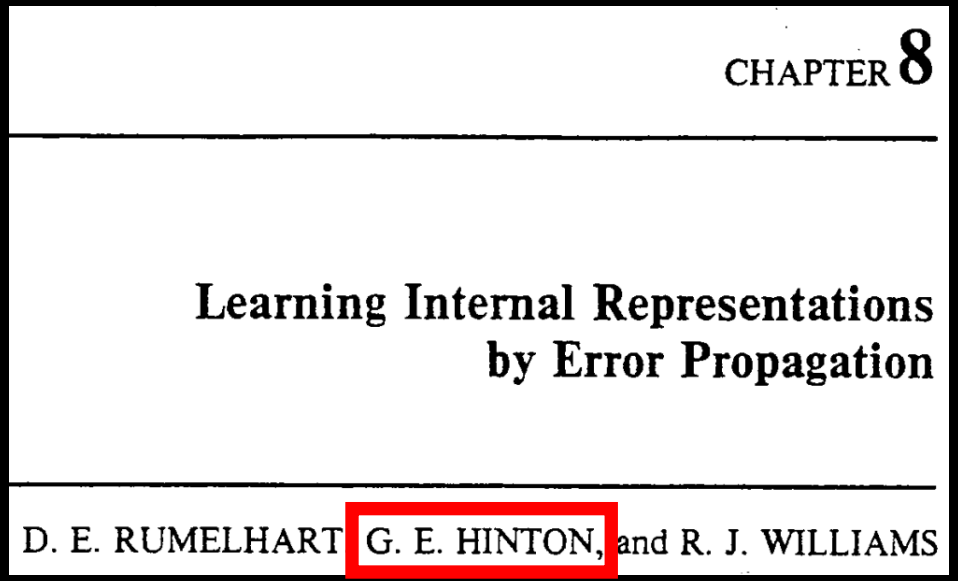
- Frank Rosenblatt; 1962 book
  - Proposed "Back-Propagating Error Correction Procedure": "Considerable improvement in performance might be obtained if the values of the S [sensory units] to A [association units] connections could somehow be optimized by a learning process… The effect of these rules is to try to turn off any A–units…whose output is currently contributing to an error in an R-unit, and to try to turn on any A –units… which are currently off, but whose output signals would help correct an error in one or more R-units if they were on."
- Paul Werbos; 1974 PhD dissertation
  - First description of training a neural network with steepest descent plus "dynamic feedback", previously used in control theory and now known as "backpropagation"
  - Timing and/or context not right… but the ideas eventually re-emerged a decade later
- LeCun and Parker; 1985 papers
  - Independently, they both discovered backpropagation (unaware of Werbos' work)

Today's Topics

- Vision: multi-layer networks provide sufficient representational power
- Breakthrough: gradients make multi-layer networks trainable
- Machine abstraction: neural networks as computational graphs
- Programming tutorial: developing neural networks in PyTorch

Backpropagation Algorithm Popularized for Training Multi-Layer Neural Networks





"We demonstrate that a general purpose and relatively simple procedure is powerful enough…"

Expanded discussion in book "Parallel Distributed Processing: Explorations in the Microstructure of Cognition"

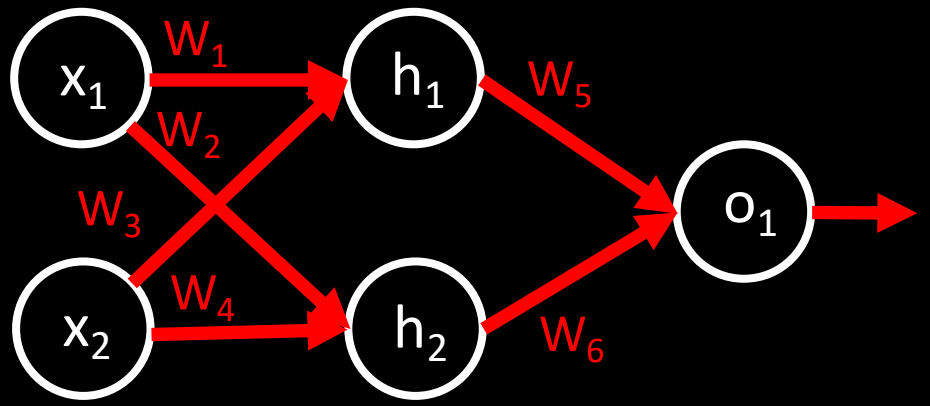
A Movement That Aged into a 2024 Nobel Prize



https://www.npr.org/2024/10/08/g-s1-26848/nobel-prize-in-physics-awarded-to-scientists-machine-learning

Critical Foundation: Representing Multi-Layer Networks As Non-Linear, Differentiable Functions

Why non-linear? Look at what happens if we chain together linear functions:

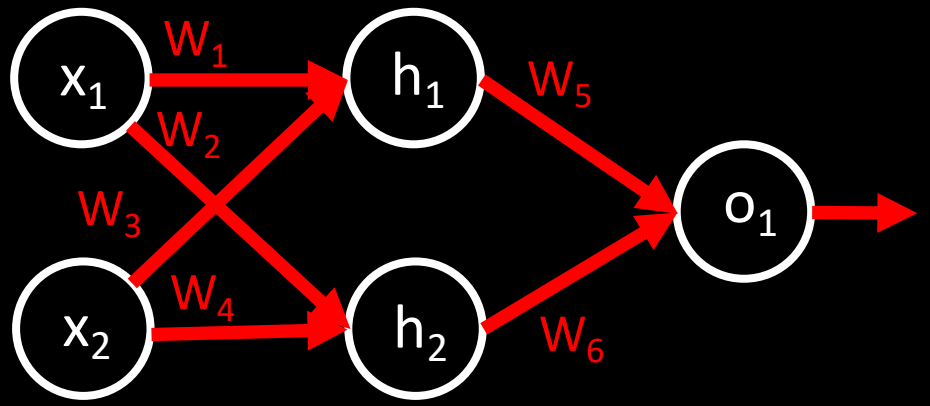


- What is function for h_1?
  - $h_1 = w_1x_1 + w_3x_2 + b_1$
- What is function for h_2?
  - $h_2 = w_2x_1 + w_4x_2 + b_2$
- What is function for o_1?
  - $o_1 = h_1w_5 + h_2w_6 + b_3$
  - $o_1 = (w_1x_1 + w_3x_2 + b_1)w_5 + (w_2x_1 + w_4x_2 + b_2)w_6 + b_3$
  - $o_1 = w_1w_5x_1 + w_3w_5x_2 + w_5b_1 + w_2w_6x_1 + w_4w_6x_2 + w_6b_2 + b_3$

A chain of LINEAR functions at any depth can be reduced to a single LINEAR function!

Critical Foundation: Representing Multi-Layer Networks As Non-Linear, Differentiable Functions

Why non-linear? Look at what happens if we chain together linear functions:



- What is function for h_1?
  - $h_1 = w_1x_1 + w_3x_2 + b_1$
- What is function for h_2?
  - $h_2 = w_2x_1 + w_4x_2 + b_2$
- What is function for o_1?
  - $o_1 = h_1w_5 + h_2w_6 + b_3$

Constant x linear function = linear function

A chain of LINEAR functions at any depth can be reduced to a single LINEAR function!

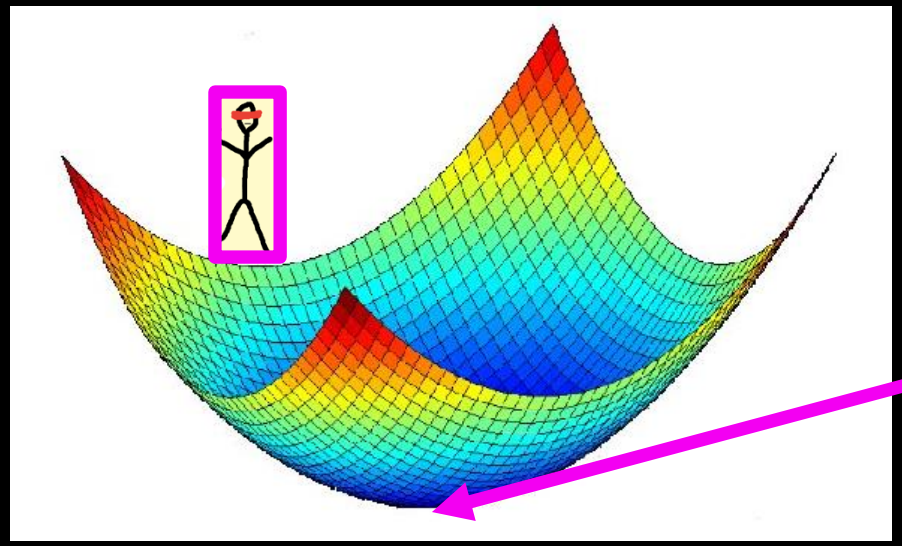
Critical Foundation: Representing Multi-Layer Networks As Non-Linear, Differentiable Functions

Why differentiable? Can move from learning as a "rule" to goal-driven "optimization"

- i.e., descend an error surface so that the model's predictions increasingly align with the targets

What goal might we optimize?

e.g., minimize sum of squared errors between target and prediction: $E(\mathbf{w}) = \frac{1}{2}\sum_i (y_i - \mathbf{w}^\top \mathbf{x_i})^2$



medium.com/@kritikajaiswal48/convex-vs-non-convex-functions-why-it-is-important-for-optimization-in-machine-learning-b290403a44cd

How many model parameters $(\mathbf{w})$ are in this example?

- 2; Each possible pair of weight values (horizontal plane) used to compute error values (vertical axis)
- What model parameters would be optimal?
- Gradient descent: "walking" down this bowl

Louis Augustin Cauchy: Compte Rendu `a l'Acad´emie des Sciences of October 18, 1847

Critical Foundation: Representing Multi-Layer Networks As Non-Linear, Differentiable Functions

Why differentiable? Can move from learning as a "rule" to goal-driven "optimization"

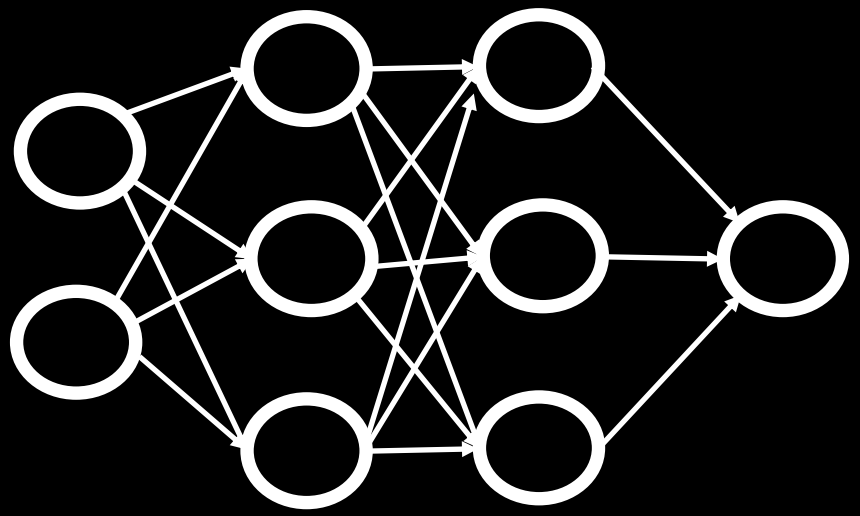
- i.e., descend an error surface so that the model's predictions increasingly align with the targets



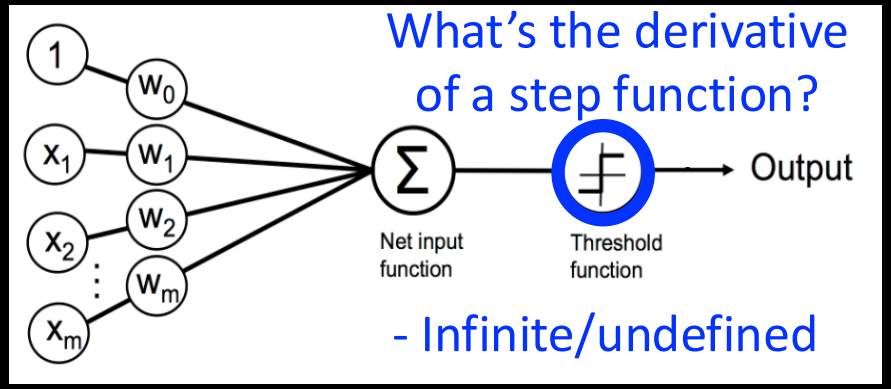
Analogy: Hiking from mountains to MBE 155 in Boulder (objective) in the dark or heavy fog (pick your favorite)!

Approach ("Gradient Descent"): Feel the slope locally to find downhill direction. Step. Repeat.

Critical Foundation: Representing Multi-Layer Networks As Non-Linear, Differentiable Functions

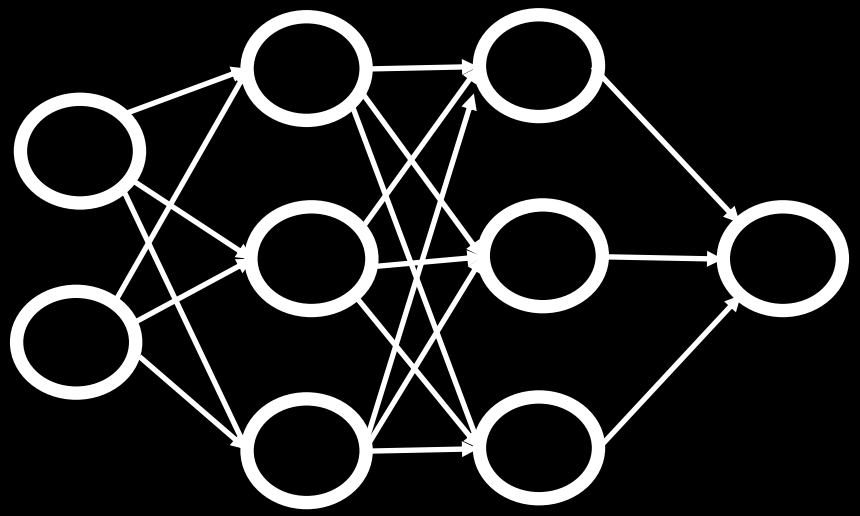


Recall how multi-layer networks relate to a perceptron:

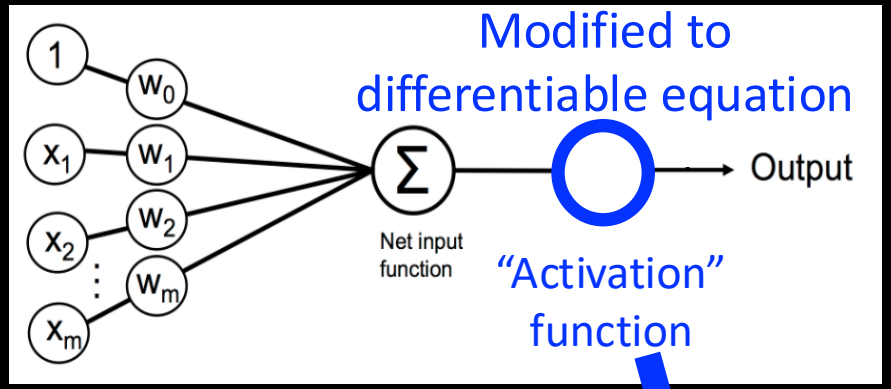


Python Machine Learning; Raschka & Mirjalili

Critical Foundation: Representing Multi-Layer Networks As Non-Linear, Differentiable Functions



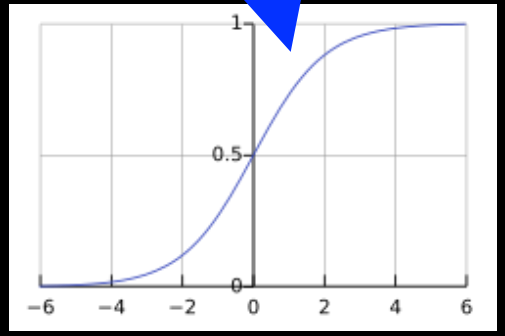
Recall how multi-layer networks relate to a perceptron:



Python Machine Learning; Raschka & [illegible]alili

Why is it incorrect to call these "multi-layer perceptrons"?

e.g., $$\sigma(x) = \frac{1}{1 + e^{-x}}$$:



Gradient Descent: Implementation

- Repeat until stopping criterion met:

    1. Propagate training data through model to predict output
    2. Measure error of the predictions using a loss function (e.g., sum of squared errors)
    3. Calculate gradients of the loss function to determine how each model parameter contributed to the error
    4. Update each parameter using calculated gradients

Note: this is not a biologically plausible learning approach



Idea: process to get to target water temperature for a shower

Gradient Descent: Implementation Intuition

- Repeat:
  1. Guess
  2. Calculate error
- e.g., learn constant multiplier to convert US dollars to Israeli shekels when only observing the input “dollars” and output “shekels”



Gradient Descent: Implementation Intuition

- Repeat:
  1. Guess
  2. Calculate error
- e.g., learn constant multiplier to convert US dollars to Israeli shekels when only observing the input “dollars” and output “shekels”



Gradient Descent: Implementation Intuition

- Repeat:
  1. Guess
  2. Calculate error
- e.g., learn constant multiplier to convert US dollars to Israeli shekels when only observing the input “dollars” and output “shekels”



Gradient Descent: Implementation Intuition

- Repeat:
  1. Guess
  2. Calculate error
- e.g., learn constant multiplier to convert US dollars to Israeli shekels when only observing the input “dollars” and output “shekels”



Gradient Descent: Implementation Intuition

- Repeat:
  1. Guess
  2. Calculate error
- e.g., learn constant multiplier to convert US dollars to Israeli shekels when only observing the input “dollars” and output “shekels”



Gradient Descent: Implementation Intuition

- Repeat:
  1. Guess
  2. Calculate error
- e.g., learn constant multiplier to convert US dollars to Israeli shekels when only observing the input “dollars” and output “shekels”



- Idea: iteratively adjust constant (i.e., model parameter) to reduce error

Gradient Descent: Implementation

- Repeat until stopping criterion met:

    1. Propagate training data through model to predict output
    2. Measure error of the predictions using a loss function (e.g., sum of squared errors)
    3. Calculate gradients of the loss function to determine how each model parameter contributed to the error
    4. Update each parameter using calculated gradients

We babysit this step to see how learning is going; what we expect:



Gradient Descent: Definitions

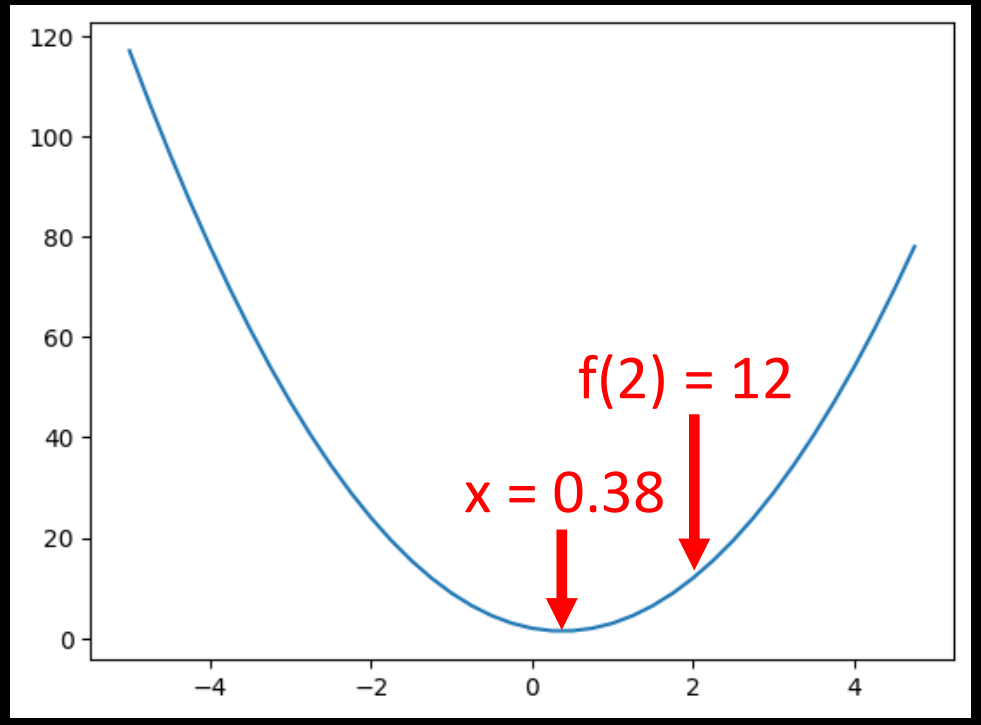
- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables
- **Derivative**: indicates slope (rise/run) of a function at any point

e.g., $f(x) = 2 - 3x + 4x^2$

Is this function convex or non-convex?

What is the value of the function at x =2?

What is the value of x at the minimum of f(x)?



Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables
- **Derivative**: indicates slope (rise/run) of a function at any point

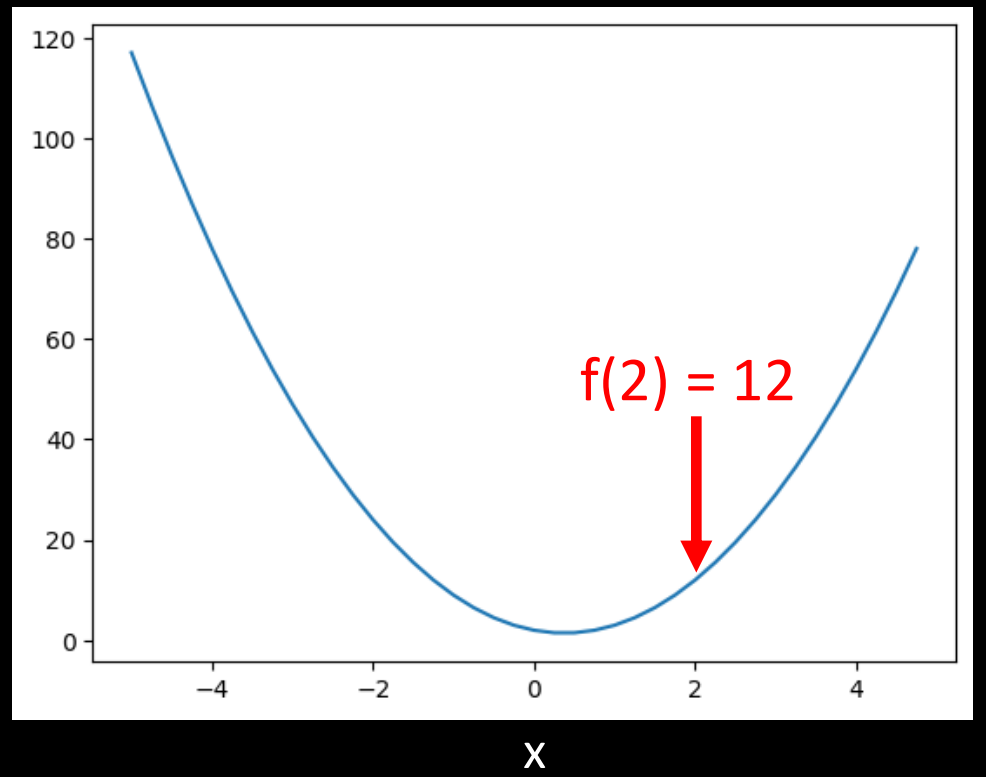
e.g., $f(x) = 2 - 3x + 4x^2$

How a slight change (i.e., h) to x changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(x+h) - f(x)}{h}$$

If *h* is slightly positive, at x=2, will *f(x+h)* be greater than or less than 12?

By how much?



Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables
- **Derivative**: indicates slope (rise/run) of a function at any point

e.g., $f(x) = 2 - 3x + 4x^2$

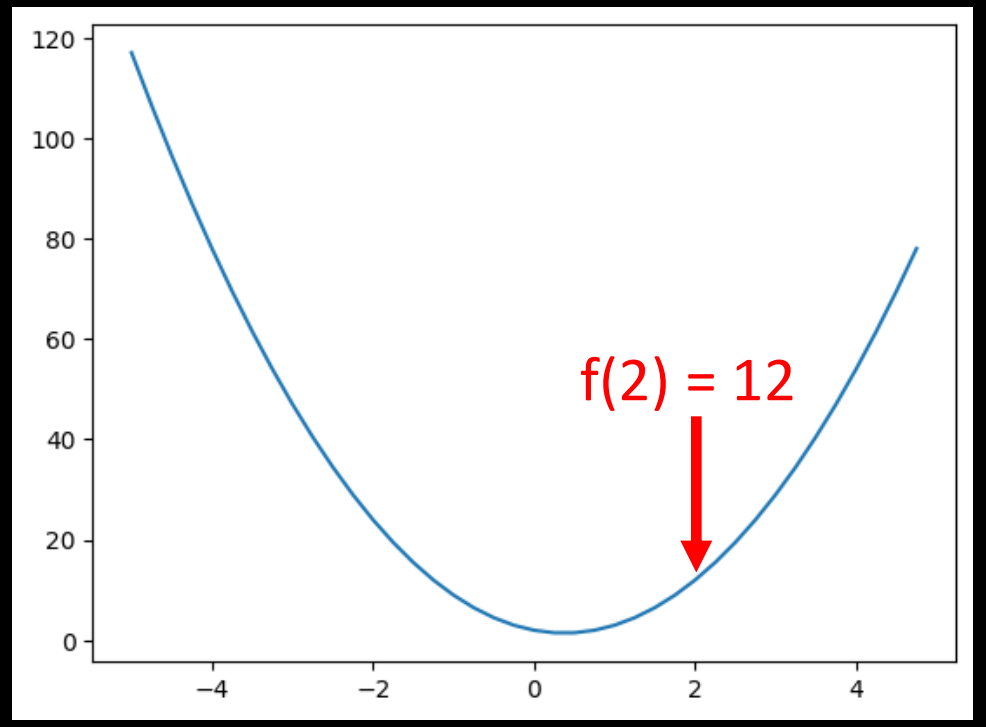
How a slight change (i.e., h) to x changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(x+h) - f(x)}{h}$$

e.g., at x = 2

- for h = 0.01: f(x+h) = 12.1304, slope = 13.04

Increasing x by h=0.01 results in an increase of ~13.04 times that amount (13.04 * .01)



Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables
- **Derivative**: indicates slope (rise/run) of a function at any point

e.g., $f(x) = 2 - 3x + 4x^2$

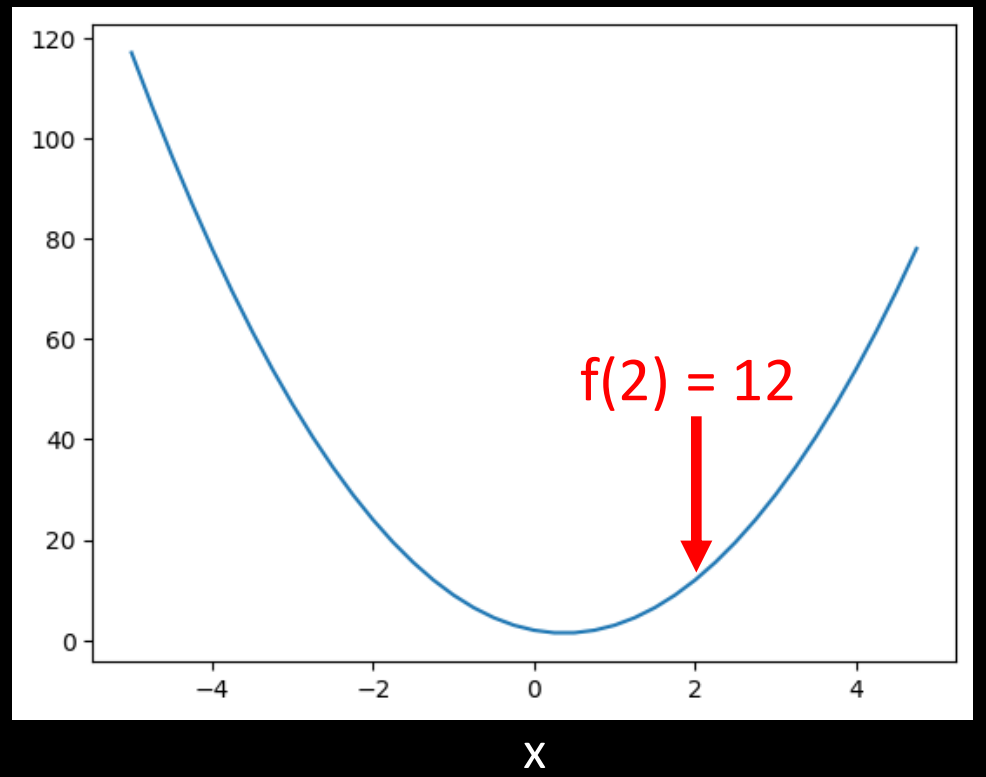
How a slight change (i.e., h) to x changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(x+h) - f(x)}{h}$$

e.g., at x = 2

- for h = 0.01: f(x+h) = 12.1304, slope = 13.04
- for h = 0.001: f(x+h) = 12.013004 & slope = 13.004
- for h = 0.0001: f(x+h) = 12.0013004 & slope = 13.0004

As h gets closer to 0, slope approaches what?
Mathematically, f'(x) = -3 + 8x; f'(2) = -3 + 16 = 13



Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables
- **Derivative**: indicates slope (rise/run) of a function at any point

e.g., $f(x) = 2 - 3x + 4x^2$

How a slight change (i.e., h) to x changes the output:

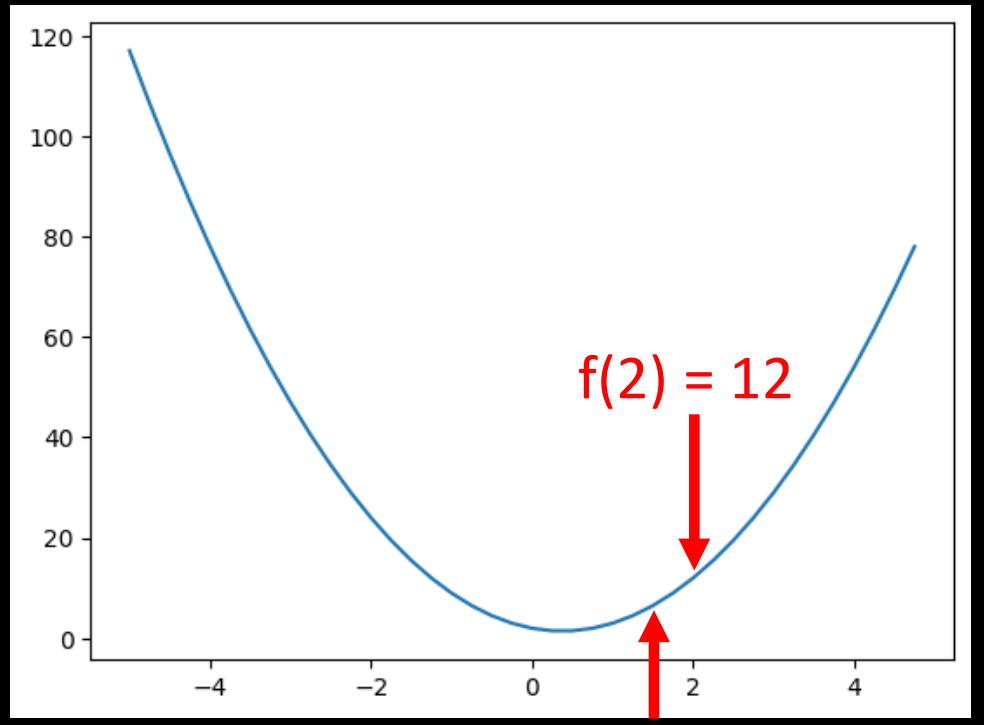
$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(x+h) - f(x)}{h}$$

e.g., at x = 2; f'(x) = 13

Approach minimum point by stepping in opposite direction of derivative (e.g., assume learning rate=0.01).

- What should the new value for x be?

2 – (0.01 * 13) = 1.87



Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables
- **Derivative**: indicates slope (rise/run) of a function at any point

e.g., $f(x) = 2 - 3x + 4x^2$

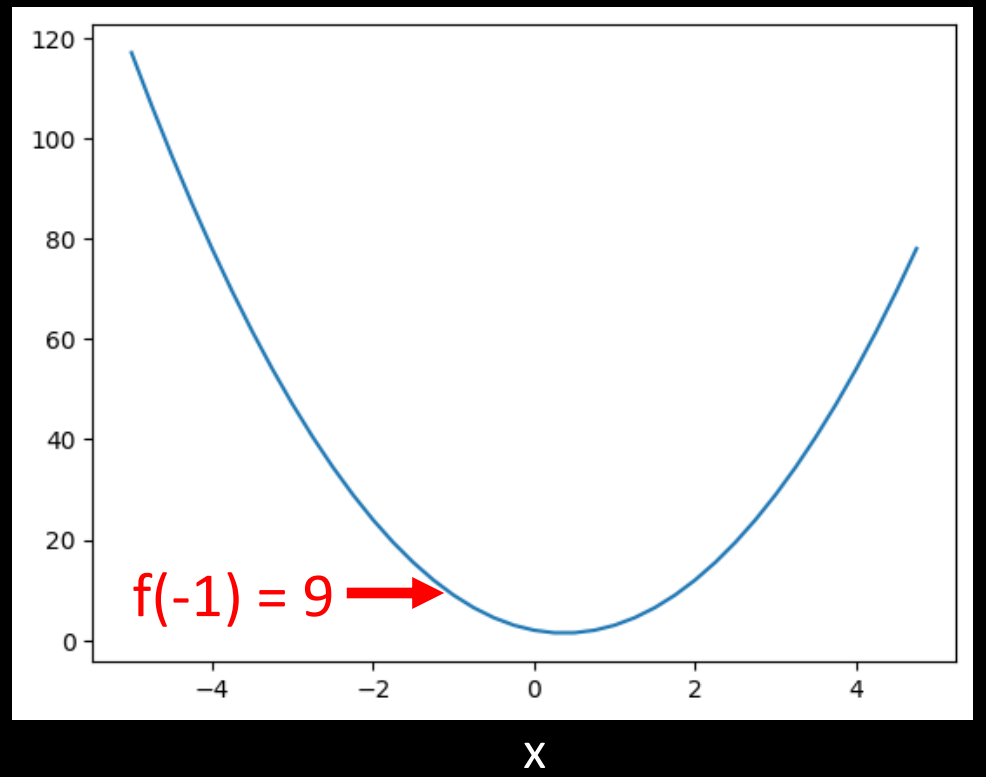
How a slight change (i.e., h) to x changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(x+h) - f(x)}{h}$$

e.g., at x = -1

- for h = 0.01: f(x+h) = 8.8904, slope = -10.96
- for h = 0.001: f(x+h) = 8.989004, slope = -10.996
- for h = 0.0001: f(x+h) = 8.99890004, slope = -10.9996

As h gets closer to 0, slope approaches what?
Mathematically, f'(x) = -3 + 8x; f'(-1) = -3 + 8*-1 = -11



Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables
- **Derivative**: indicates slope (rise/run) of a function at any point

e.g., $f(x) = 2 - 3x + 4x^2$

How a slight change (i.e., h) to x changes the output:

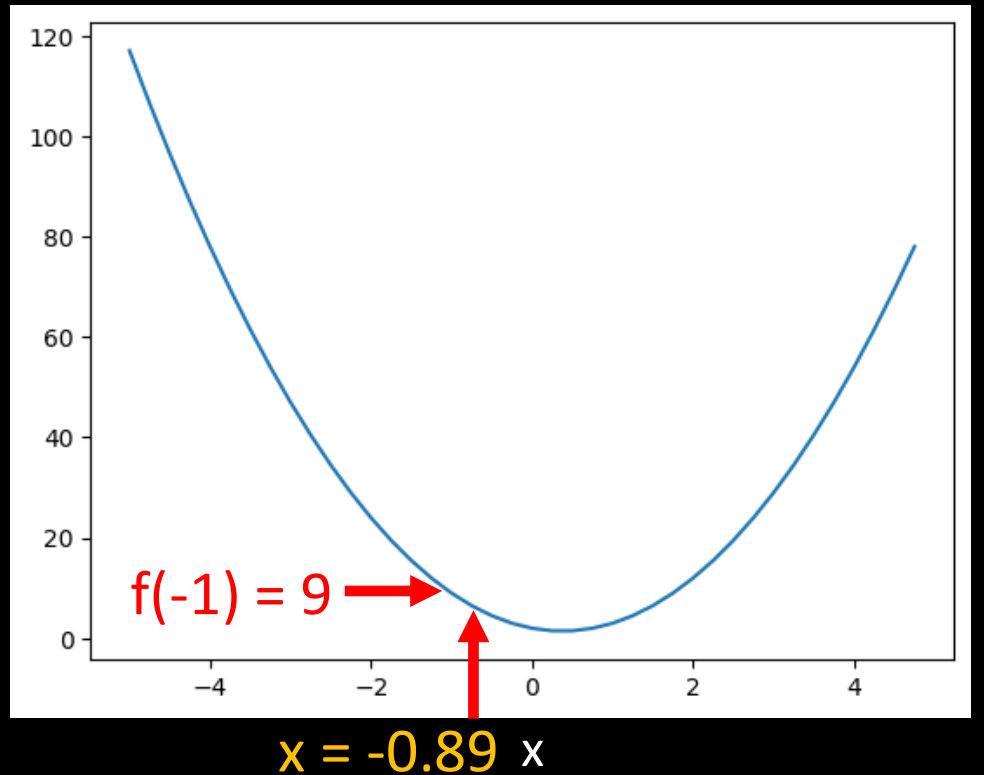
$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(x+h) - f(x)}{h}$$

e.g., at x = -1; f'(x) = -11

Approach minimum point by stepping in opposite direction of derivative (e.g., assume learning rate=0.01).

- What should the new value for x be?

-1 – (0.01 * -11) = -0.89



Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables

  e.g., f(**x**) = a + bc; where **x** is a vector with all parameters

How a slight change (i.e., h) to any input in **x** changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(\mathbf{x} + h) - f(\mathbf{x})}{h}$$

What is the value of the function when a = 1, b = -2, and c = 3?

If we only change *a* with a slightly positive *h*, will f(**x**+h) be greater than or less than -5?

By how much?

Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables

  e.g., f(**x**) = a + bc; where **x** is a vector with all parameters

  How a slight change (i.e., h) to any input in **x** changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(\mathbf{x} + h) - f(\mathbf{x})}{h}$$

  e.g., at a = 1, b = -2, and c = 3 when only changing the value of a
  - for h = 0.01: f(**x**+h) = (1+.01) + -2*3 = -4.99, slope = 1.000
  - for h = 0.001: f(**x**+h) = (1+.001) + -2*3 = -4.999, slope = 1.000

  As h gets closer to 0, slope approaches 1

  Mathematically, df/da = 1

Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables

  e.g., f(**x**) = a + bc; where **x** is a vector with all parameters

How a slight change (i.e., h) to any input in **x** changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(\mathbf{x} + h) - f(\mathbf{x})}{h}$$

Recall the value of the function when a = 1, b = -2, and c = 3 is -5

If we change only *b* with a slightly positive *h*, will f(**x**+h) be greater than or less than -5?

By how much?

Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables

  e.g., f(**x**) = a + bc; where **x** is a vector with all parameters

  How a slight change (i.e., h) to any input in **x** changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(\mathbf{x} + h) - f(\mathbf{x})}{h}$$

  e.g., at a = 1, b = -2, and c = 3 when only changing the value of b (rounding to 3 decimals)
  - for h = 0.01: f(**x**+h) = 1 + (-2 +.01)*3 = -4.970, slope = 3.000
  - for h = 0.001: f(**x**+h) = 1 + (-2 +.001)*3 = -4.997, slope = 3.000

  As h gets closer to 0, slope approaches 3

  Mathematically, df/db = c (which we set to 3)

Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables

  e.g., f(**x**) = a + bc; where **x** is a vector with all parameters

How a slight change (i.e., h) to any input in **x** changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(\mathbf{x} + h) - f(\mathbf{x})}{h}$$

Recall the value of the function when a = 1, b = -2, and c = 3 is -5

If we change only *c* with a slightly positive *h*, will f(**x**+h) be greater than or less than -5?

By how much?

Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables

  e.g., f(**x**) = a + bc; where **x** is a vector with all parameters

How a slight change (i.e., h) to any input in **x** changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(\mathbf{x} + h) - f(\mathbf{x})}{h}$$

e.g., at a = 1, b = -2, and c = 3 when only changing the value of c (rounding to 3 decimals)
- for h = 0.01: f(**x**+h) = 1 + -2*(3 +.01) = -5. 02, slope = -2.000
- for h = 0.001: f(**x**+h) = 1 + -2*(3 +.001) = -5.002, slope = -2.000

As h gets closer to 0, slope approaches -2

Mathematically, df/dc = b (which we set to -2)

Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables

  e.g., f(**x**) = a + bc; where **x** is a vector with all parameters

  How a slight change (i.e., h) to any input in **x** changes the output:

$$\text{slope} = \frac{\text{rise}}{\text{run}} = \frac{f(\mathbf{x} + h) - f(\mathbf{x})}{h}$$

  In summary, gradient for any input can be computed with these equations:

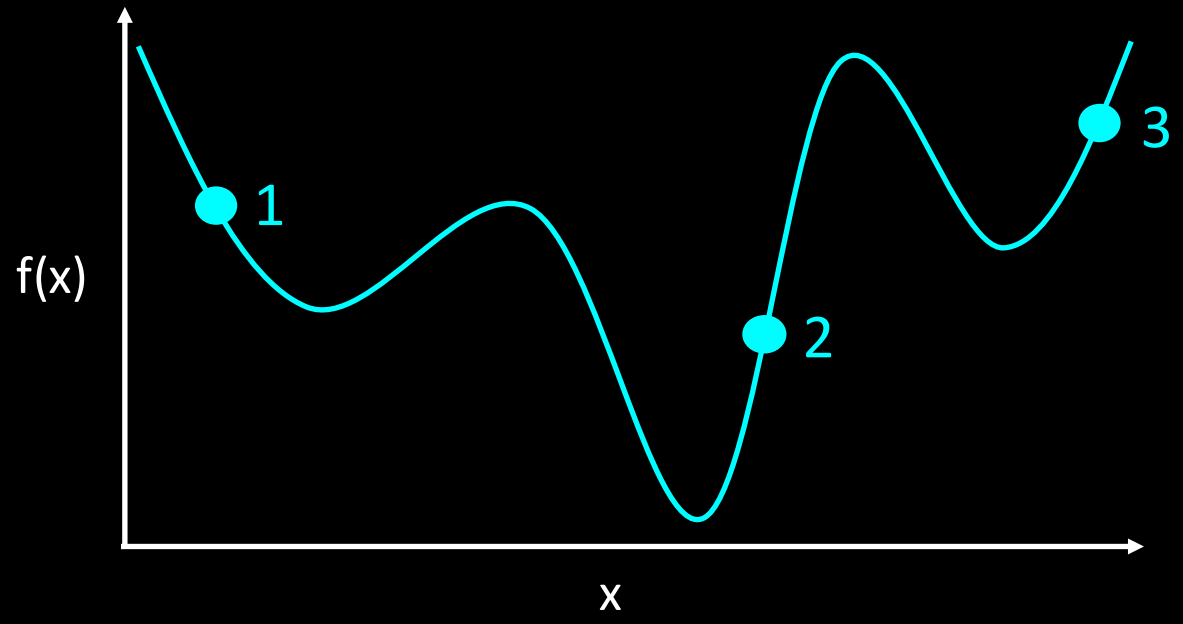
  df/da = 1; df/db = c; df/dc =b

  Can approach minimum point by stepping (e.g., learning rate of 0.01) in opposite direction of derivative; e.g., at a = 1, b = -2, and c = 3

  a - 0.01*1 = 0.99; b - 0.01*c = -2.03; c - 0.01*b = 3.02 f(x) = -5.14

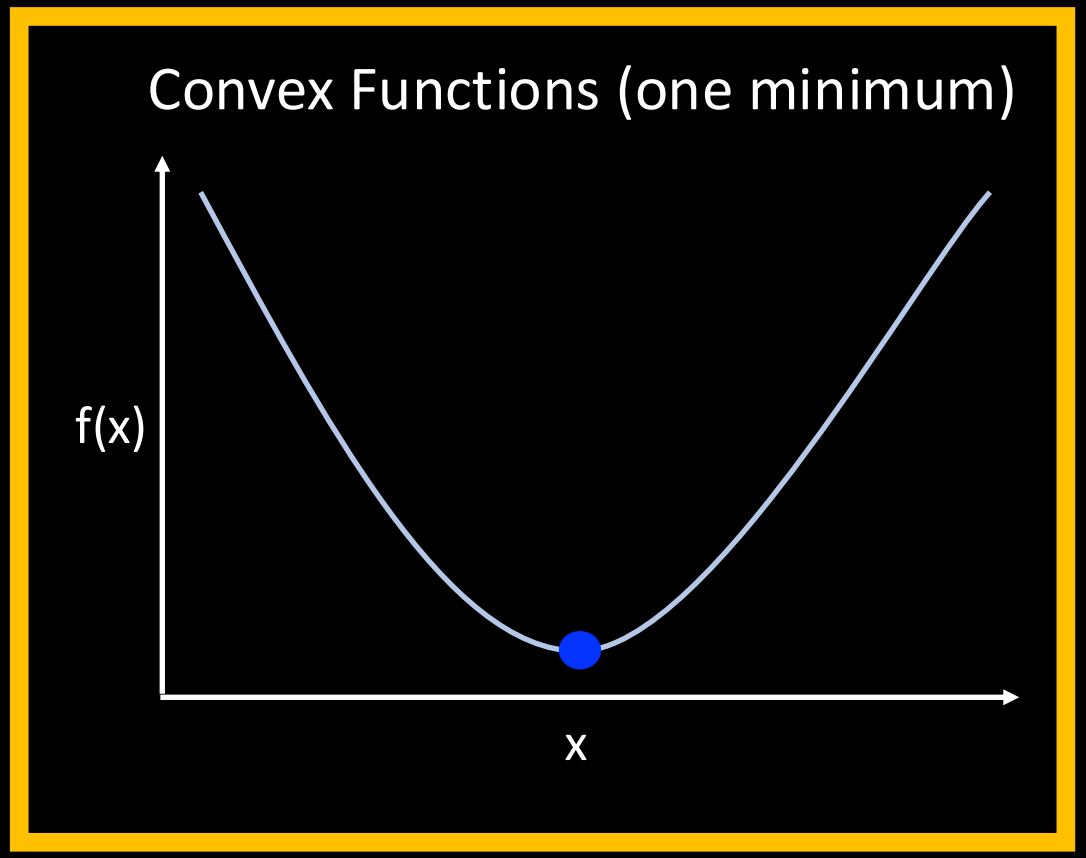
Gradient Descent: Definitions

- **Gradient:** a vector indicating how a slight change to each function variable in ***x*** increases the output ***f(x)***; i.e., partial derivatives for multiple variables
- **Gradient descent**: to ***minimize*** the function, iteratively step from each starting point in opposite direction of gradient (i.e., descent rather than ascent)

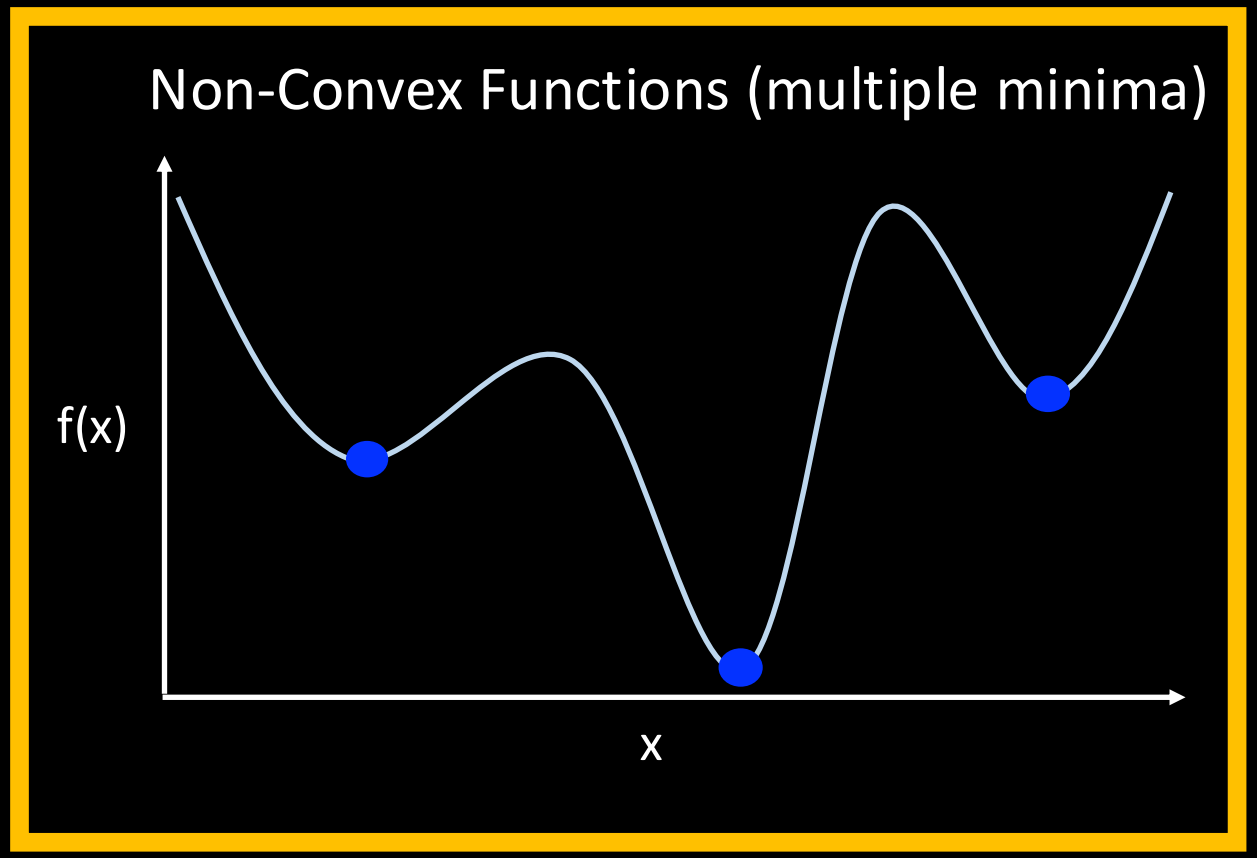


Gradient Descent: Possible Scenarios

(simple 1-parameter plots)



Currency problem



Our focus: neural network learning

Gradient Descent: Implementation

- Repeat until stopping criterion met:

    1. Propagate training data through model to predict output

    2. Measure error of the predictions using a loss function (e.g., sum of squared errors)

    3. Calculate gradients of the loss function to determine how each model parameter contributed to the error

    4. Update each parameter using calculated gradients

Key breakthrough: "backpropagation"

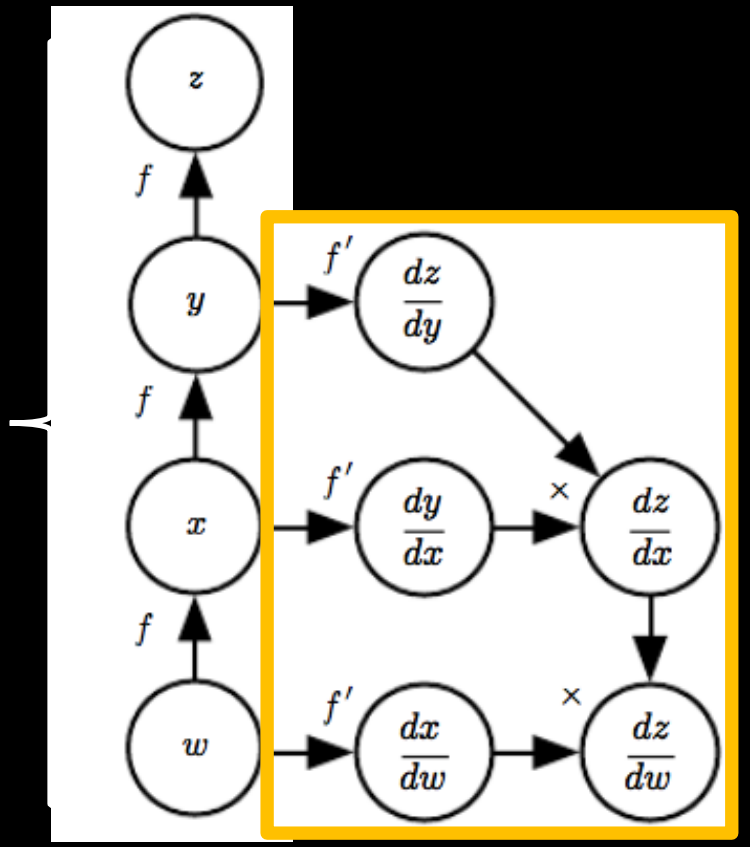
Backpropagation Basics

- **Key observation**: loss function is a mathematical function connected in a chain
- **What we want:** partial derivatives for function variables to inform how to minimize the loss function
- **Backpropagation**: achieves this with chain rule of calculus, which relies only on local derivatives; e.g.,

$$z = f(y)$$

$$y = f(x)$$

$$x = f(w)$$



Indicates how much a small change to each variable changes the function's output

Ian Goodfellow, Yoshua Bengio, and Aaron Courville; Deep Learning, 2016.

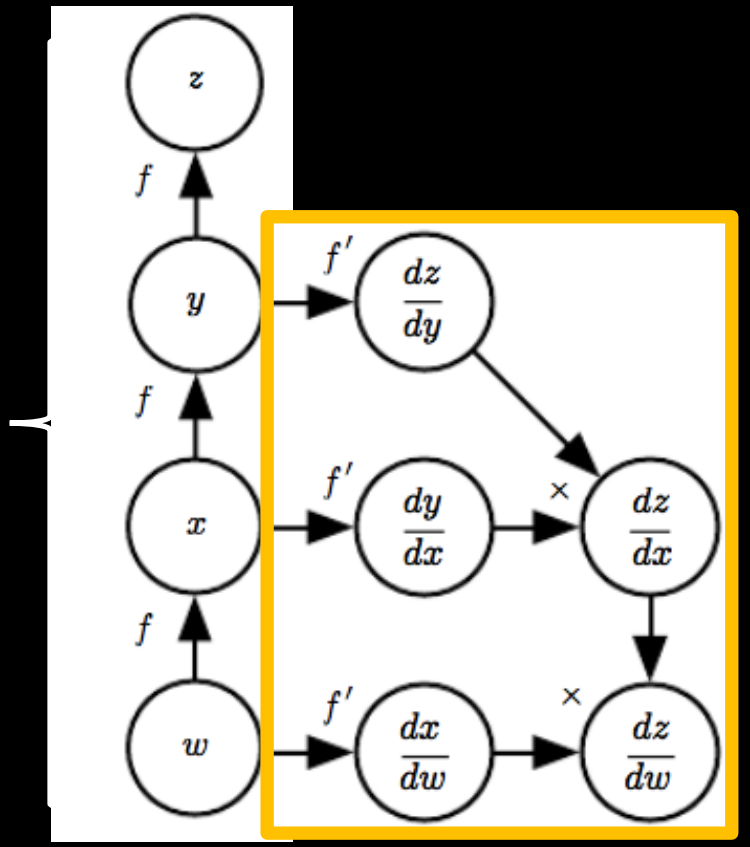
Backpropagation Basics

- **Key observation**: loss function is a mathematical function connected in a chain
- **What we want:** partial derivatives for function variables to inform how to minimize the loss function
- **Backpropagation**: achieves this with chain rule of calculus, which relies only on local derivatives; e.g.,

$$z = f(y)$$

$$y = f(x)$$

$$x = f(w)$$



What is *dz/dx*?

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx}$$

(how changes in x will change the output, z)

Ian Goodfellow, Yoshua Bengio, and Aaron Courville; Deep Learning, 2016.

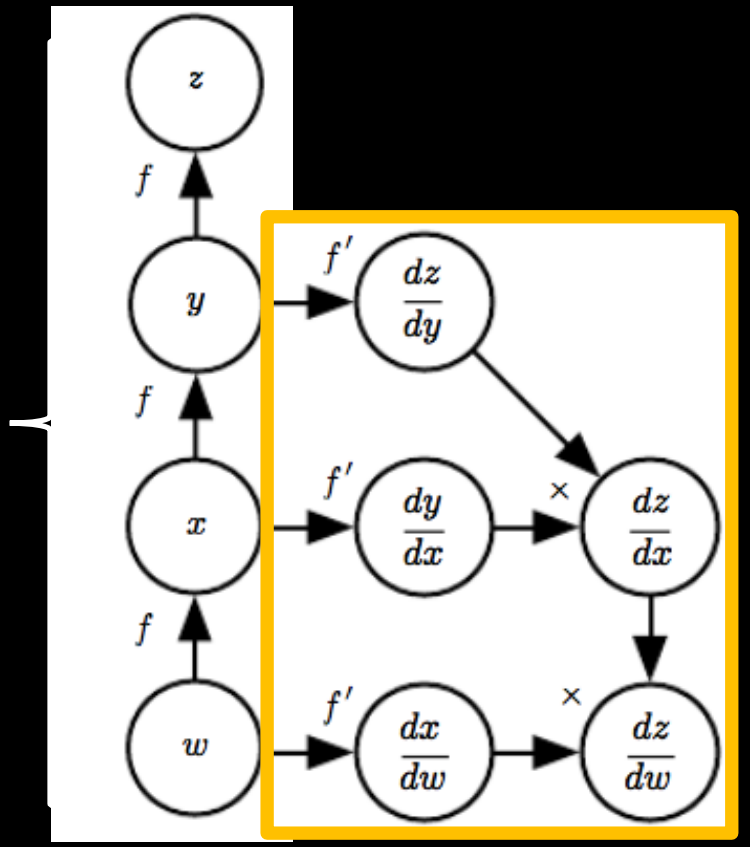
Backpropagation Basics

- **Key observation**: loss function is a mathematical function connected in a chain
- **What we want:** partial derivatives for function variables to inform how to minimize the loss function
- **Backpropagation**: achieves this with chain rule of calculus, which relies only on local derivatives; e.g.,

$$z = f(y)$$

$$y = f(x)$$

$$x = f(w)$$



What is *dz/dx*?

Intuitive example: how much faster is my husband compared to my daughter? (*dz/dx*)

dz/dy: husband is 2x faster than son



dy/dx: son is 3x faster than daughter



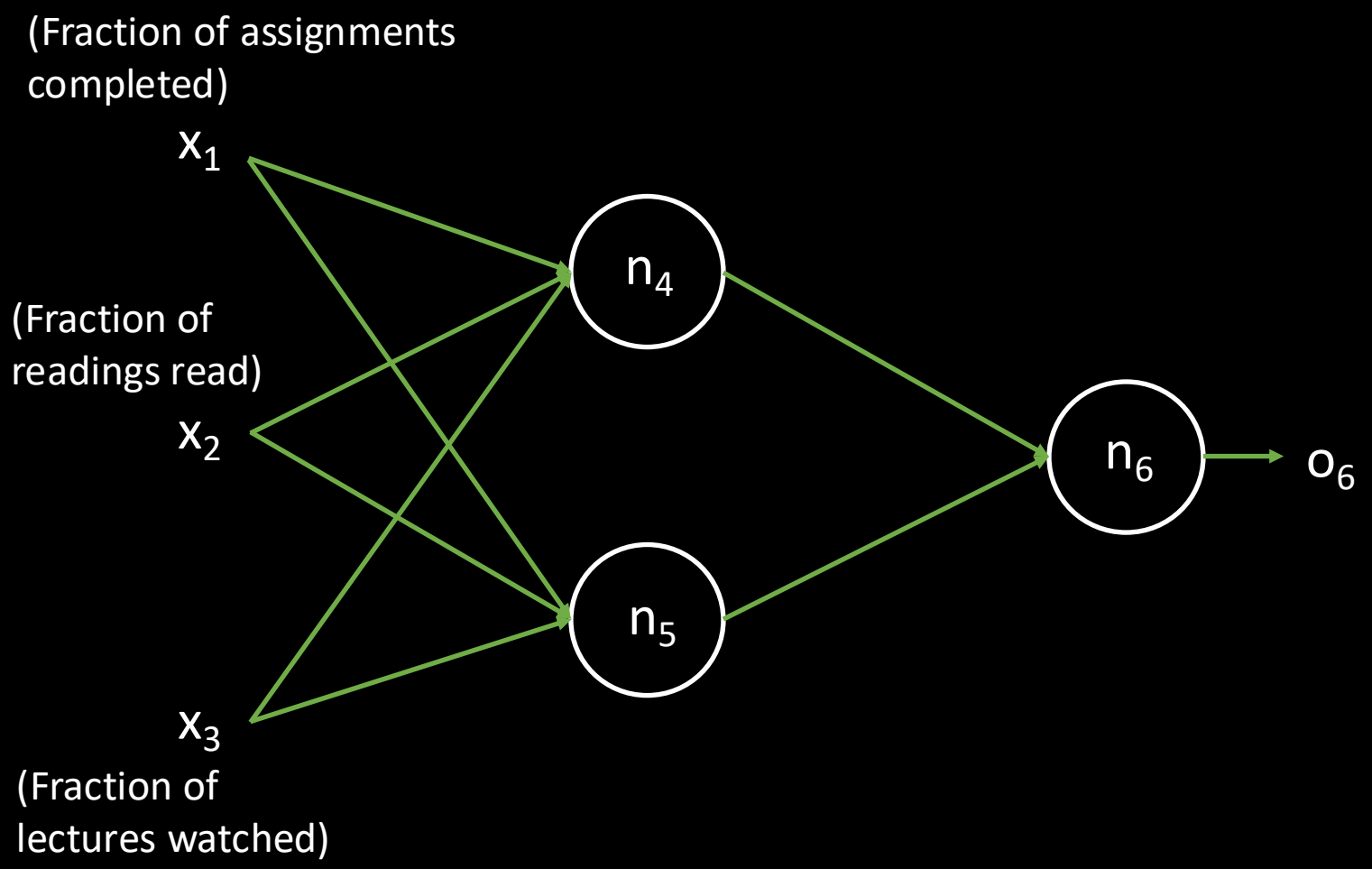
Ian Goodfellow, Yoshua Bengio, and Aaron Courville; Deep Learning, 2016.

Example

- Binary classification: predict if a student will get a B or better (minimum required for CS graduate student in this course)

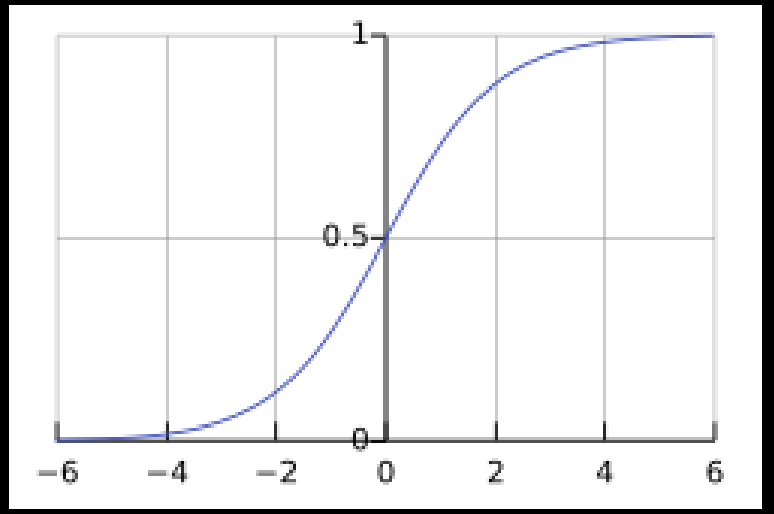
- Inputs:
    - Fraction of assignments completed
    - Fraction of readings read
    - Fraction of lectures watched

Example: Choose Neural Network Architecture



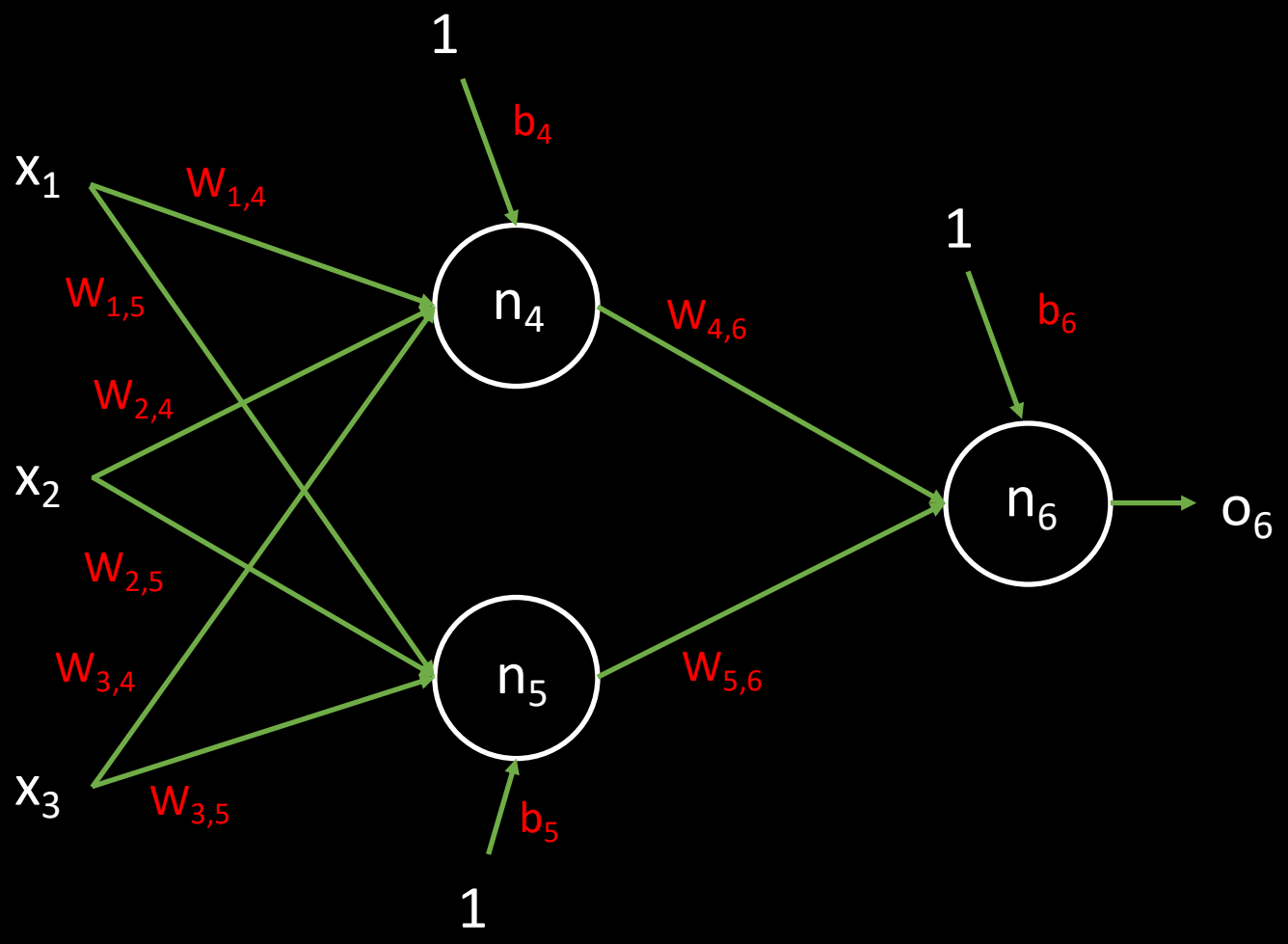
Sigmoid Activation Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Example from: Jiawei Han and Micheline Kamber; Data Mining.

Example: Choose Neural Network Architecture



2-layer neural network

How many weights will we learn?

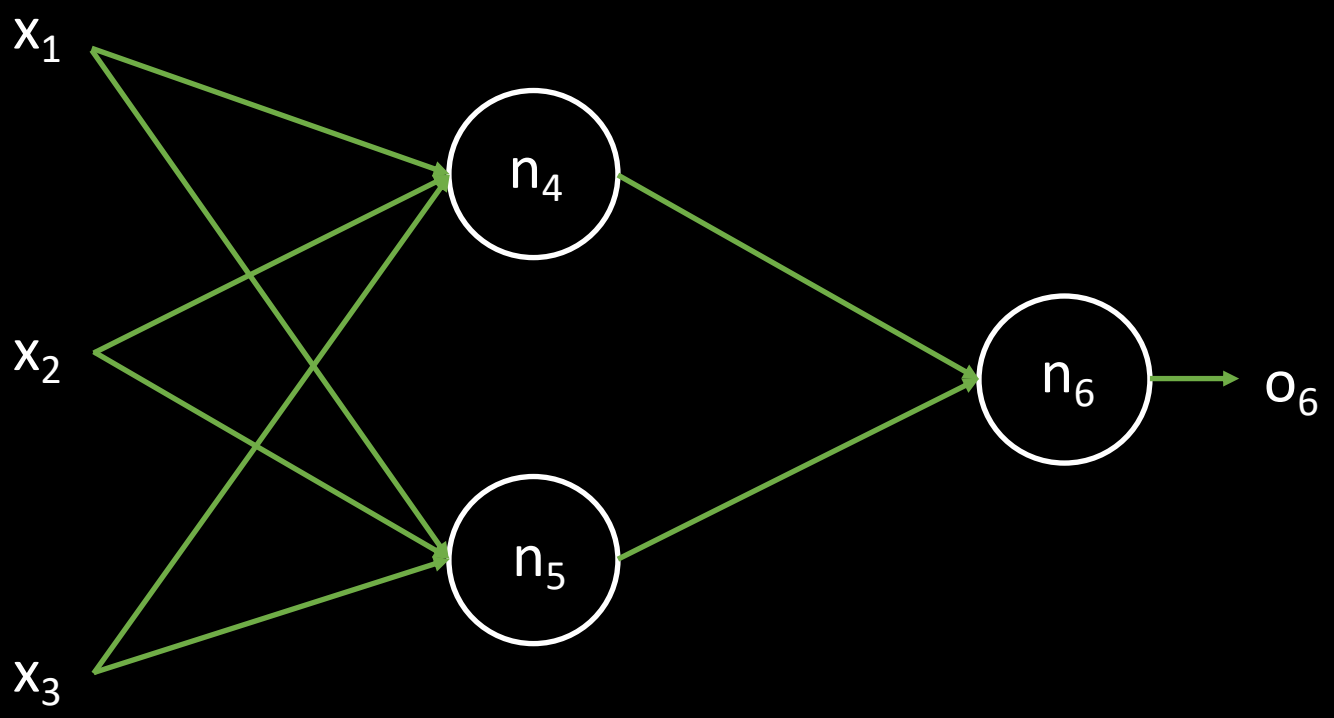
- Input to Hidden Layer 1:
  - 3x2 = 6
- Hidden Layer 1 to Hidden Layer 2:
  - 2x1 = 2

How many biases will we learn?

- 3

Example from: Jiawei Han and Micheline Kamber; Data Mining.

Example: Choose Loss Function



Squared Error

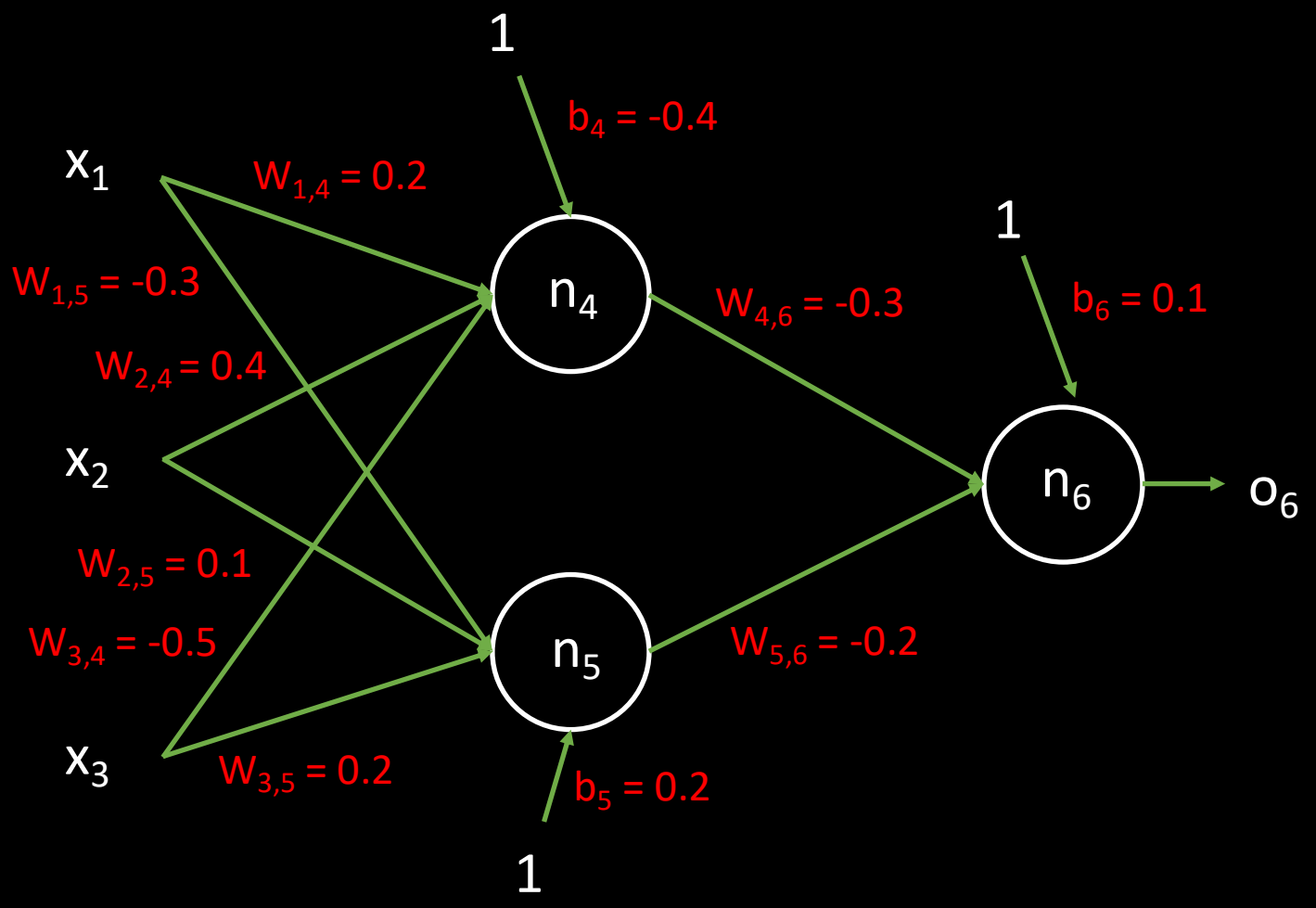
$$(Y_i - \hat{Y}_i)^2$$

True value

Predicted value

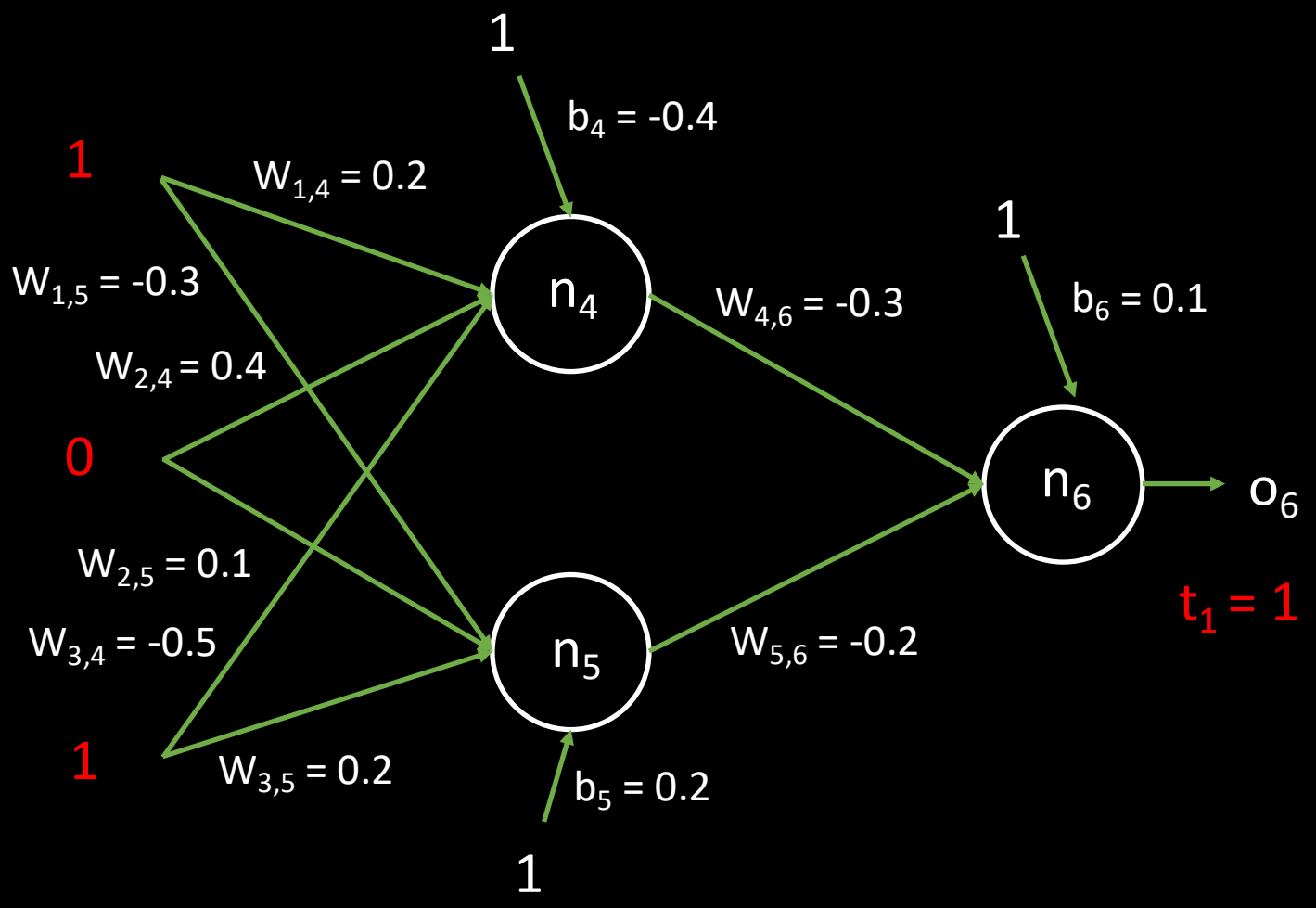
Example from: Jiawei Han and Micheline Kamber; Data Mining.

Example: Initialize Model Parameters



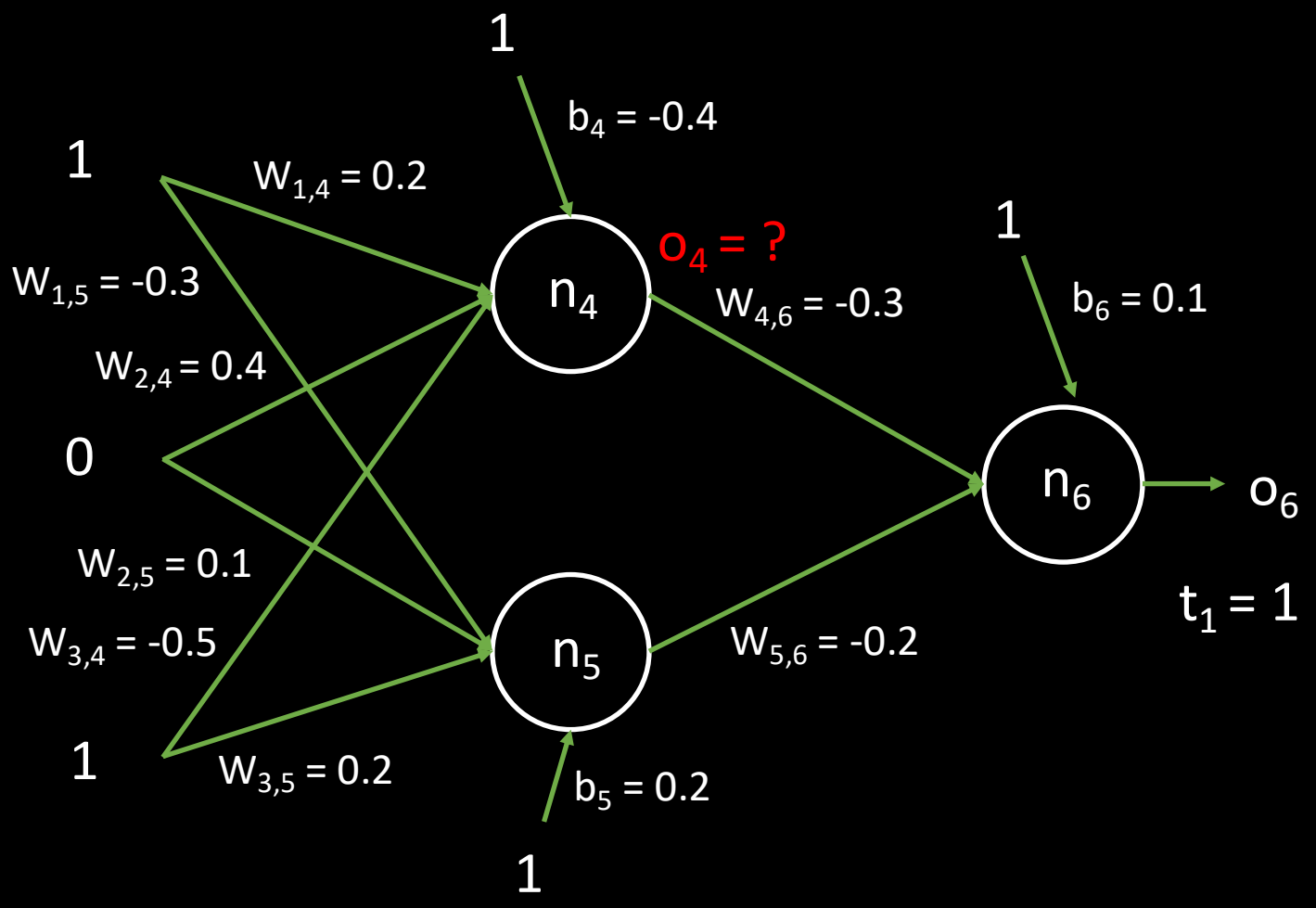
Example from: Jiawei Han and Micheline Kamber; Data Mining.

Example: Input Training Example



Example from: Jiawei Han and Micheline Kamber; Data Mining.

Example: Step 1 – Forward Pass



Weighted input sum to node 4:

$i_4 = (1 \times 0.2 + 0 \times 0.4 + 1 \times -0.5) - 0.4$

$i_4 = -0.7$

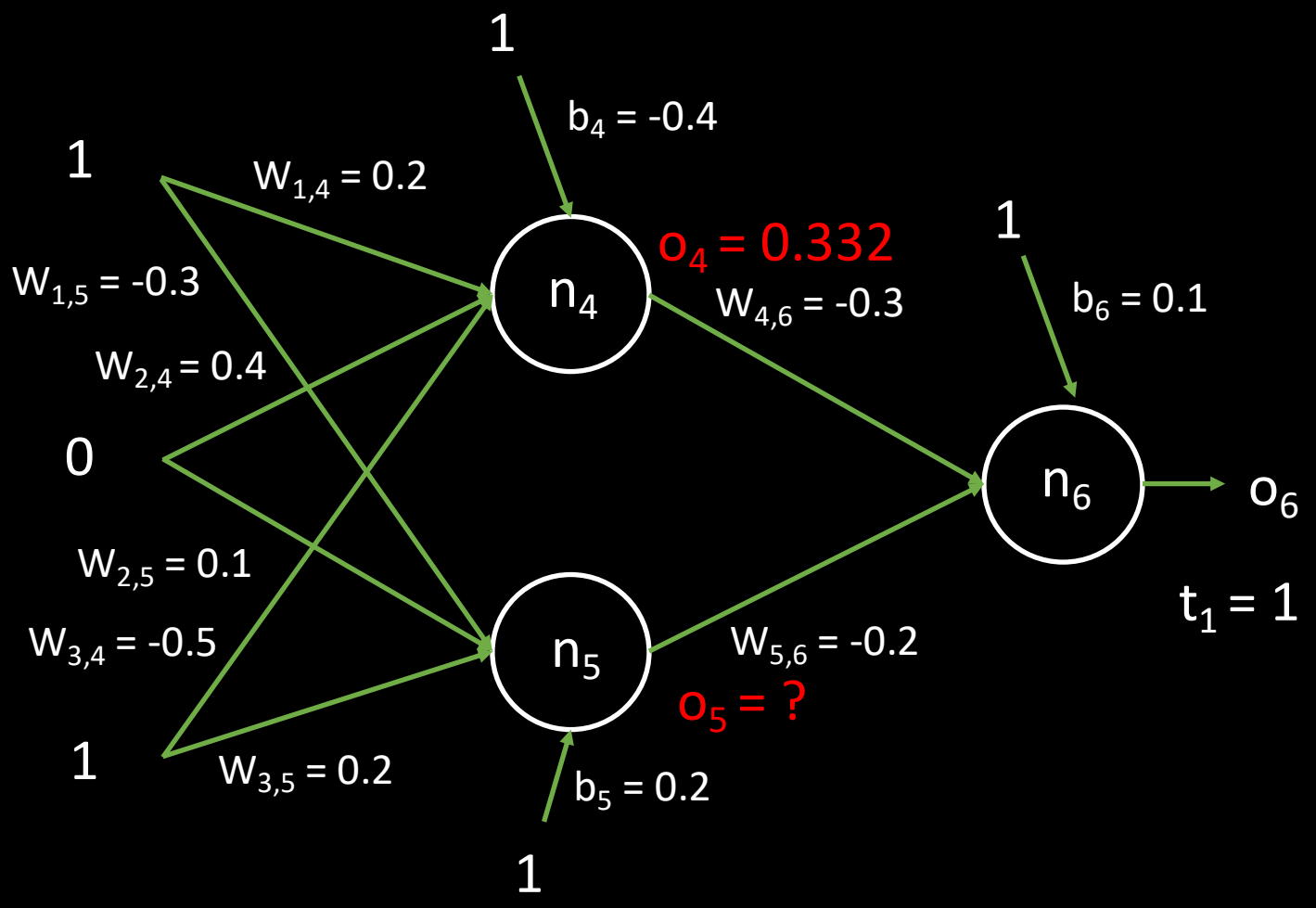
Activation result from node 4:

$o_4 = sigmoid(-0.7)$

$o_4 = 1/(1+e^{-(-0.7)})$

$o_4 = 0.332$

Example: Step 1 – Forward Pass



Weighted input sum to node 5:

$i_5 = (1 \times -0.3 + 0 \times 0.1 + 1 \times 0.2) + 0.2$

$i_5 = 0.1$

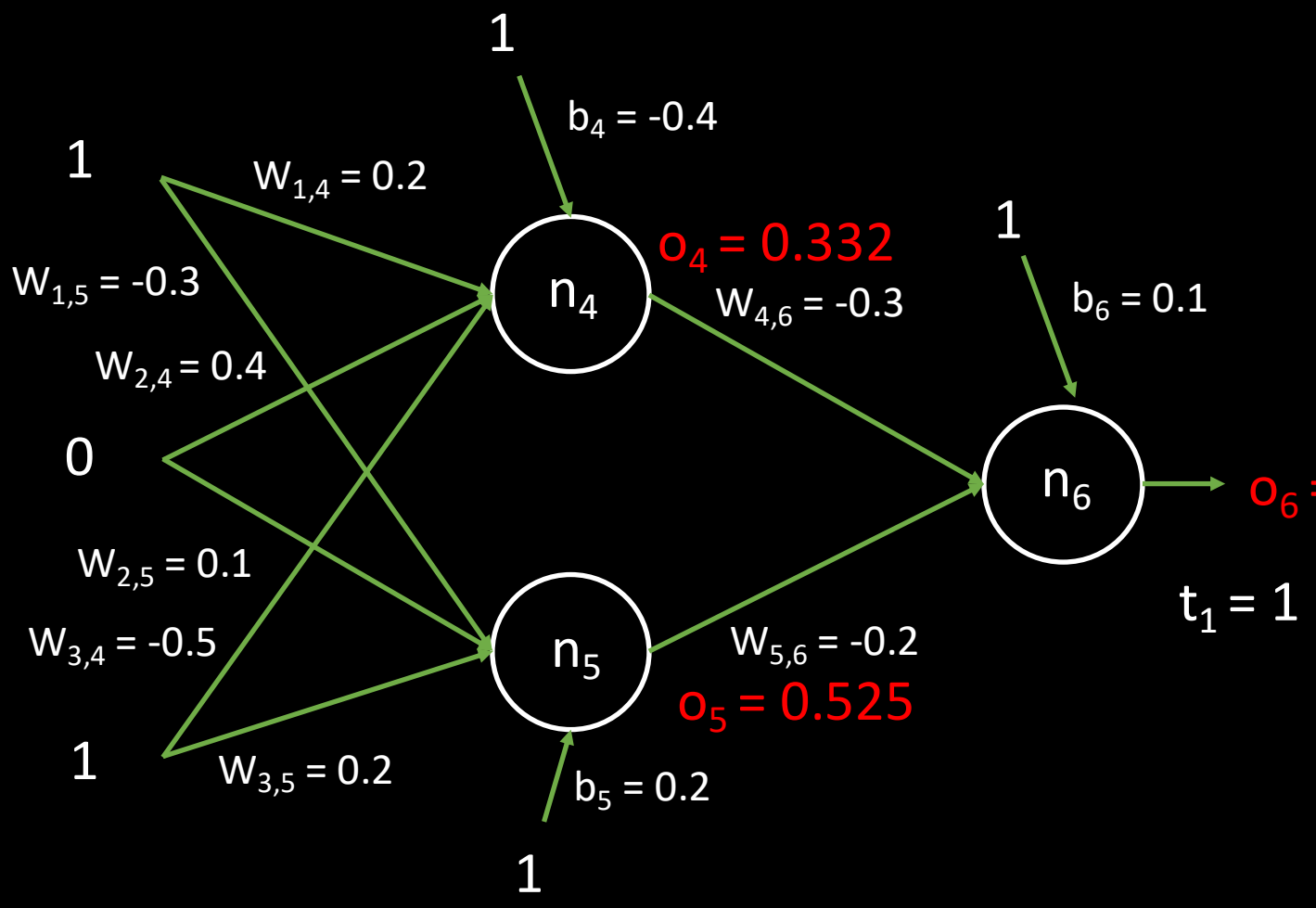
Activation result from node 5:

$o_5 = sigmoid(0.1)$

$o_5 = 1/(1+e^{-0.1})$

$o_5 = 0.525$

Example: Step 1 – Forward Pass



Weighted input sum to node 6:

$i_6 = (0.332 \times -0.3 + 0.525 \times -0.2) + 0.1$

$i_6 = -0.105$

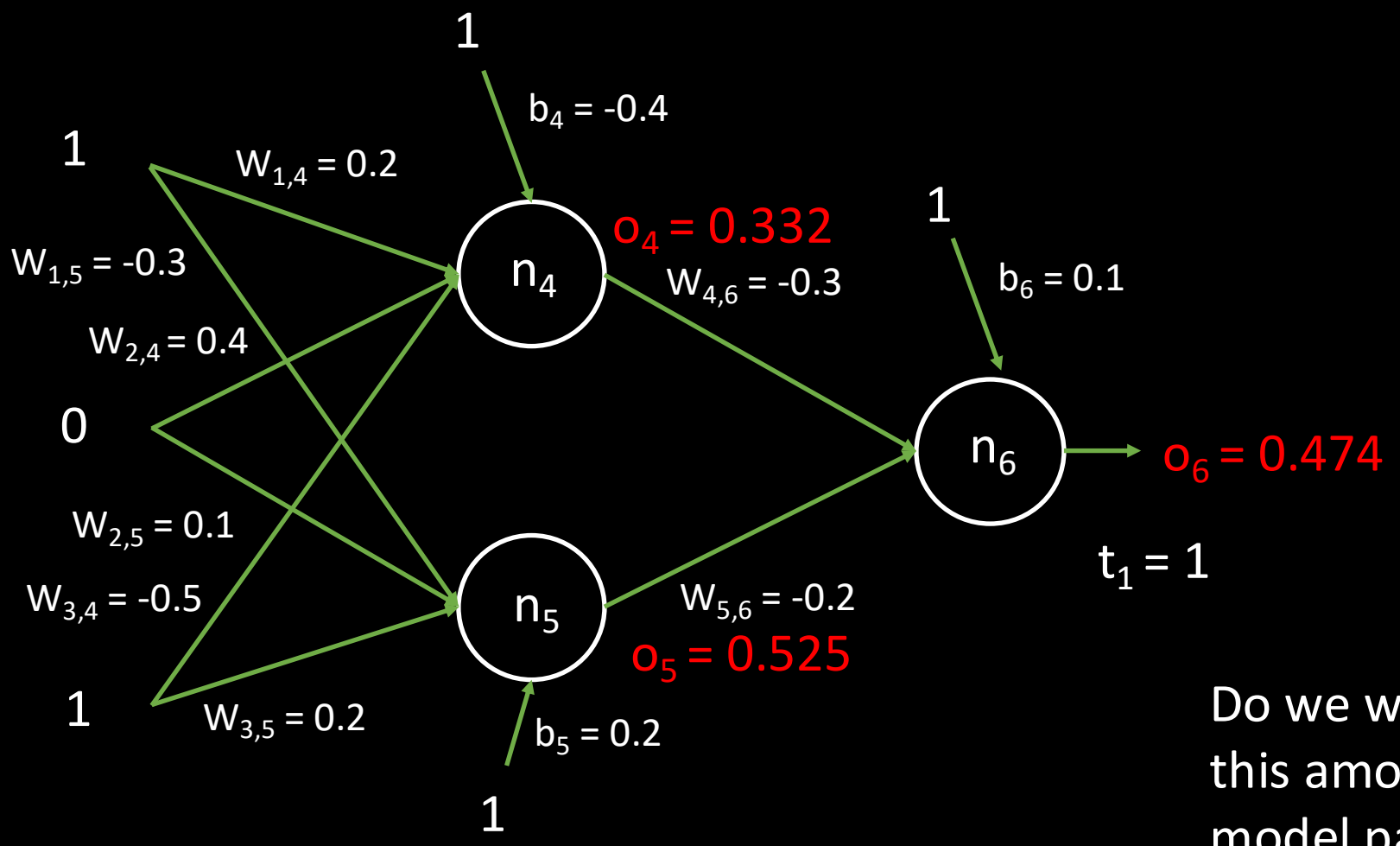
Activation result from node 6:

$o_6 = sigmoid(-0.105)$

$o_6 = 1/(1+e^{-(-0.105)})$

$o_6 = 0.474$

Example: Step 2 – Calculate Error (Loss)



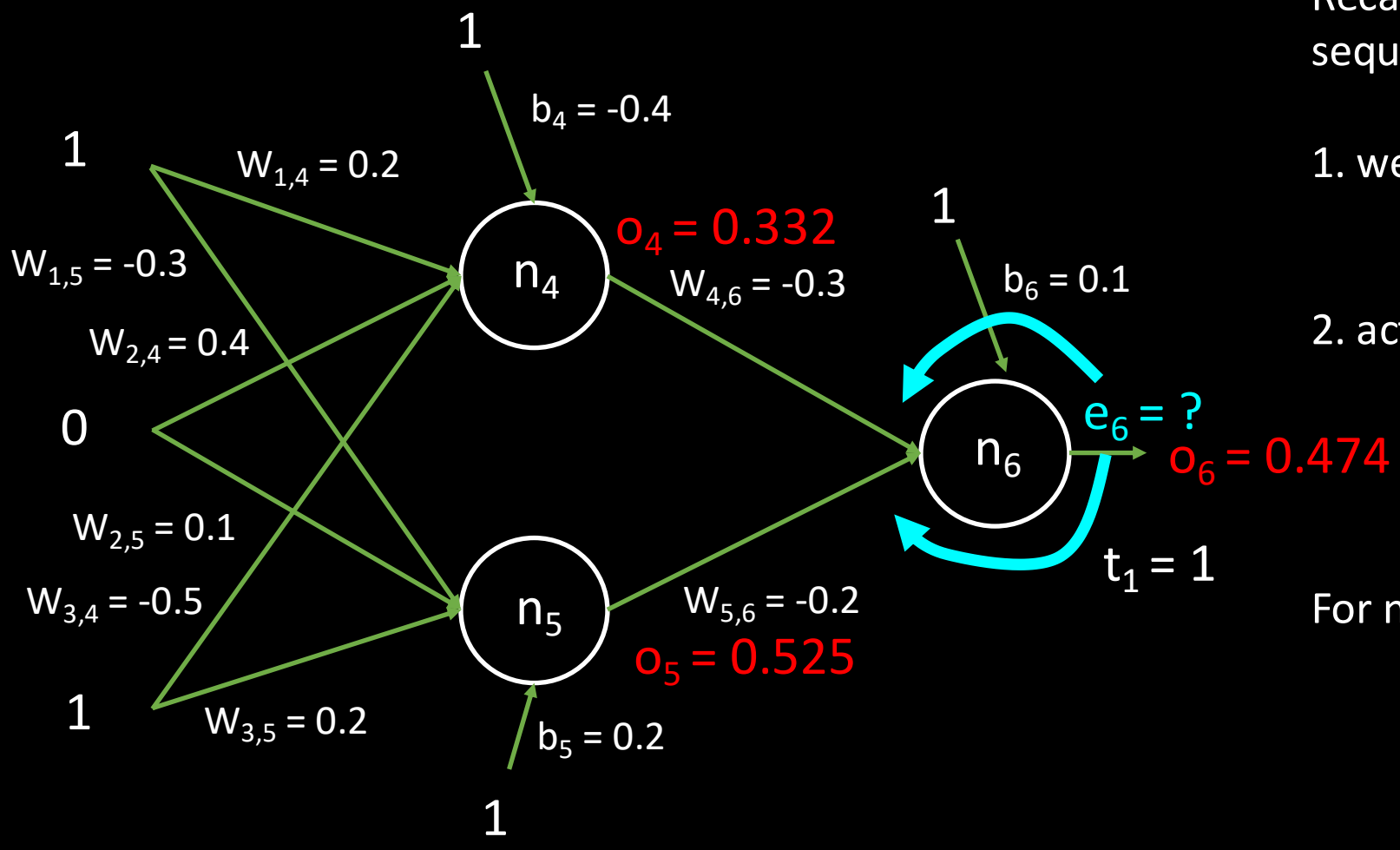
Squared Error

$= (1-0.474)^2$

$= 0.277$

Do we want to increase or reduce this amount when modifying the model parameters during training?

Example: Step 3 – Backward Pass



Recall, each node contains 2 sequential functions:

1. weighted sum: $z_j = \sum_{i=1}^{n} w_{ij} x_i + b_j$

2. activation function: $o_j = \frac{1}{1 + e^{-z}}$

For node 6, we first compute:

$$\frac{\partial E}{\partial z_6} = \frac{\partial E}{\partial o_6} x \frac{\partial o_6}{\partial z_6}$$

Example: Step 3 – Backward Pass

Loss function:

constant value

$$\frac{\partial E}{\partial o_6} = \frac{\partial}{\partial o_6}(t - o_6)^2 = -2(t - o_6)$$

Activation function (sigmoid):

$$\frac{\partial o_6}{\partial z_6} = \frac{\partial}{\partial z_6}\frac{1}{1+e^{-z_6}} = (1- o_6)o_6$$

Putting them together:

$$\frac{\partial E}{\partial z_6} = -2(t - o_6) \times (1- o_6)o_6$$

Can drop constant (does not affect learning)

Recall, each node contains 2 sequential functions:

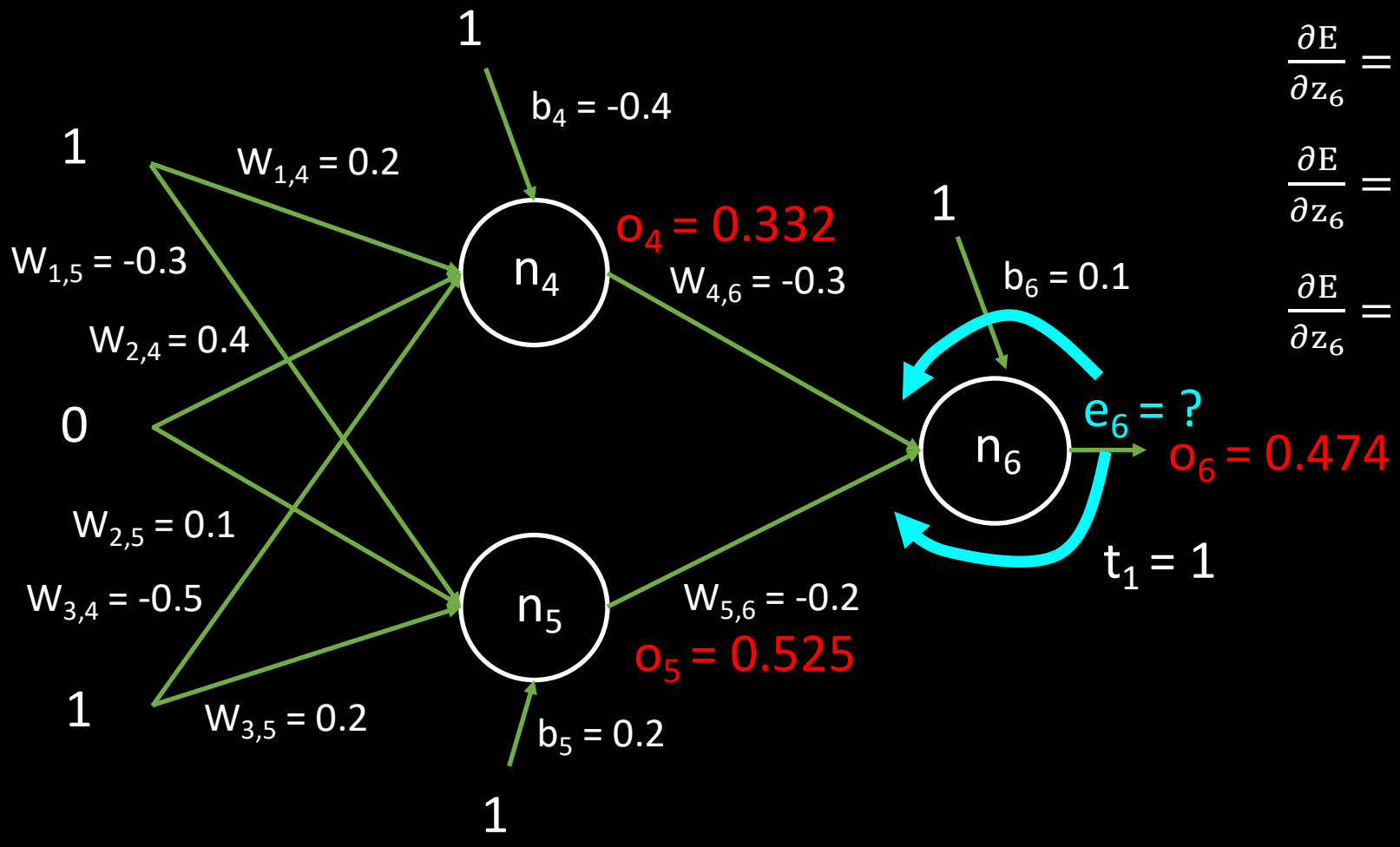
1. weighted sum: $z_j = \sum_{i=1}^{n} w_{ij}x_i + b_j$

2. activation function: $o_j = \frac{1}{1+e^{-z}}$

For node 6, we first compute:

$$\frac{\partial E}{\partial z_6} = \frac{\partial E}{\partial o_6} \times \frac{\partial o_6}{\partial z_6}$$

Example: Step 3 – Backward Pass

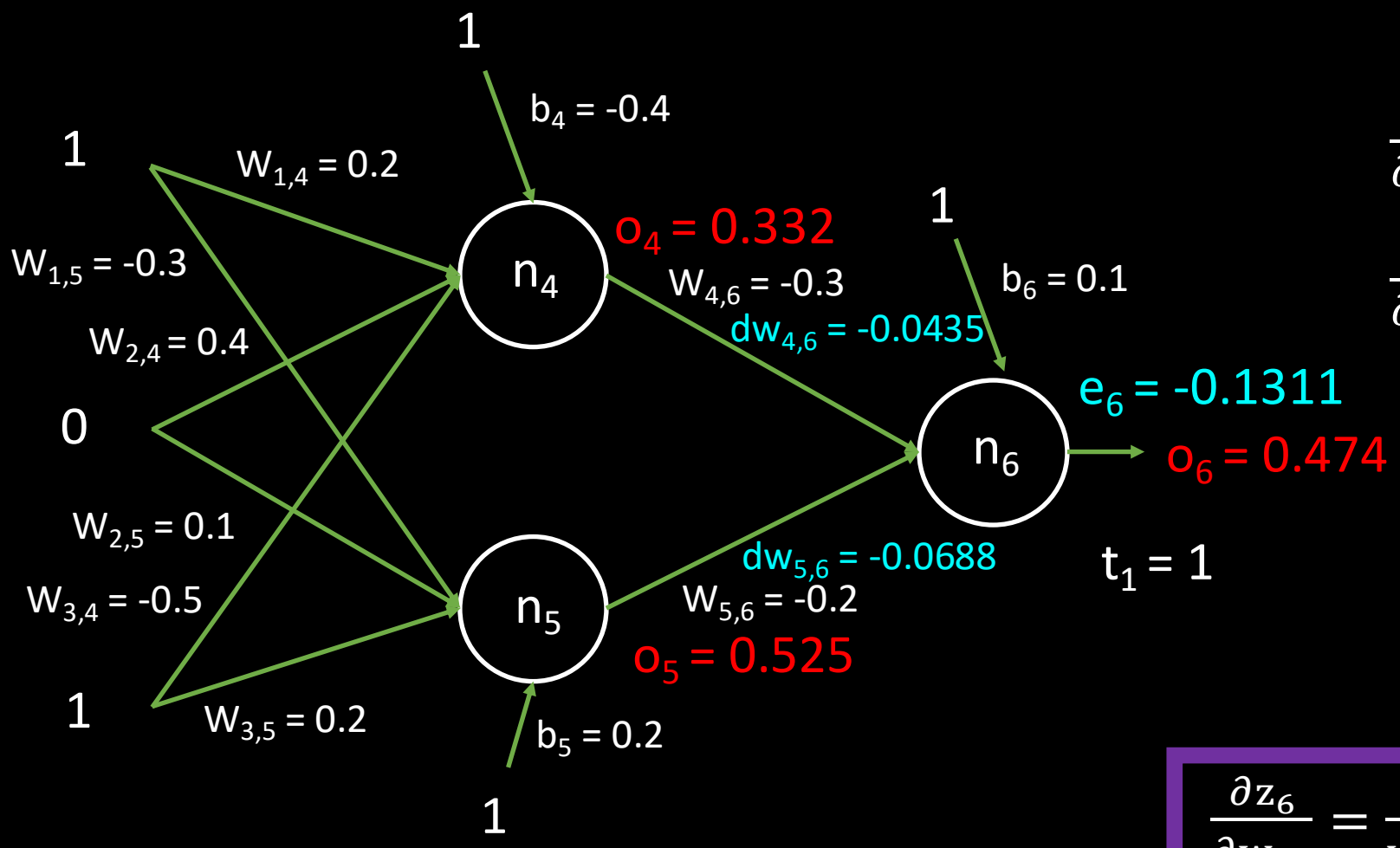


$$\frac{\partial E}{\partial z_6} = -(t - o_6) \text{ x } (1 - o_6)o_6$$

$$\frac{\partial E}{\partial z_6} = -(1 - 0.474) \text{ x } (1 - 0.474)\text{x}0.474$$

$$\frac{\partial E}{\partial z_6} = -0.1311$$

Example: Step 3 – Backward Pass



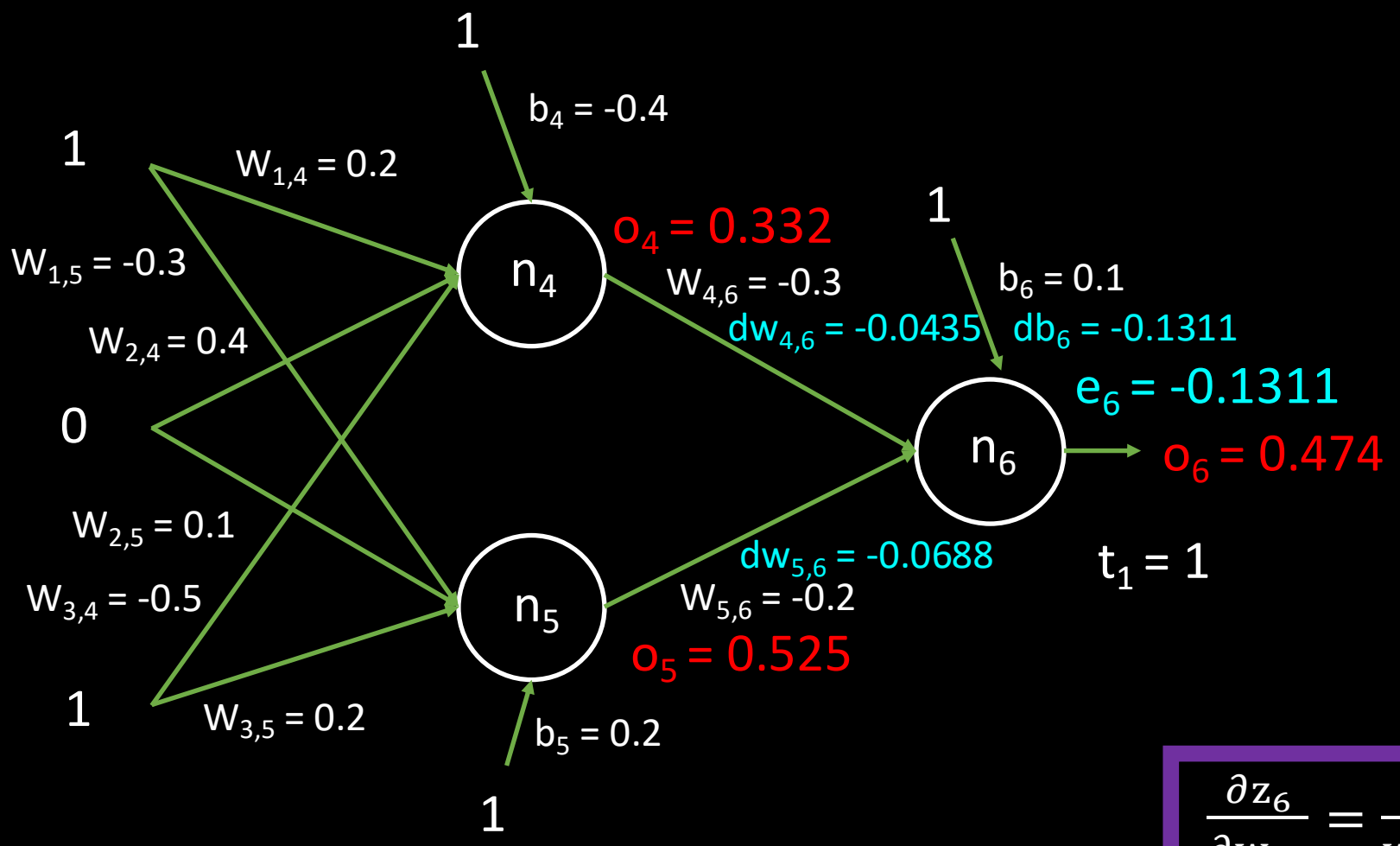
$$\frac{\partial E}{\partial w_{i,6}} = \frac{\partial E}{\partial o_6} \times \frac{\partial o_6}{\partial z_6} \times \frac{\partial z_6}{\partial w_{i,6}}$$

$$\frac{\partial E}{\partial w_{4,6}} = e_6 \times o_4 = -0.0435$$

$$\frac{\partial E}{\partial w_{5,6}} = e_6 \times o_5 = -0.0688$$

$$\frac{\partial z_6}{\partial w_{i,6}} = \frac{\partial}{w_{i,6}} (o_i\, wi_{,6} + ojwj_{,6} + b_6) = o_i$$

Example: Step 3 – Backward Pass

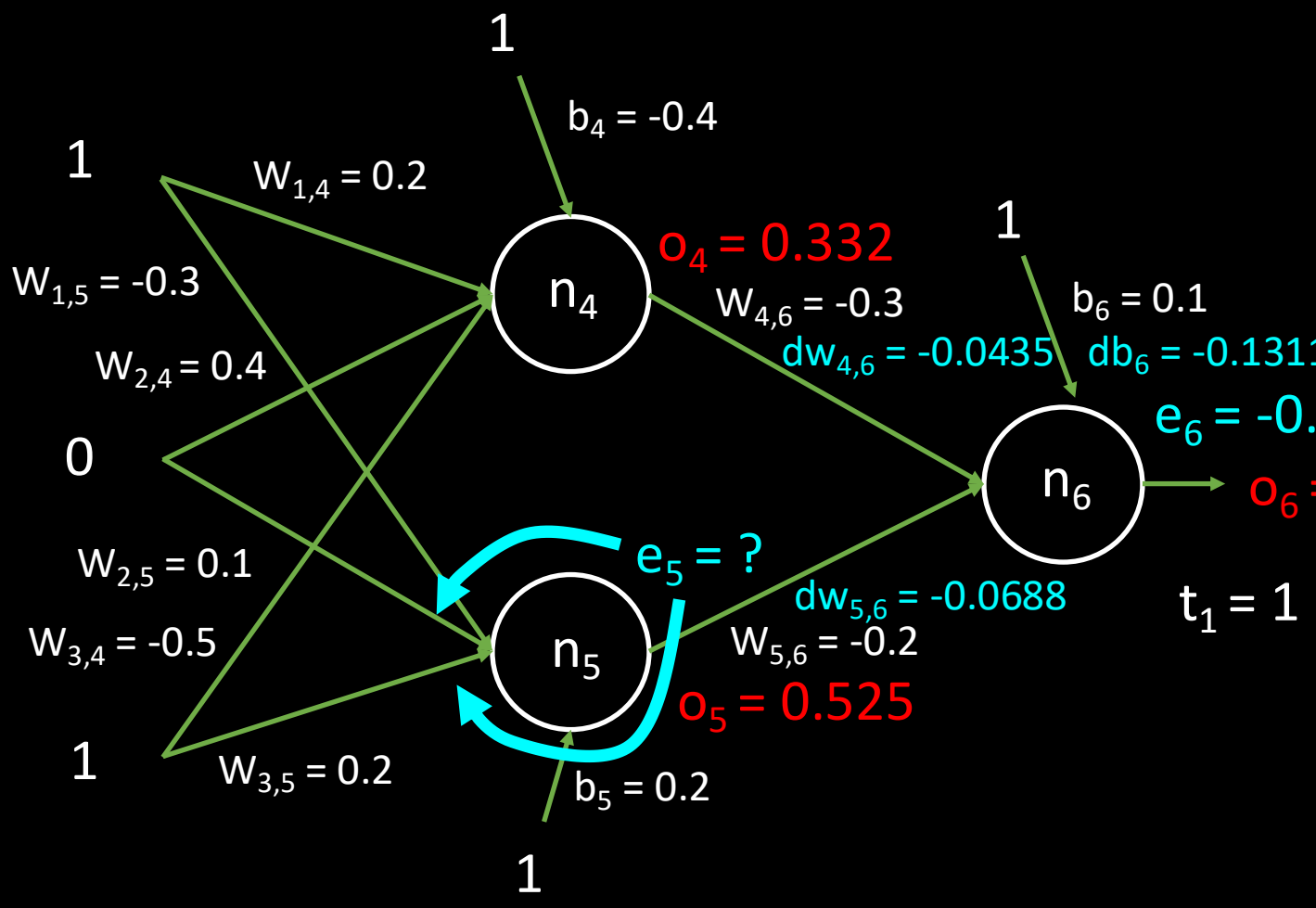


$$\frac{\partial E}{\partial b_6} = \frac{\partial E}{\partial o_6} \times \frac{\partial o_6}{\partial z_6} \times \frac{\partial z_6}{\partial b_6}$$

$$\frac{\partial E}{\partial b_6} = e_6 \times 1 = -0.1311$$

$$\frac{\partial z_6}{\partial w_{i,6}} = \frac{\partial}{w_{i,6}} (o_i\, wi_{,6} + oj\, wj_{,6} + b_6) = 1$$

Example: Step 3 – Backward Pass



Recall, each node contains 2 sequential functions:

1. weighted sum: $z_j = \sum_{i=1}^{n} w_{ij} x_i + b_j$

2. activation function: $o_j = \frac{1}{1 + e^{-z}}$

For node 5, we compute:

$$\frac{\partial E}{\partial z_5} = \frac{\partial E}{\partial o_6} \times \frac{\partial o_6}{\partial z_6} \times \frac{\partial z_6}{\partial o_5} \times \frac{\partial o_5}{\partial z_5}$$

Example: Step 3 – Backward Pass

Parent node:

$$\frac{\partial z_6}{\partial o_5} = \frac{\partial}{o_5}(o_4 w_{4,6} + o_5 w_{5,6} + b_6) = w_{5,6}$$

Activation function (sigmoid):

$$\frac{\partial o_5}{\partial z_5} = \frac{\partial}{\partial z_5}\frac{1}{1+e^{-z_5}} = (1- o_5)o_5$$

Putting them together:

$$\frac{\partial E}{\partial z_5} = e_6 \times w_{5,6} \times (1- o_5)o_5$$

Recall, each node contains 2 sequential functions:

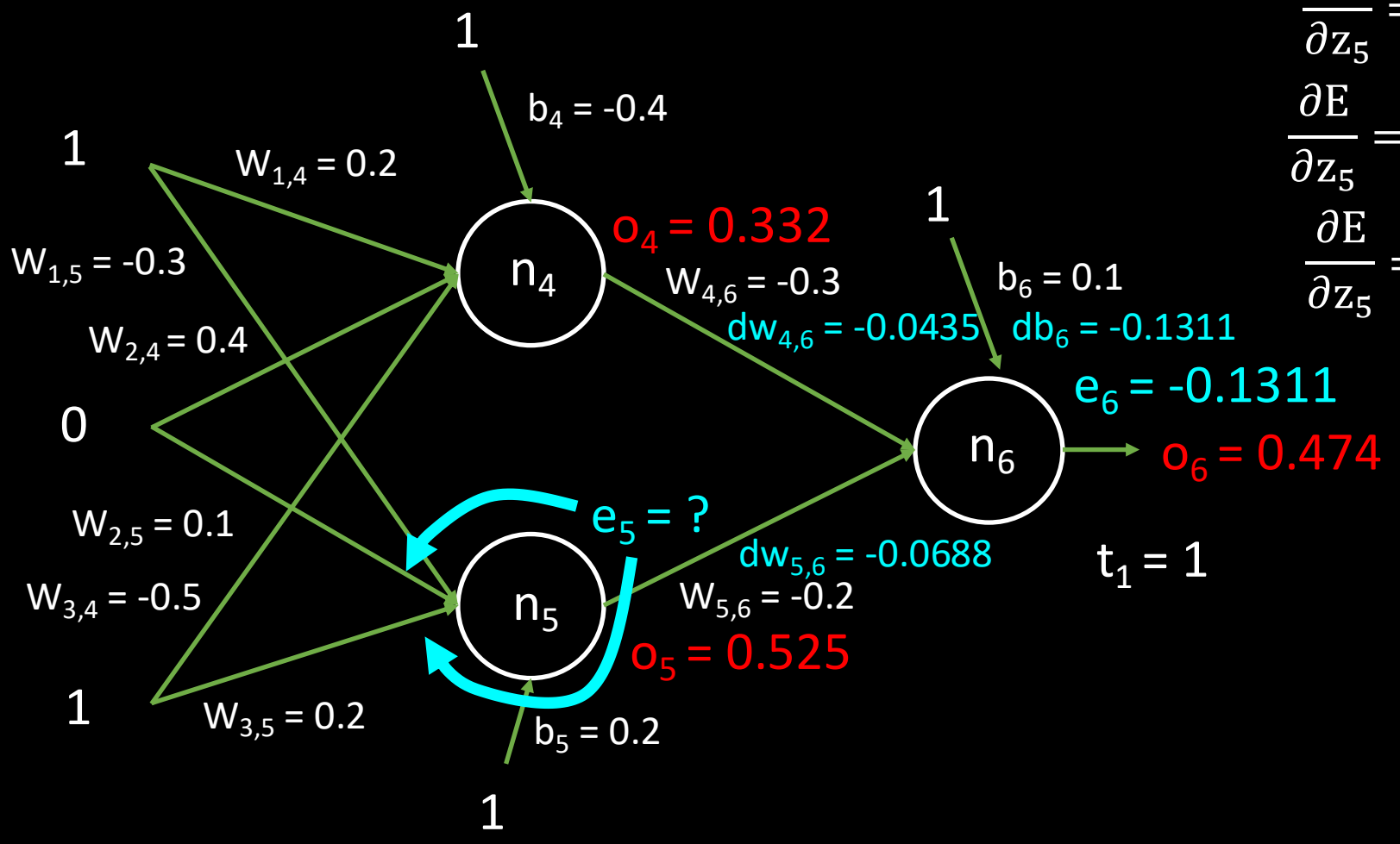
1. weighted sum: $z_j = \sum_{i=1}^{n} w_{ij}x_i + b_j$

2. activation function: $o_j = \frac{1}{1+e^{-z}}$

For node 5, we compute:

$$\frac{\partial E}{\partial z_5} = \frac{\partial E}{\partial o_6} \times \frac{\partial o_6}{\partial z_6} \times \frac{\partial z_6}{\partial o_5} \times \frac{\partial o_5}{\partial z_5}$$

Example: Step 3 – Backward Pass

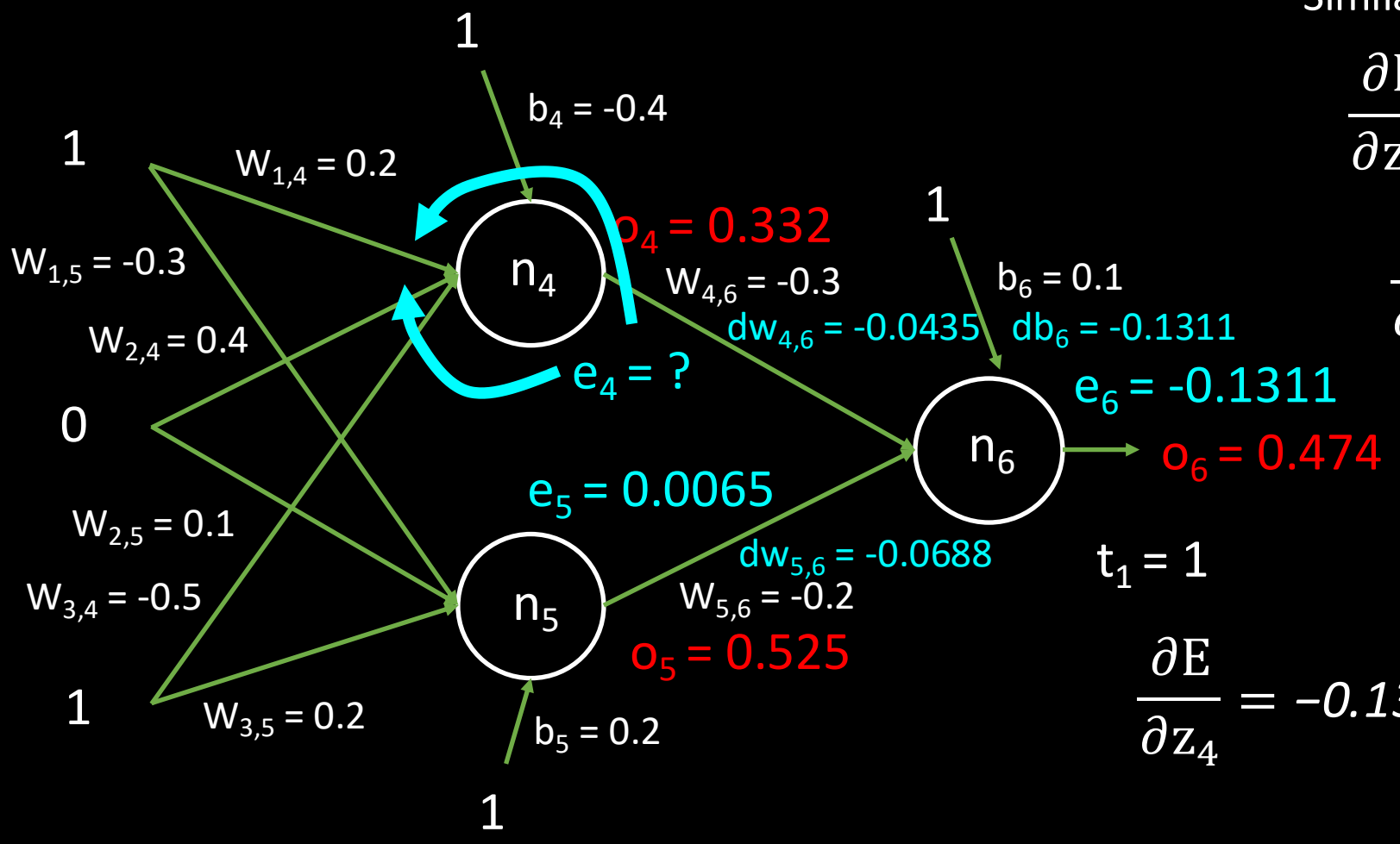


$$\frac{\partial E}{\partial z_5} = e_6 \text{ x } w_{5,6} \text{ x } (1 - o_5) o_5$$

$$\frac{\partial E}{\partial z_5} = -0.1311 \text{ x } -0.2 \text{ x } (1 - 0.525) 0.525$$

$$\frac{\partial E}{\partial z_5} = 0.0065$$

Example: Step 3 – Backward Pass



Similarly, for node 4 we compute:

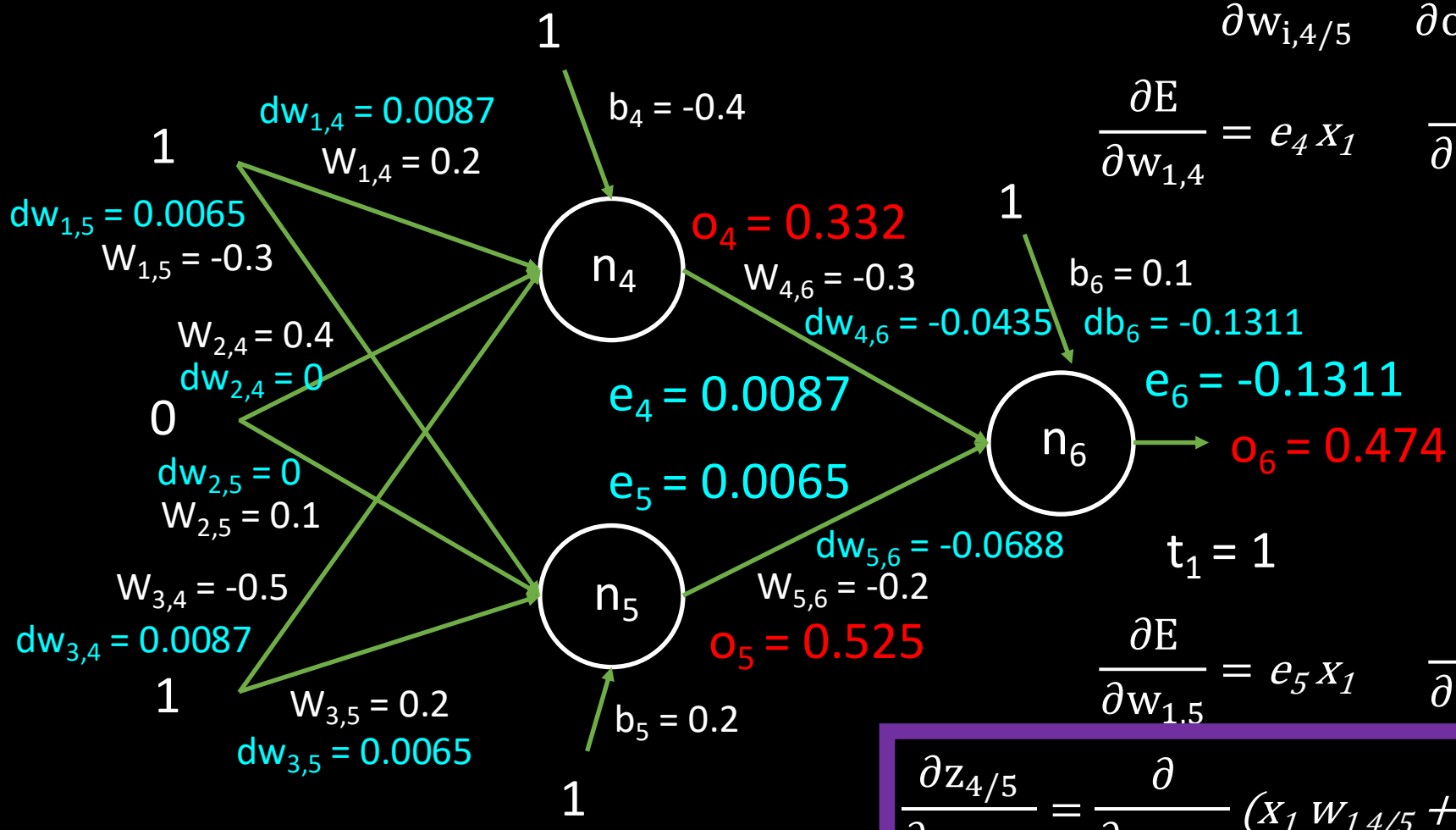
$$\frac{\partial E}{\partial z_4} = \frac{\partial E}{\partial o_6} \text{x} \frac{\partial o_6}{\partial z_4} \text{x} \frac{\partial z_6}{\partial o_4} \text{x} \frac{\partial o_4}{\partial z_4}$$

$$\frac{\partial E}{\partial z_4} = e_6 \text{x}\, w_{4,6} \text{x}\, (1 - o_4) o_4$$

$$\frac{\partial E}{\partial z_4} = -0.1311 \text{x}\, -0.3 \text{x}\, (1 - 0.332) 0.332$$

$$\frac{\partial E}{\partial z_4} = 0.0087$$

Example: Step 3 – Backward Pass



$$\frac{\partial E}{\partial w_{i,4/5}} = \frac{\partial E}{\partial o_6} \times \frac{\partial o_6}{\partial z_6} \times \frac{\partial z_6}{\partial o_{4/5}} \times \frac{\partial o_{4/5}}{\partial z_{4/5}} \times \frac{\partial z_{4/5}}{\partial w_{i,4/5}}$$

$$\frac{\partial E}{\partial w_{1,4}} = e_4 x_1 \qquad \frac{\partial E}{\partial w_{2,4}} = e_4 x_2 \qquad \frac{\partial E}{\partial w_{3,4}} = e_4 x_3$$

$$\frac{\partial E}{\partial w_{1,5}} = e_5 x_1 \qquad \frac{\partial E}{\partial w_{2,5}} = e_5 x_2 \qquad \frac{\partial E}{\partial w_{3,5}} = e_5 x_3$$

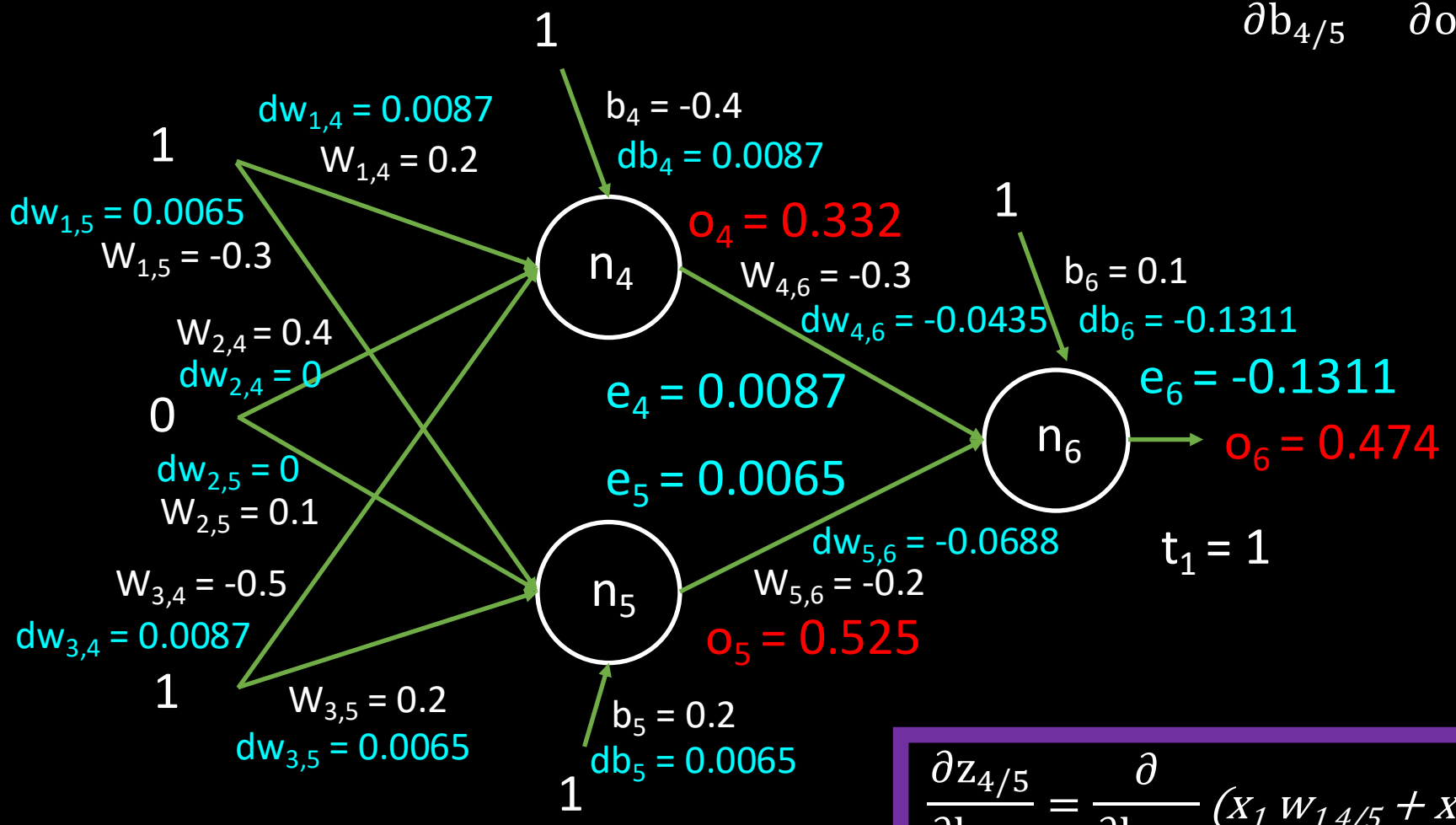
$$\frac{\partial z_{4/5}}{\partial w_{i,4/5}} = \frac{\partial}{\partial w_{i,4/5}} (x_1 w_{1,4/5} + x_2 w_{2,4/5} + x_3 w_{3,4/5} + b_{4/5}) = x_i$$

Example: Step 3 – Backward Pass

$$\frac{\partial E}{\partial b_{4/5}} = \frac{\partial E}{\partial o_6} \times \frac{\partial o_6}{\partial z_6} \times \frac{\partial z_6}{\partial o_{4/5}} \times \frac{\partial o_{4/5}}{\partial z_{4/5}} \times \frac{\partial z_{4/5}}{\partial b_{4/5}}$$

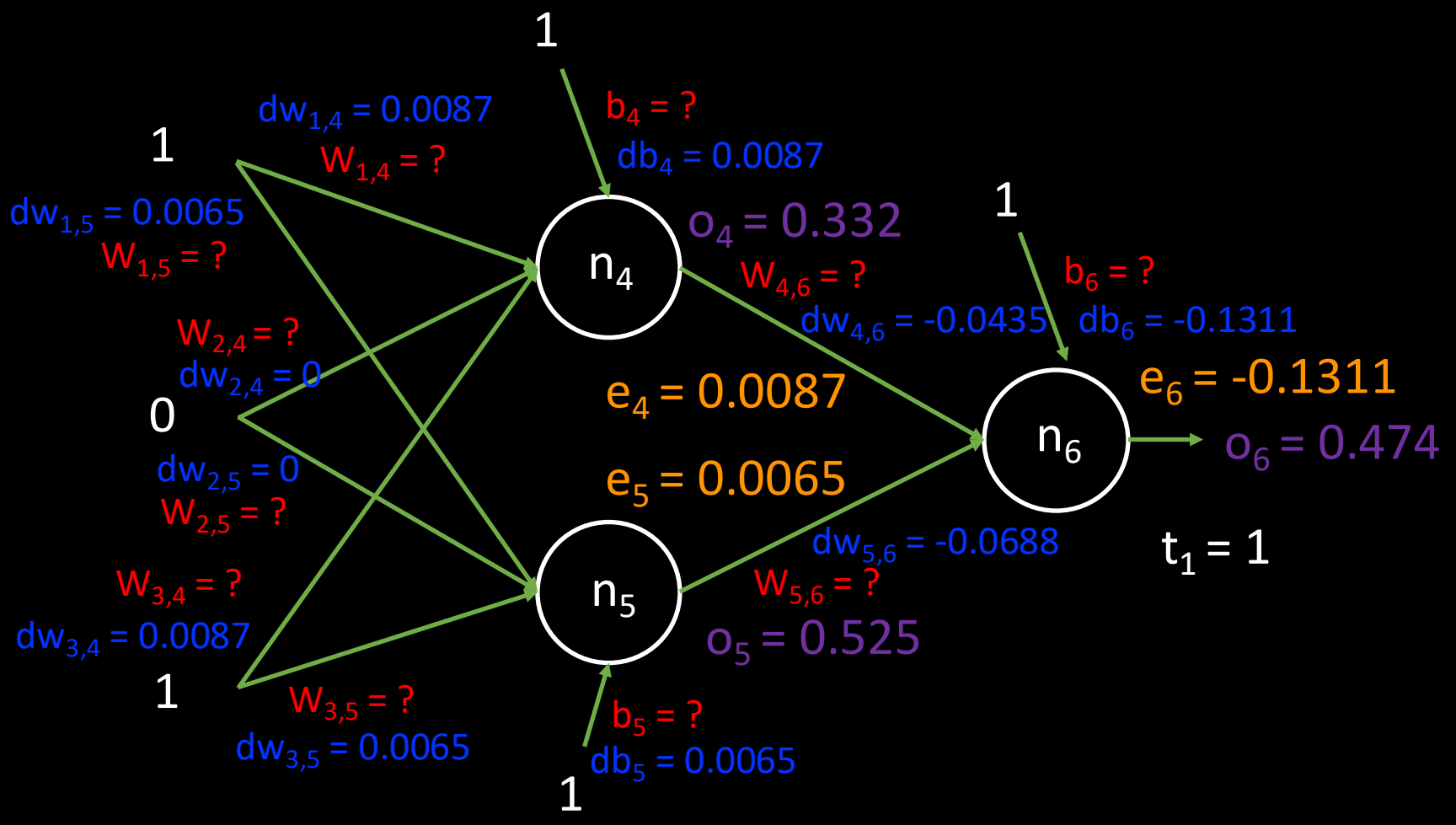
$$\frac{\partial E}{\partial b_4} = e_4$$

$$\frac{\partial E}{\partial b_5} = e_5$$

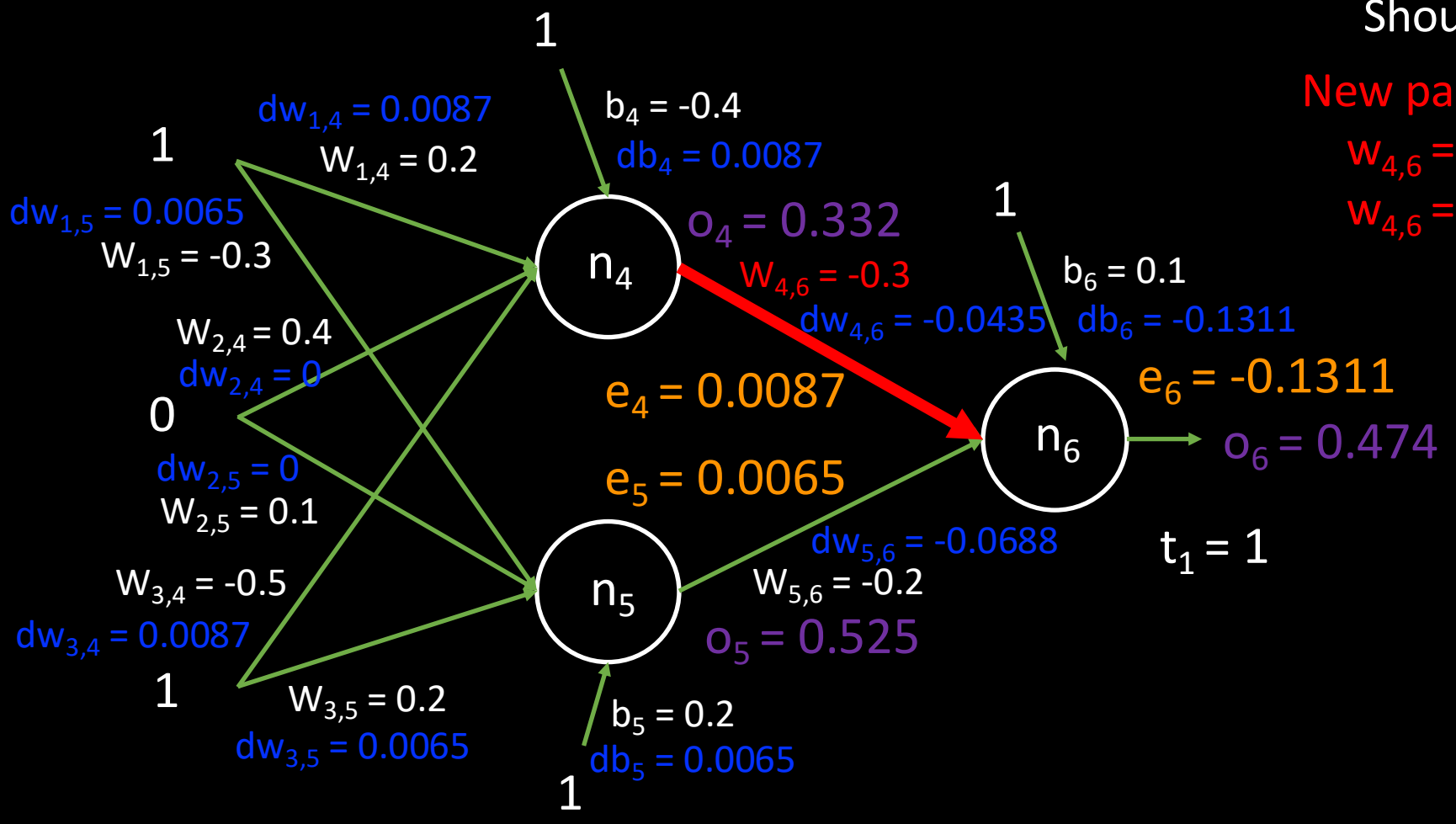


$$\frac{\partial z_{4/5}}{\partial b_{4/5}} = \frac{\partial}{\partial b_{4/5}} (x_1 w_{1,4/5} + x_2 w_{2,4/5} + x_3 w_{3,4/5} + b_{4/5}) = 1$$

Example: Step 4 – Update Weights



Example: Step 4 – Update Weights



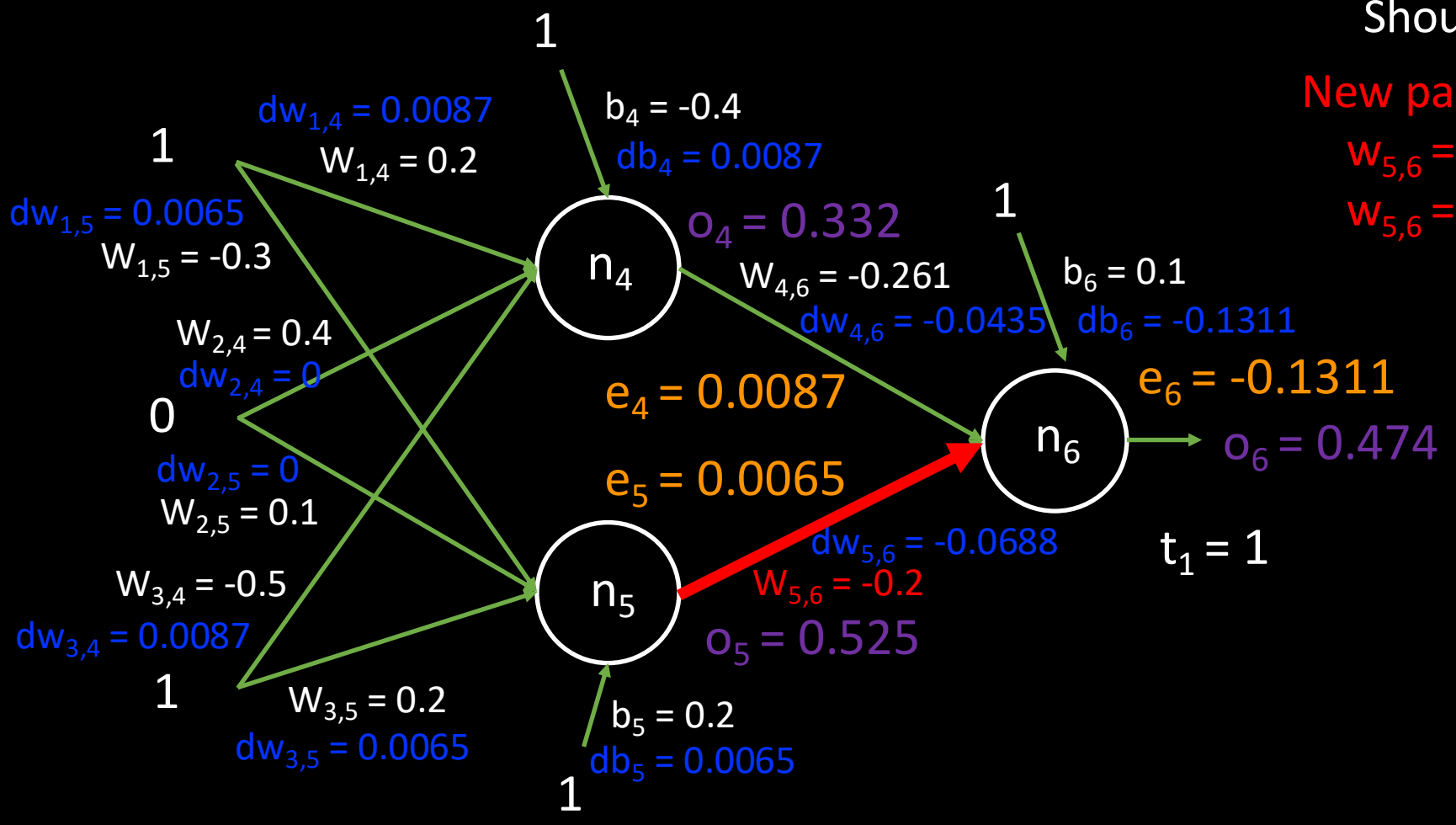
Should $w_{4,6}$ increase or decrease?

New parameters (learning rate = 0.9):

$w_{4,6}$ = -0.3 - 0.9 * -0.0435

$w_{4,6}$ = -0.261

Example: Step 4 – Update Weights



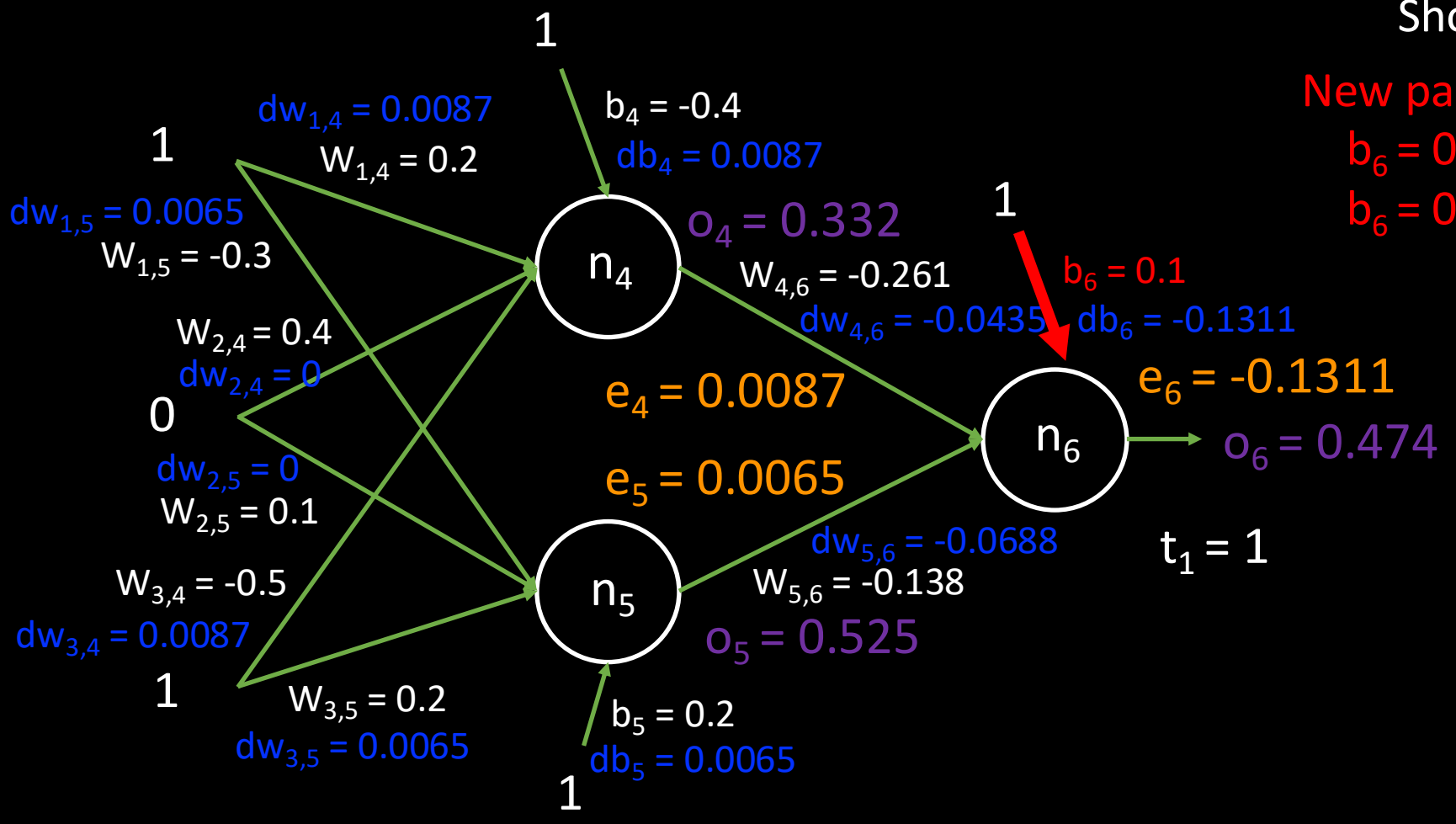
Should $w_{5,6}$ increase or decrease?

New parameters (learning rate = 0.9):

$w_{5,6}$ = -0.2 - 0.9 * -0.0688

$w_{5,6}$ = -0.138

Example: Step 4 – Update Weights



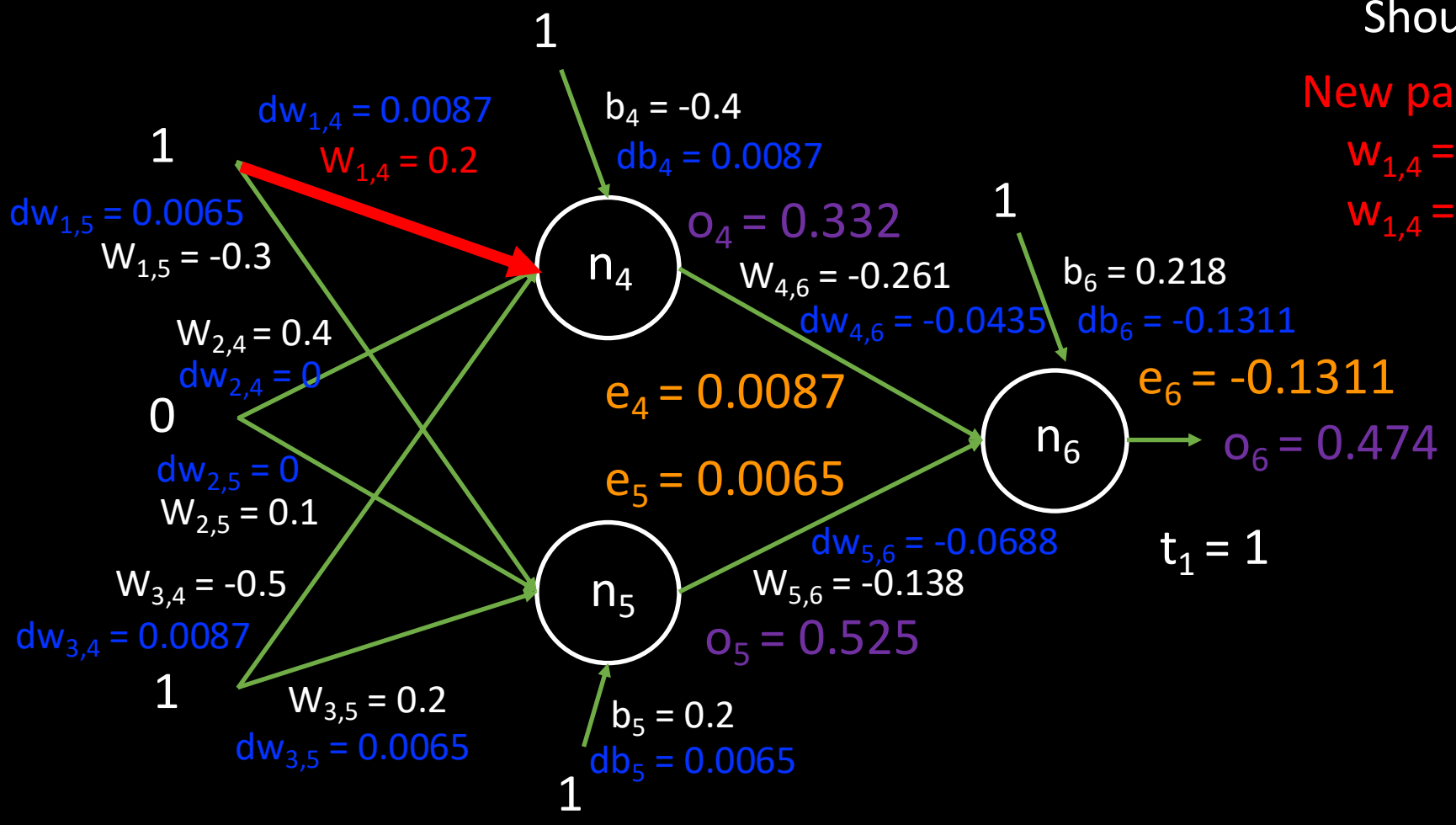
Should b_6 increase or decrease?

New parameters (learning rate = 0.9):

$b_6 = 0.1 - 0.9 * -0.1311$

$b_6 = 0.218$

Example: Step 4 – Update Weights



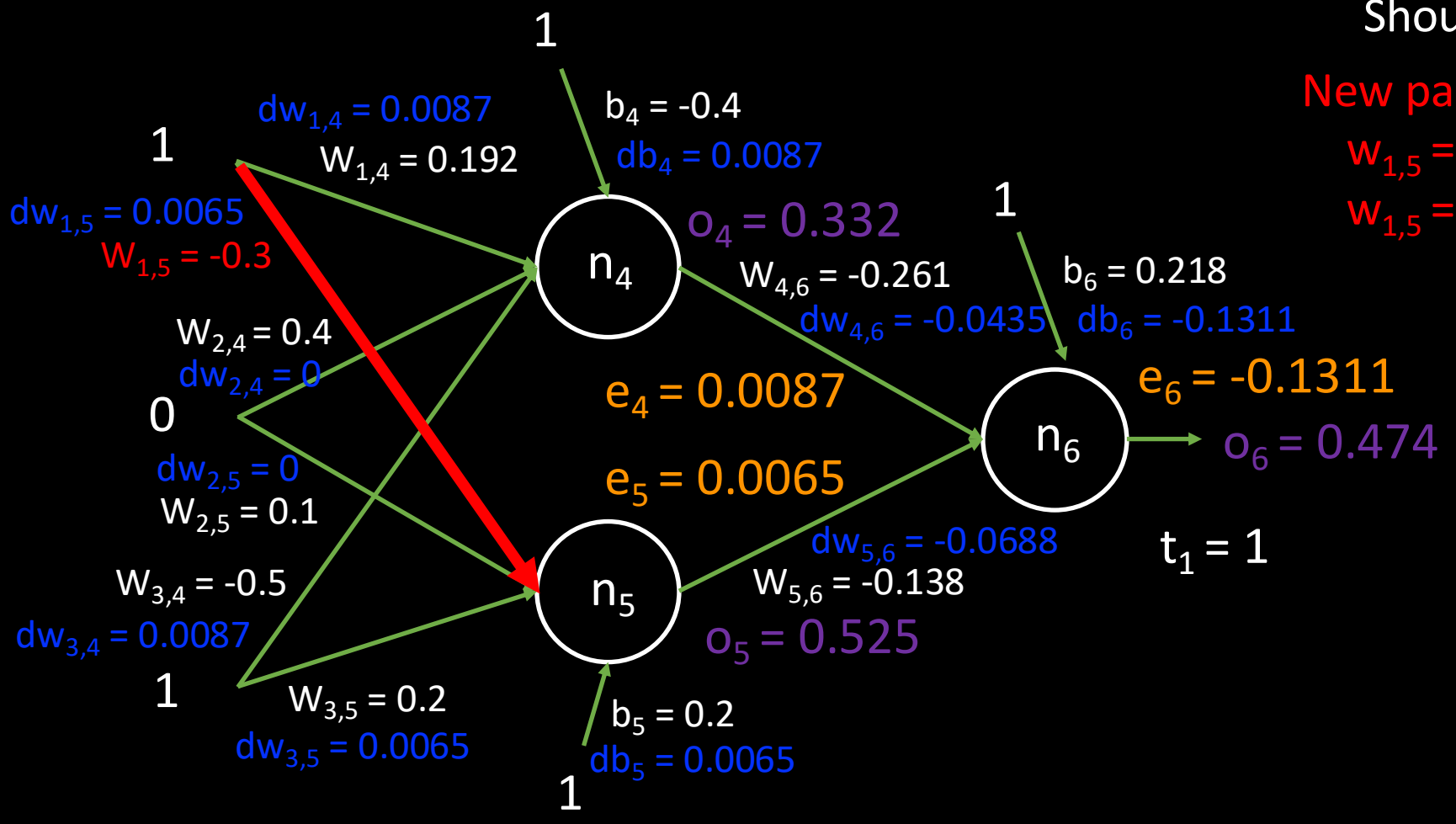
Should $w_{1,4}$ increase or decrease?

New parameters (learning rate = 0.9):

$w_{1,4} = 0.2 - 0.9 * 0.0087$

$w_{1,4} = 0.192$

Example: Step 4 – Update Weights



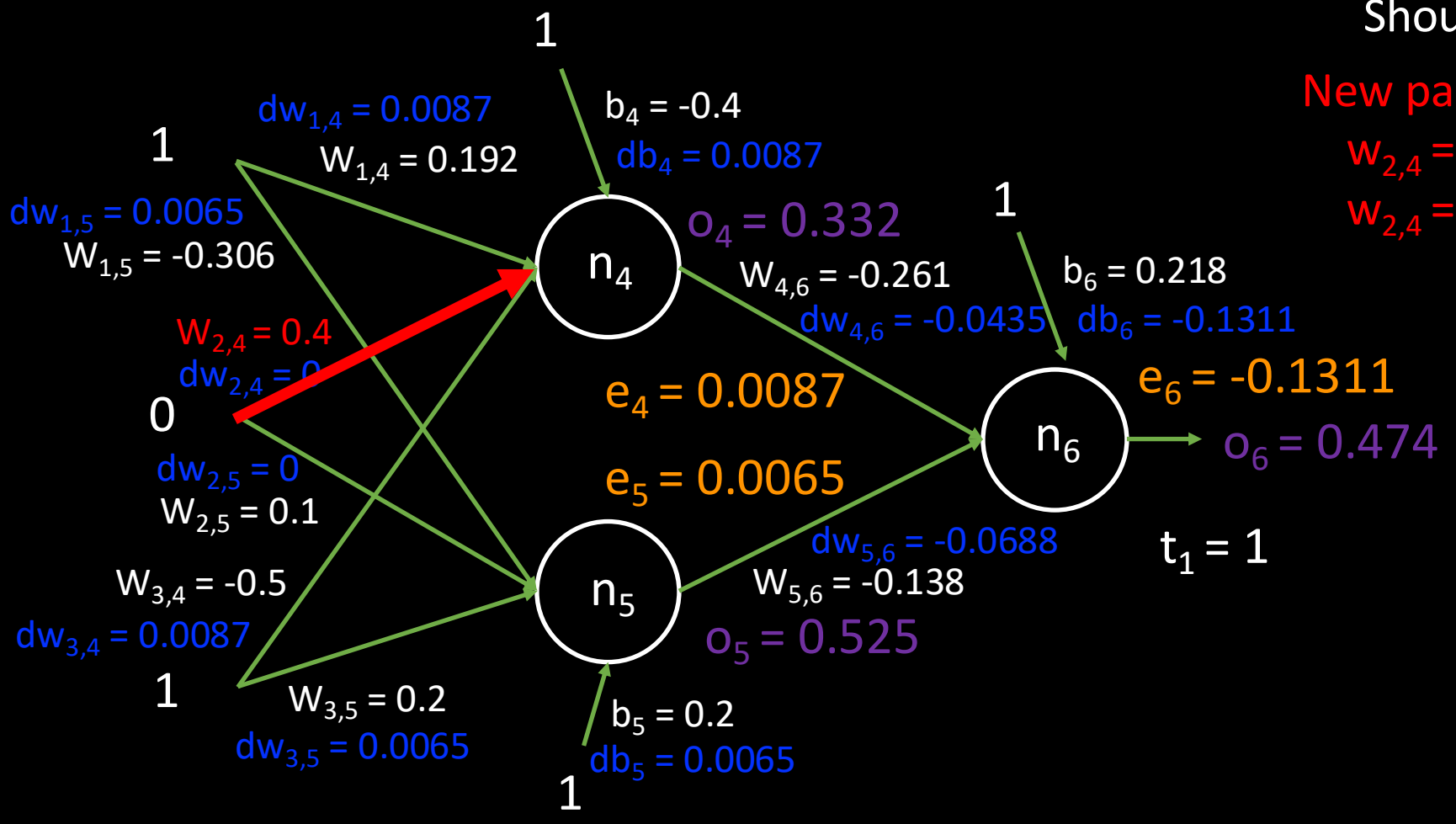
Should $w_{1,5}$ increase or decrease?

New parameters (learning rate = 0.9):

$w_{1,5} = -0.3 - 0.9 * 0.0065$

$w_{1,5} = -0.306$

Example: Step 4 – Update Weights



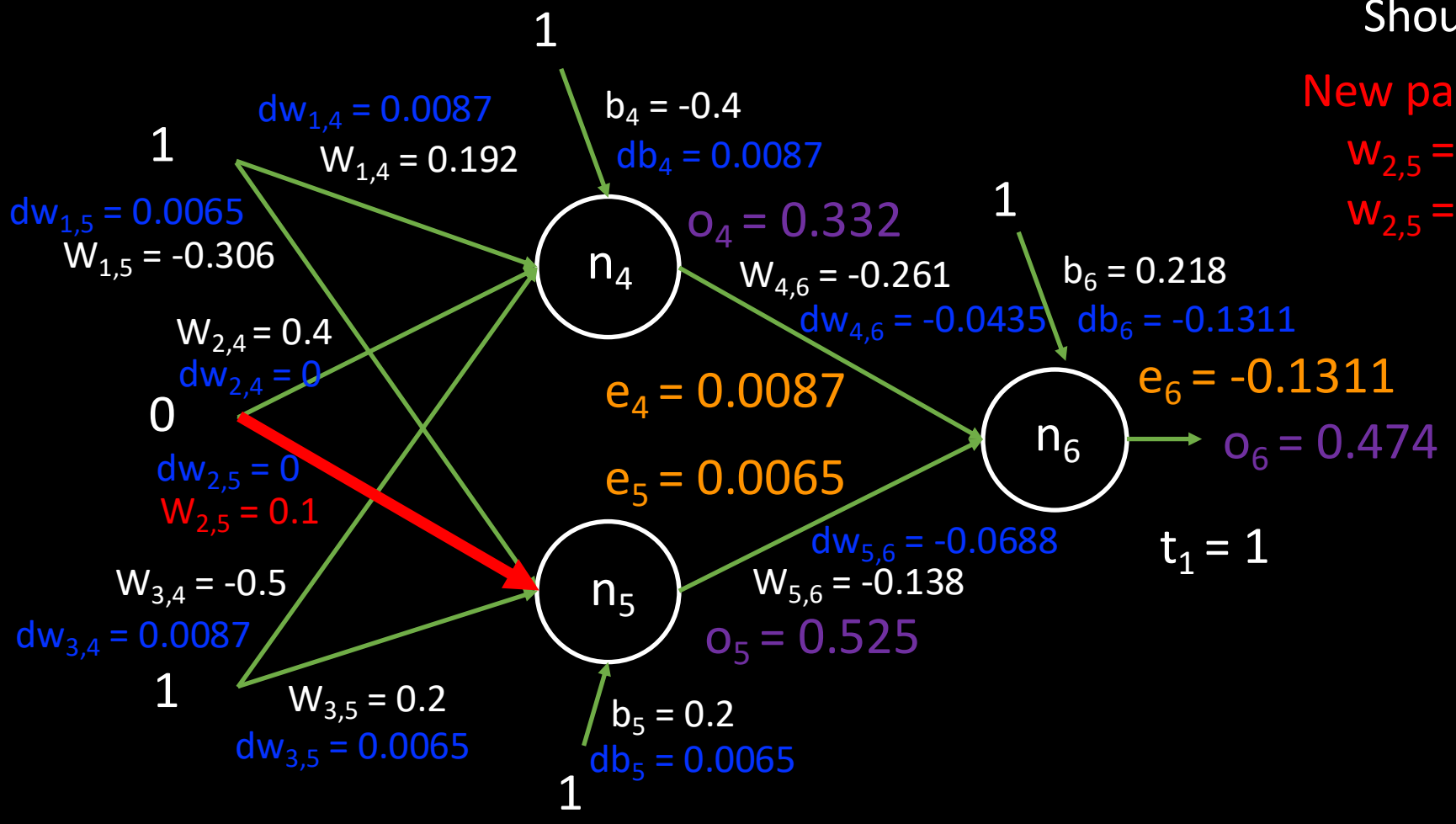
Should $w_{2,4}$ increase or decrease?

New parameters (learning rate = 0.9):

$w_{2,4} = 0.4 - 0.9 * 0$

$w_{2,4} = 0.4$

Example: Step 4 – Update Weights



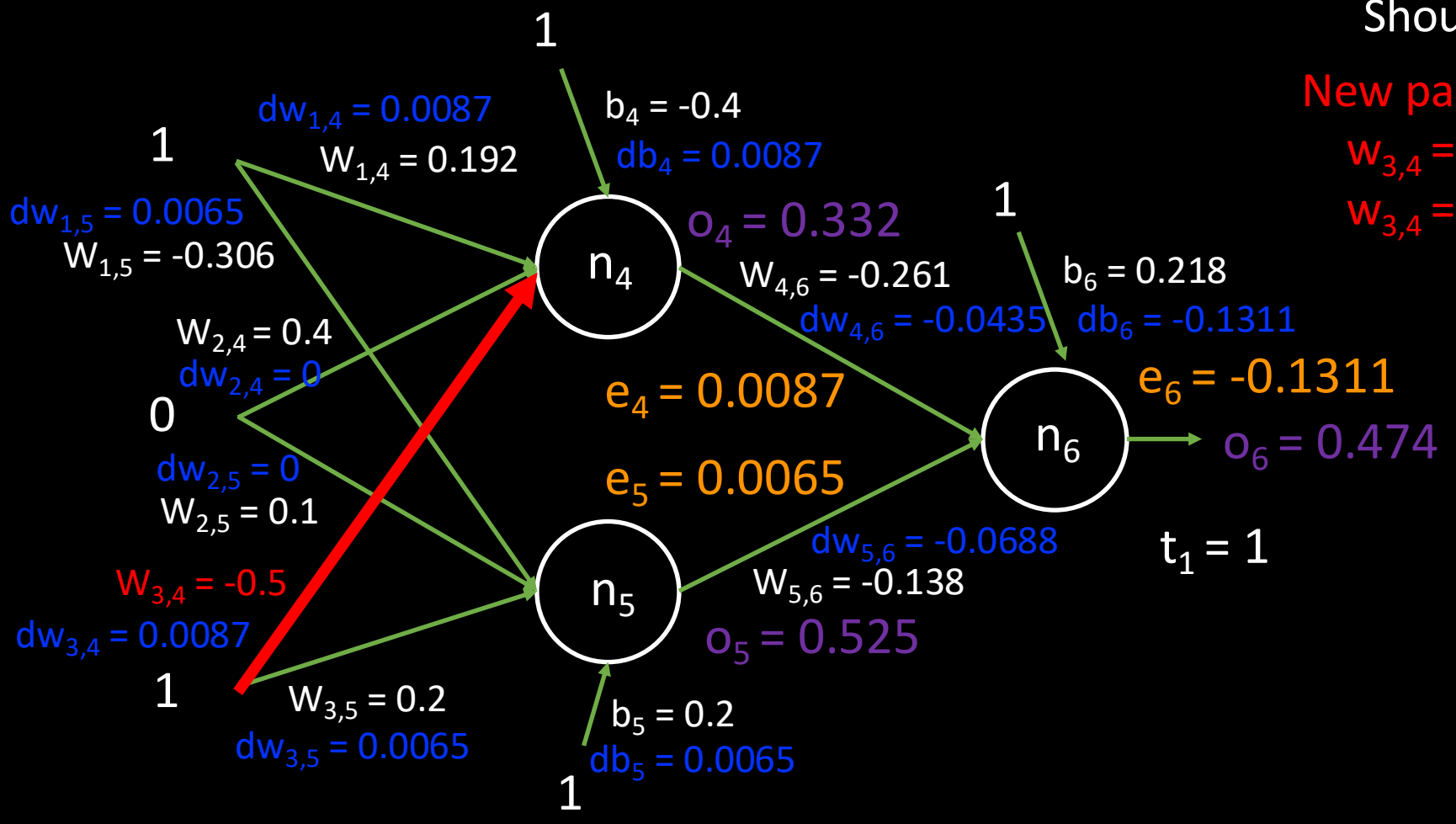
Should $w_{2,5}$ increase or decrease?

New parameters (learning rate = 0.9):

$w_{2,5} = 0.1 - 0.9 * 0$

$w_{2,5} = 0.1$

Example: Step 4 – Update Weights



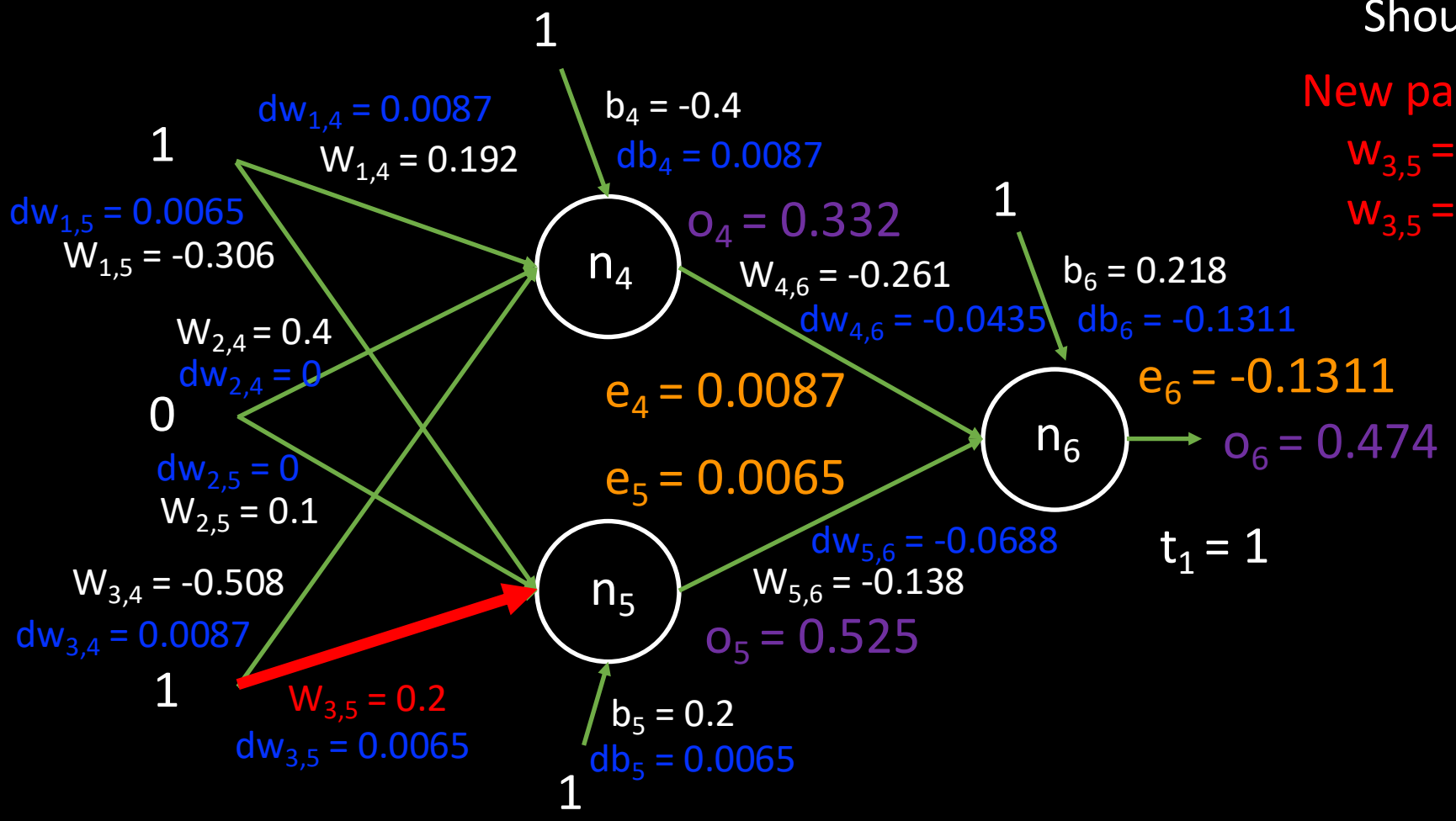
Should $w_{3,4}$ increase or decrease?

New parameters (learning rate = 0.9):

$w_{3,4} = -0.5 - 0.9 * 0.0087$

$w_{3,4} = -0.508$

Example: Step 4 – Update Weights



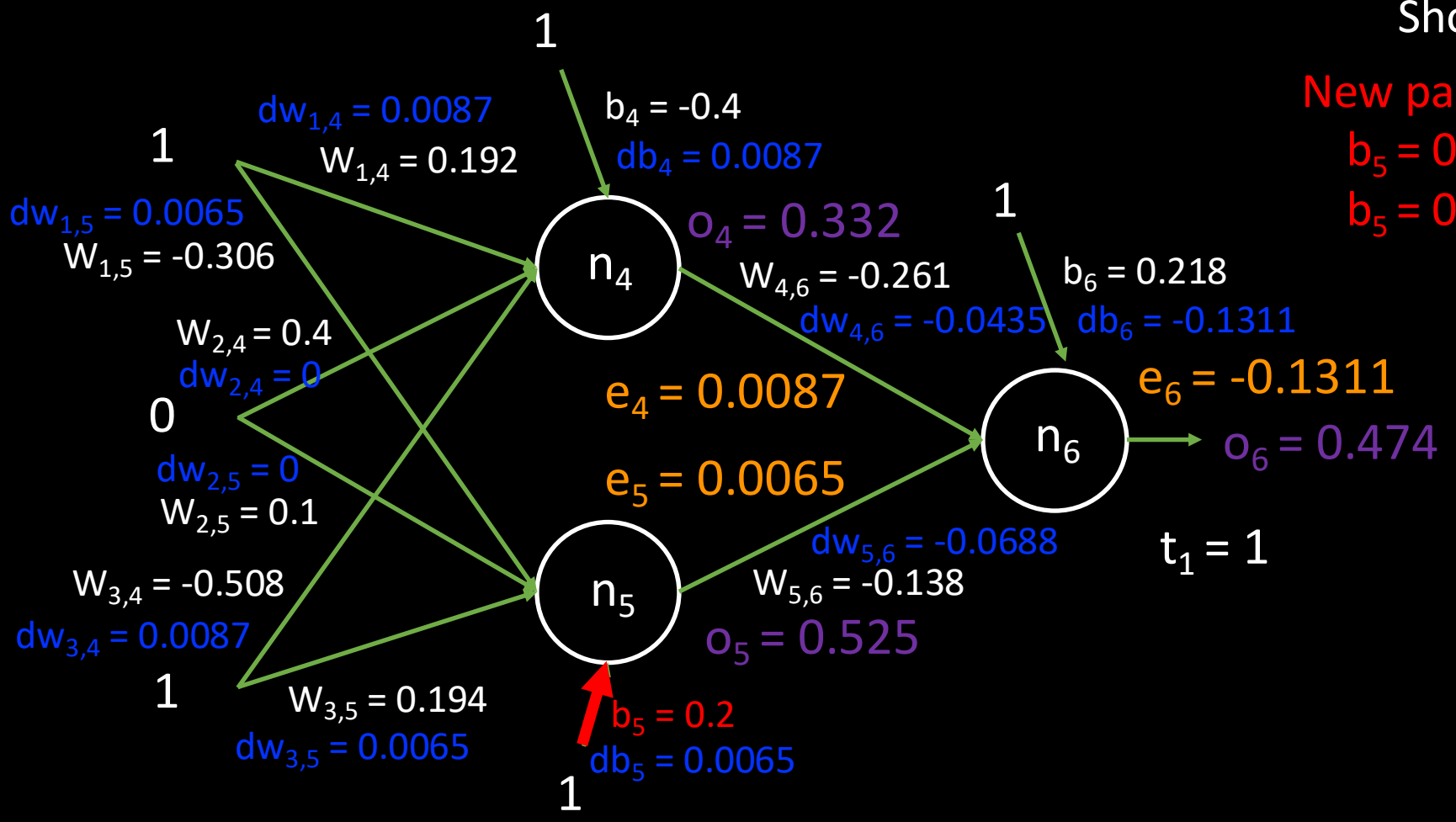
Should $w_{3,5}$ increase or decrease?

New parameters (learning rate = 0.9):

$w_{3,5} = 0.2 - 0.9 * 0.0065$

$w_{3,5} = 0.194$

Example: Step 4 – Update Weights



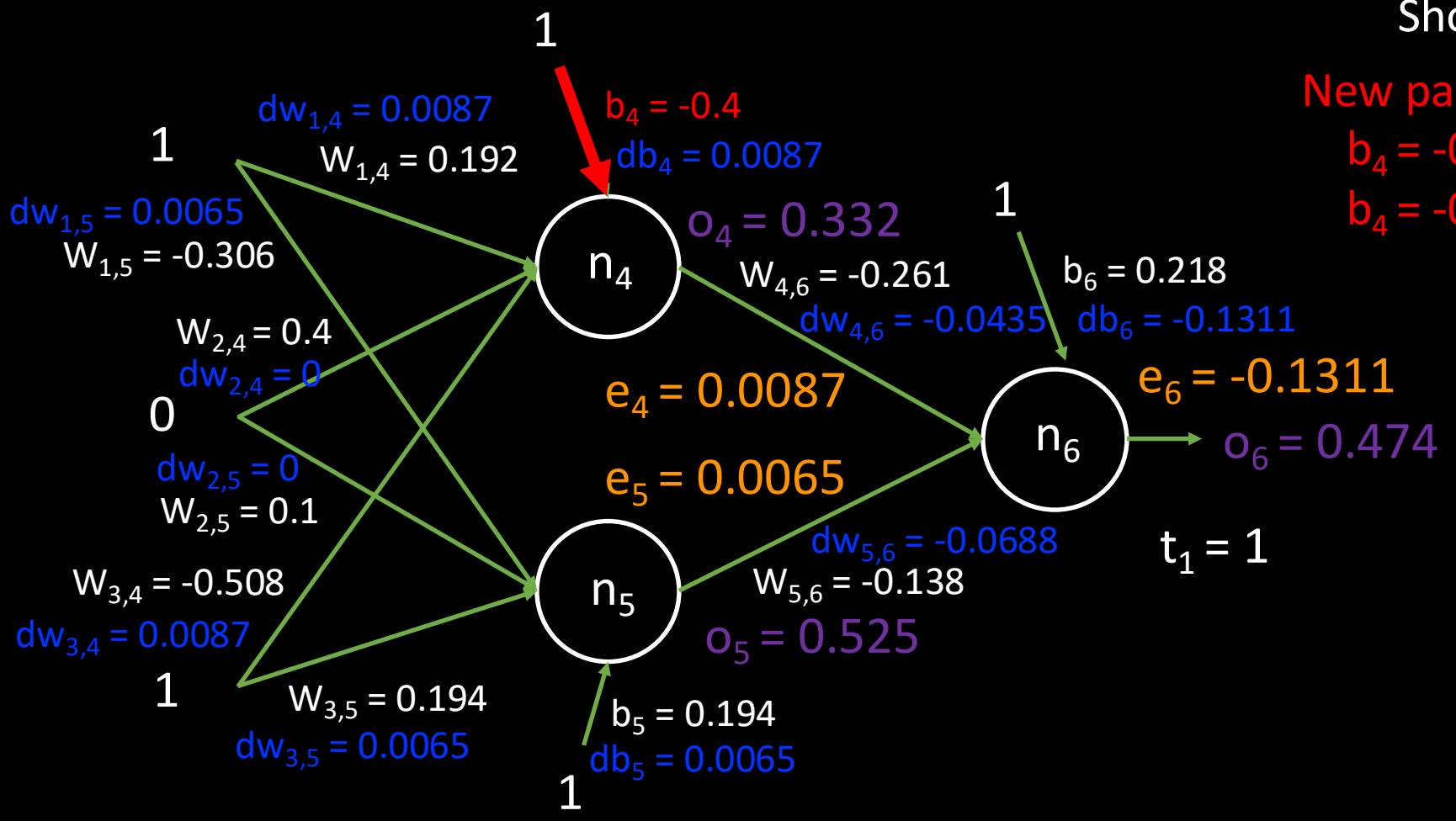
Should b_5 increase or decrease?

New parameters (learning rate = 0.9):

$b_5 = 0.2 - 0.9 * 0.0065$

$b_5 = 0.194$

Example: Step 4 – Update Weights



Should b_4 increase or decrease?

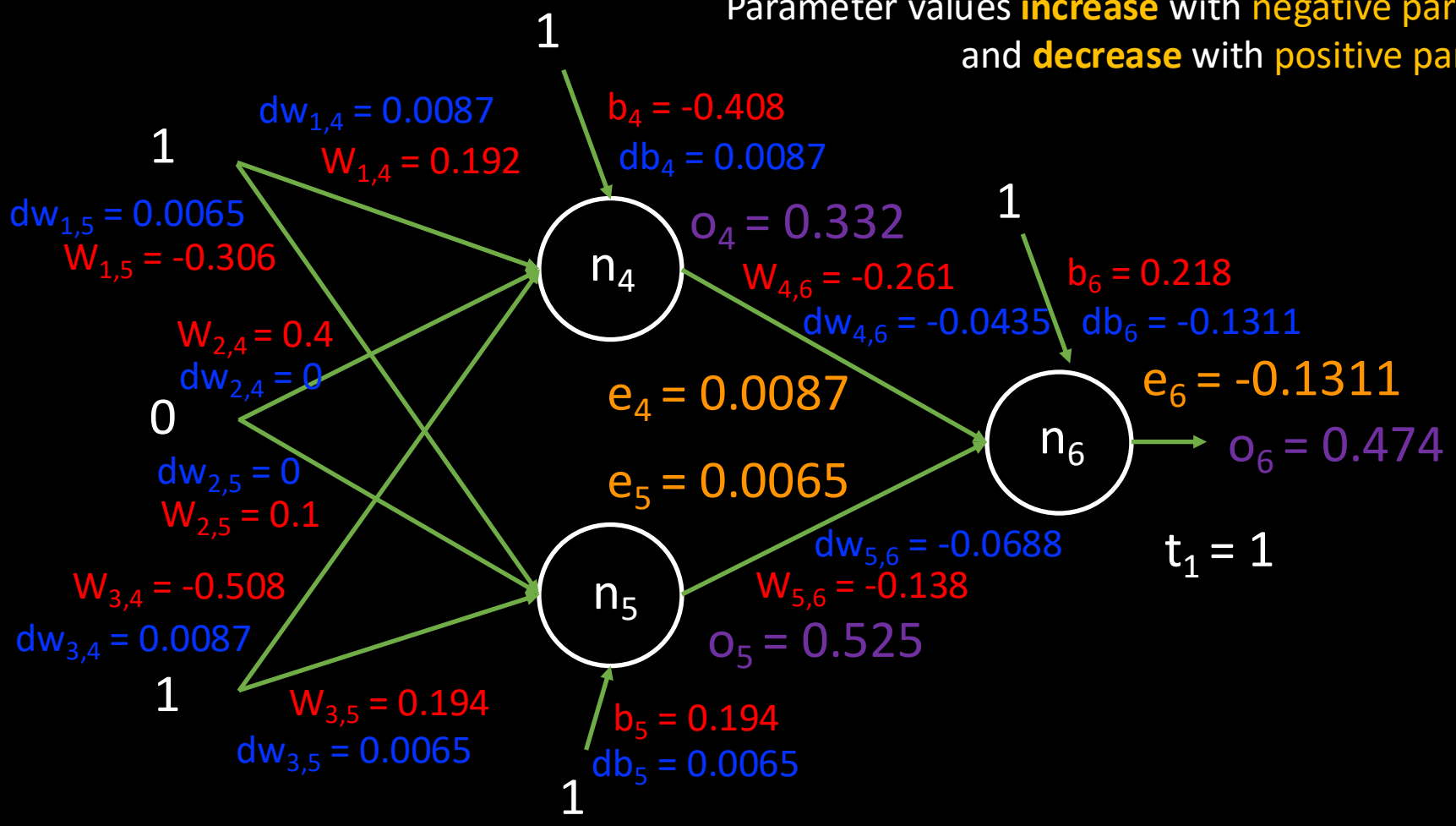
New parameters (learning rate = 0.9):

$b_4 = -0.4 - 0.9 * 0.0087$

$b_4 = -0.408$

Example: Repeat Steps 1-4 With New Examples

Parameter values **increase** with negative partial derivative (become less negative)
and **decrease** with positive partial derivative (become less positive)

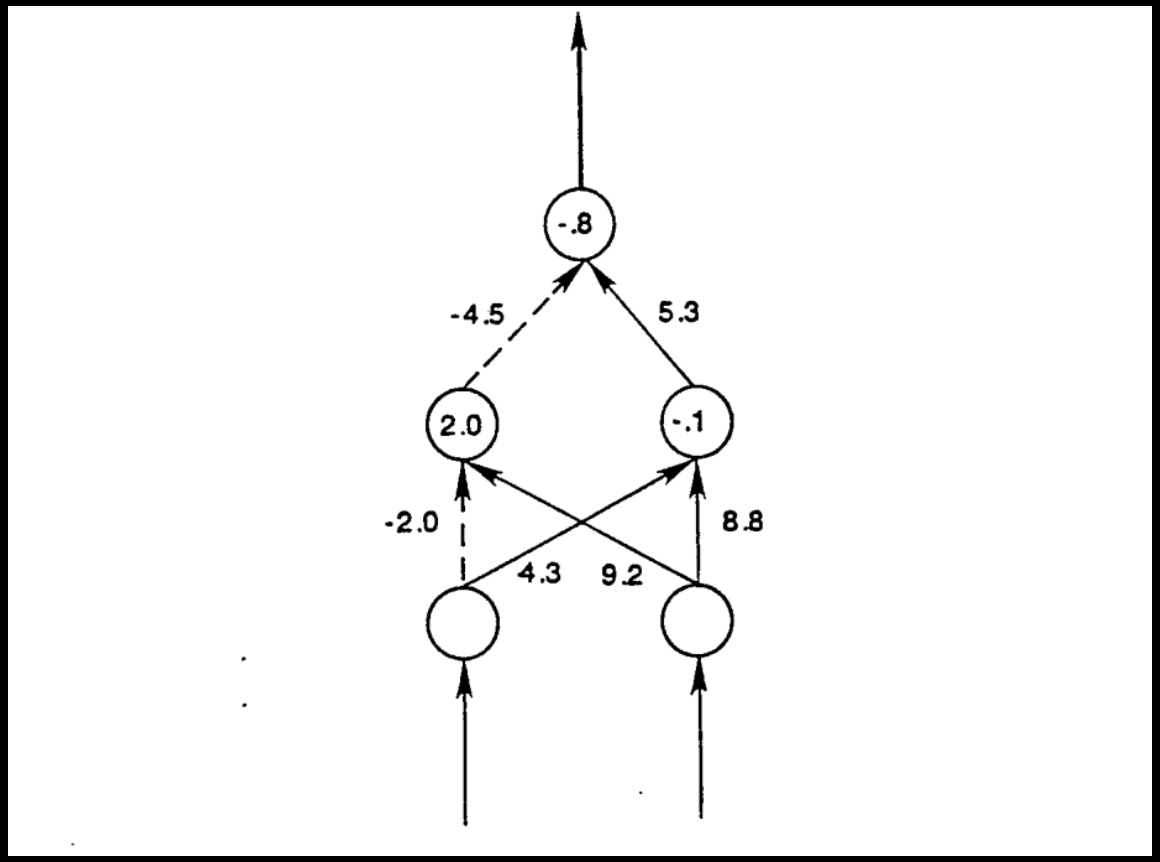


Example: Completed Training Example For…

- Binary classification: predict if a student will get a B or better (minimum required for CS graduate student in this course)

- Inputs:
    - Fraction of assignments completed
    - Fraction of readings read
    - Fraction of lectures watched

1986 Experiments: Neural Networks Learned!



e.g., Successful weights and biases learned for XOR problem

(and many more solved problems discussed in Ch. 8 of 1986 PDP book)

An Engaging Paper About Events Until this Point

A Sociological History of the Neural Network Controversy

MIKEL OLAZARAN

Universidad Pública de Navarra
Departamento de Sociología
Campus de Arrosadía
Pamplona, Spain

1. Introduction: A Sociological View of Scientific Controversies 335
2. The Controversy of the Perceptron 338
3. The Problems of Early Neural Networks 346
4. Training Multilayer Networks: A "Reverse Salient" of Neural Network Research 350
5. Interpretative Flexibility 355
6. Closure of the Controversy 1: Widrow's Group 368
7. Closure of the Controversy 2: The SRI Group 370
8. Closure of the Controversy 3: Rosenblatt 375
9. The 1980s: A Changing Context 386
10. History of Back-Propagation 390
11. Back-Propagation: Learning in Multilayer Perceptrons 396
12. The Neural Network Explosion 406
13. The Current Debate: Conclusions 411
    13.1 Debate Continues 411
    13.2 Conclusions 416

Advances in computers. 1993.

Again, An “AI Summer” Took Off

- Researchers gained interest and joined the field
  - e.g., authors on topic grew from 200 to 1,200 from 1986 to 1988 [Rappa & Debackere; 1989]
- Funding agencies gained interest (e.g., governments in US, Europe, Japan)

Again, Theory Supported the Movement

- Theoretical publications emerged in the late 1980s and early 1990s in support of the universal approximation theorem
  - e.g., Hornik, Stinchcombe and White. Multilayer feedforward networks are universal approximators. Neural Networks, 1989.

- "The universal approximation theorem means that regardless of what function we are trying to learn, we know that a large MLP [multilayer perceptron] will be able to *represent* this function."

  – Goodfellow, Bengio, and Courville; 2016 Deep Learning book

Today's Topics

- Vision: multi-layer networks provide sufficient representational power
- Breakthrough: gradients make multi-layer networks trainable
- Machine abstraction: neural networks as computational graphs
- Programming tutorial: developing neural networks in PyTorch

Historical Context

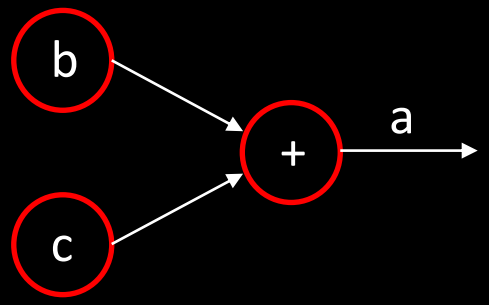
1960s and 1970s: the field of “automatic differentiation” emerged, revealing how to **efficiently compute derivatives** of complex functions

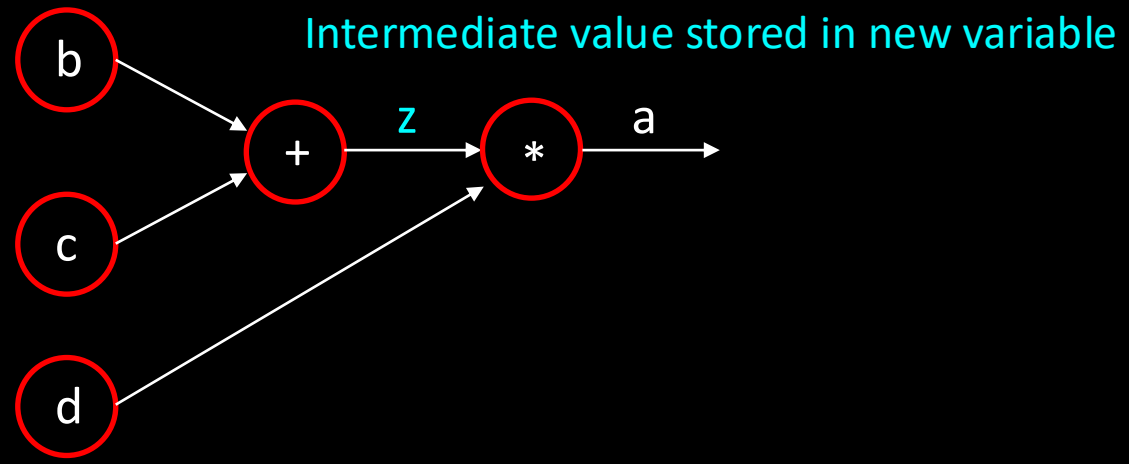
1980s: these ideas, now called reverse-mode differentiation, were repurposed to support “backpropagation” for training neural networks

Abstraction for Training Neural Networks: Computational Graphs

- A directed graph expressing a mathematical expression
  - Nodes: either variables (e.g., input values) or functions (applied to the in-flowing edges)
  - Edges: values output from functions

- For example: a = b + c OR a = d*(b + c)

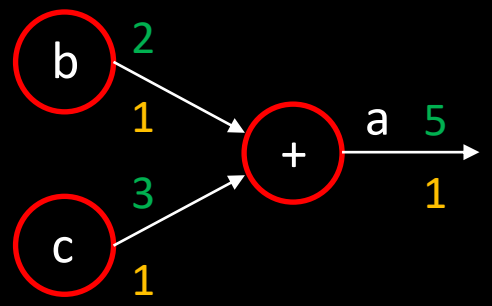




How to Use a Computational Graph?

- Forward pass: to evaluate an expression, set input variables and then perform all sequential computational steps in the graph
- Backward pass: compute local gradient at each edge and then combine all relevant gradients for each function variable
- For example, using b=2 and c=3:

f(b,c) = b + c



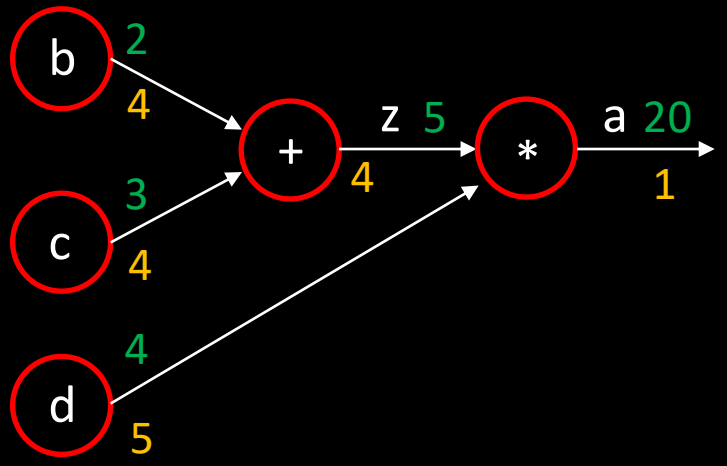
From calculus, we know:

- *df/df = 1*
- *df/db = 1*
- *df/dc = 1*

How to Use a Computational Graph?

- Forward pass: to evaluate an expression, set input variables and then perform all sequential computational steps in the graph
- Backward pass: compute local gradient at each edge and then combine all relevant gradients for each function variable
- For example, using b=2, c=3, and d=4:

f(b,c,d) = d*(b + c)



From calculus, we know:

- *df/df = 1*
- *df/dz = d*
- *df/dd = z*
- *df/db = df/dz * dz/db*
- *dz/db = 1*
- *df/dc = df/dz * dz/dc*
- *df/dc = 1*

Addition passes the incoming gradient while multiplication passes the other value through

Motivation for Computational Graphs

- Many deep learning frameworks rely on computational graphs; why?
  - **Efficient computation:** derivatives can be calculated based on "local" derivatives of the primitive functions in the graph, which enables them to be computed in parallel and stored for later reuse
  - **Efficient representation**: modular, primitive functions can be reused and combined to build complex models

Today's Topics

- Vision: multi-layer networks provide sufficient representational power
- Breakthrough: gradients make multi-layer networks trainable
- Machine abstraction: neural networks as computational graphs
- Programming tutorial: developing neural networks in PyTorch

Today's Topics

- Vision: multi-layer networks provide sufficient representational power
- Breakthrough: gradients make multi-layer networks trainable
- Machine abstraction: neural networks as computational graphs
- Programming tutorial: developing neural networks in PyTorch

The End